

Министерство просвещения Республики Башкортостан
Государственное автономное профессиональное образовательное
учреждение

Уфимский колледж статистики, информатики и вычислительной техники

	УТВЕРЖДАЮ Заместитель директора по учебной работе _____ З.З. Курмашева «__» _____ 2026 г.
--	--

**РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ МОНИТОРИНГА
ФИЗИЧЕСКОЙ АКТИВНОСТИ И ПИТАНИЯ ПОЛЬЗОВАТЕЛЕЙ**

Пояснительная записка к дипломному проекту

Рецензент _____ Д. Х. Каллимулин «__» _____ 2026 г.	Руководитель _____ Л.Р. Набиева «__» _____ 2026 г.
	Выпускник гр. 22П-2 _____ У.Ю. Хуснияров «__» _____ 2026 г.

2026

АННОТАЦИЯ

Пояснительная записка к дипломному проекту содержит постановку задачи и программную реализацию информационной системы Liga Gym для мониторинга физической активности, питания, шаговой активности, персональных целей и взаимодействия пользователя с тренером.

Программный продукт разработан с использованием Flutter и языка Dart, Firebase Authentication, Cloud Firestore, SQLite, Supabase Storage, Riverpod, фонового сервиса шагомера, геолокации и AI-сервиса для фитнес-консультаций. Приложение предназначено для работы на мобильных устройствах и отлажено на данных контрольного примера.

В работе спроектирована структура данных, реализованы регистрация и авторизация, заполнение профиля, учет тренировок и маршрутов, дневник питания, расчет КБЖУ, поиск продуктов по штрихкоду, шагомер, аналитика целей, экспорт отчета в PDF, AI-помощник, список друзей, чат, рейтинг активности и тренерские материалы.

					40.X-1441-2026 09.02.07 ДП-ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Хуснияров У. Ю.			Разработка информационной системы мониторинга физической активности и питания пользователей	Лит.	Лист	Листов
Провер.		Набиева Л. Р.					2	87
Реценз.		Каллимулин Д. Х.				УКСИВТ 22П-2		
Н. Контр.		Каримова Р.Ф.						
Утверд.		Курмашева З.З.						

СОДЕРЖАНИЕ

	ЛИСТ
Введение	4
1 Постановка задачи	6
1.1 Описание предметной области	6
1.2 Проектирование бизнес-процессов предметной области	11
1.3 Описание входной информации	12
1.4 Описание выходной информации	14
1.5 Общие требования к программному продукту	17
1.6 Описание структуры базы данных	18
1.7 Контрольный пример	22
2 Экспериментальный раздел	28
2.1 Описание программы	28
2.2 Протокол тестирования программного продукта	48
2.3 Руководство пользователя	59
Заключение	85
Список использованных источников	86
Приложение А	89

ВВЕДЕНИЕ

Актуальность темы определяется тем, что современные пользователи нуждаются в единой цифровой среде для контроля физической активности, питания и динамики достижения персональных целей. Разрозненное ведение тренировок, дневника питания, шагов и веса затрудняет анализ состояния пользователя, поэтому разработка мобильной информационной системы мониторинга физической активности и питания имеет практическую значимость.

Целью дипломного проекта является разработка информационной системы Liga Gym, обеспечивающей регистрацию и авторизацию пользователя, заполнение профиля, настройку целей, учет тренировок и маршрутов, ведение дневника питания, расчет калорийности и БЖУ, фиксацию шаговой активности, анализ прогресса, формирование PDF-отчета, социальное взаимодействие, работу с тренером и получение фитнес-рекомендаций через AI-помощника.

Для достижения цели требуется решить следующие задачи: изучить предметную область мониторинга активности и питания; определить входные и выходные данные; спроектировать структуру хранения данных; разработать мобильное приложение на Flutter и Dart; реализовать интеграцию с Firebase Authentication, Cloud Firestore, SQLite, Supabase Storage и Riverpod; выполнить тестирование основных пользовательских сценариев.

Объектом автоматизации является процесс персонального учета физической активности, питания и тренировочных результатов пользователя. Предметом разработки является мобильное приложение Liga Gym, в котором реализованы интерфейсные формы, сервисы хранения данных, локальная работа, синхронизация с облаком, аналитика и пользовательские сценарии фитнес-дневника.

Информационная система — это взаимосвязанная совокупность технических средств, программного обеспечения, данных, процедур и персонала, предназначенная для сбора, хранения, обработки и выдачи информации с целью удовлетворения информационных потребностей

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		4

пользователей и поддержки принятия управленческих или рабочих решений. Такая система обеспечивает сбор, хранение, обработку, передачу и представление данных, необходимых для принятия решений и повышения эффективности деятельности организации.

Тема дипломного проекта – разработка информационной системы мониторинга физической активности, питания и персонального прогресса пользователей с использованием современных мобильных, облачных и AI-технологий.

Целью работы является разработка и тестирование мобильного приложения Liga Gym, обеспечивающего учет тренировок, питания, шаговой активности, целей пользователя, аналитики прогресса и взаимодействия с друзьями или тренером.

Актуальность работы обусловлена ростом интереса пользователей к цифровым инструментам самоконтроля здоровья, тренировок и питания. Мобильное приложение позволяет объединить учет спортивной активности, расчет пищевой ценности продуктов и хранение персональной статистики в единой информационной системе.

Практическая значимость проекта заключается в возможности применения приложения как основы для персонального фитнес-дневника, учебного программного продукта или расширяемого сервиса для сопровождения тренировочного процесса.

Разрабатываемая система ориентирована на пользователя мобильного приложения и поддерживает регистрацию, авторизацию, заполнение профиля, настройку целей, запуск тренировки, просмотр истории активности, ведение дневника питания, расчет КБЖУ, подсчет шагов, аналитику прогресса, AI-рекомендации, социальное взаимодействие и получение материалов от тренера.

Для достижения цели предусмотрено решение следующих задач:

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

- анализ предметной области и требований к системе мониторинга физической активности, питания, шаговой активности и персональных целей пользователей;
- проектирование структуры данных для хранения профиля, целей, веса, тренировок, маршрутов, питания, шагов, друзей, чатов и тренерских материалов;
- разработка мобильного приложения на Flutter с использованием модульной архитектуры и библиотеки Riverpod;
- интеграция Firebase Authentication, Cloud Firestore, Supabase Storage, SQLite, фонового шагомера и AI-сервиса фитнес-помощника;
- тестирование сценариев регистрации, авторизации, питания, профиля, тренировки, шагомера, целей, аналитики, AI-помощника, друзей, чата и тренерских материалов.

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

1 Постановка задачи

1.1 Описание предметной области

Таблица 1.А – Основные сущности предметной области Liga Gym

Сущность	Назначение	Основные атрибуты
Пользователь	Учетная запись и профиль владельца данных	email, name, gender, birthDate, city, role, friendCode
Цели пользователя	Параметры контроля прогресса	goalType, dailyStepGoal, dailyCalorieGoal, targetWeightKg
История веса	Фиксация динамики массы тела	recordedAt, weightKg
Тренировка	Сохраненная спортивная активность	type, startedAt, endedAt, duration, calories, distanceMeters, route
Точка маршрута	Координата движения при тренировке	latitude, longitude, recordedAt, speedMetersPerSecond
Запись питания	Факт приема продукта за выбранную дату	mealType, productNameRu, grams, calories, proteins, fats, carbs
Продукт	Справочная или сохраненная карточка продукта	name_ru, barcode, calories, proteins, fats, carbs
Шаговая активность	Итог шагов пользователя за дату	user_id, date_key, steps, updated_at
Друг и заявка в друзья	Социальная связь пользователей	fromUserId, toUserId, status, displayName
Чат и сообщение	Обмен сообщениями и результатами	title, memberCount, senderId, message, sentAt, type
Лидерборд	Рейтинг активности с учетом приватности	score, workoutsCount, caloriesBurned, stepsCount
Тренерский материал	Упражнения, рецепты и шаблоны тренировок	title, description, media, exerciseIds, scheduledAt

Представленные сущности соответствуют фактической структуре проекта: модели профиля находятся в модуле auth, тренировки и маршруты - в модуле workout, питание - в модуле nutrition, шаги - в модуле steps, социальные функции - в модуле social, тренерские материалы - в модуле coach.

Необходимо разработать информационную систему для учета физической активности и питания пользователей мобильного приложения Liga Gym. Система должна обеспечивать регистрацию и авторизацию пользователей, ведение персонального профиля, запуск и сохранение тренировок, контроль рациона питания, а также синхронизацию данных между локальным хранилищем мобильного устройства и облачной базой данных. Разрабатываемая система ориентирована на повышение удобства контроля спортивной активности, анализа результатов тренировок и соблюдения режима питания.

В системе реализована модель пользователя, включающая учетные и персональные данные. Для каждого пользователя хранится следующая информация:

- идентификатор пользователя;
- адрес электронной почты;
- имя;
- пол;
- дата рождения;
- дата создания профиля;
- дата обновления профиля.

Адрес электронной почты пользователя должен быть уникальным. Учетные данные пользователя используются для регистрации и авторизации в системе.

Основной сущностью подсистемы тренировок является тренировка. Для каждой тренировки хранится следующая информация:

- идентификатор тренировки;
- идентификатор пользователя;
- тип тренировки;
- дата и время начала;
- дата и время окончания;
- продолжительность;
- количество затраченных калорий;
- пройденная дистанция;
- маршрут тренировки;
- признак синхронизации.

Тип тренировки может принимать следующие значения:

- бег;
- велотренировка;
- ходьба;

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

- силовая тренировка;
- кардио.

Для построения маршрута тренировки используются точки маршрута. Для каждой точки маршрута хранится следующая информация:

- широта;
- долгота;
- дата и время фиксации координаты.

Для учета питания в системе используется каталог продуктов. Для каждого продукта хранится следующая информация:

- идентификатор продукта;
- штрихкод;
- наименование продукта;
- калорийность на 100 граммов;
- белки на 100 граммов;
- жиры на 100 граммов;
- углеводы на 100 граммов.

Для ведения дневника питания используется запись о приеме пищи. Для каждой записи хранится следующая информация:

- идентификатор записи;
- идентификатор пользователя;
- тип приема пищи;
- наименование продукта;
- штрихкод;
- масса порции;
- калорийность;
- белки;
- жиры;
- углеводы;
- дата и время добавления;

- способ ввода данных;
- признак синхронизации.

Тип приема пищи может принимать следующие значения:

- завтрак;
- обед;
- ужин;
- перекус.

Способ ввода данных может быть следующим:

- ручной ввод;
- ввод по штрихкоду.

На основании записей о приеме пищи формируется дневник питания за выбранную дату. Дневник содержит перечень продуктов по приемам пищи и суммарные показатели калорий, белков, жиров и углеводов за день.

Система обеспечивает локальное хранение данных на мобильном устройстве и их последующую синхронизацию с облачным хранилищем. Это позволяет сохранять работоспособность приложения даже при отсутствии подключения к сети.

При использовании данного приложения целесообразно реализовать следующие выходные документы:

- карточка завершенной тренировки;
- история тренировок;
- дневник питания за день;
- сводная статистика по питанию и активности.

Карточка завершенной тренировки нужна пользователю для просмотра результата сразу после окончания занятия. Она содержит сведения о длительности тренировки, пройденной дистанции и затраченных калориях.

История тренировок нужна пользователю для анализа своей физической активности за определенный период. Она позволяет просматривать ранее

сохраненные тренировки, фильтровать их по дате и типу, а также отслеживать прогресс.

Дневник питания за день нужен пользователю для контроля рациона. Он содержит список добавленных продуктов по приемам пищи и итоговые показатели КБЖУ за сутки.

Сводная статистика по питанию и активности нужна пользователю для быстрой оценки текущего состояния: сколько калорий потреблено, сколько белков, жиров и углеводов получено, а также какие результаты достигнуты в тренировках.

К входным данным относятся:

- анкета пользователя;
- данные тренировки;
- маршрут тренировки;
- запись дневника питания;
- каталог продуктов питания;

К выходным данным относятся:

- карточка результата тренировки;
- история тренировок;
- дневник питания за выбранную дату;
- сводная статистика по активности и питанию.

С данной информационной системой будет работать следующий пользователь:

- пользователь системы.

Пользователь имеет следующие возможности:

- регистрация и авторизация в системе;
- заполнение и редактирование профиля;
- запуск, приостановка и завершение тренировки;
- сохранение маршрута тренировки;
- просмотр истории тренировок;

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						11
Изм.	Лист	№ докум.	Подпись	Дата		

- добавление продуктов в дневник питания;
- расчет КБЖУ;
- синхронизация данных с облачным хранилищем.

В системе предусмотрены следующие ограничения:

- пользователь должен быть зарегистрирован и авторизован для работы с системой;
- пользователь может получать доступ только к собственным данным;
- адрес электронной почты должен быть уникальным;
- для записи маршрута тренировки требуется доступ к геолокации;
- при отсутствии интернета данные сохраняются локально, а синхронизация выполняется позднее.

Дополнительно система содержит подсистему персональных целей пользователя. Для каждого профиля сохраняются город, код друга, рост, начальный вес, текущий вес, целевой вес, тип цели, дневная цель по шагам и дневная цель по калориям. Эти данные используются при построении аналитики и рекомендаций.

В системе предусмотрена подсистема шагомера. Она получает данные датчика физической активности, сохраняет дневное количество шагов, сравнивает результат с целью пользователя и отображает прогресс на отдельном экране.

Для расширения взаимодействия пользователей реализованы социальные функции: создание приглашения по коду друга, отправка и принятие заявок, список друзей, персональный чат и таблица активности друзей с учетом настроек приватности.

Для пользователей с ролью тренера предусмотрен отдельный режим работы. Тренер может приглашать учеников, создавать упражнения, рецепты, шаблоны тренировок, прикреплять медиафайлы и назначать материалы конкретным ученикам.

AI-помощник Liga Gym анализирует дневник питания, цели пользователя и показатели активности, после чего формирует краткие рекомендации по питанию и тренировкам. Помощник ограничен тематикой фитнеса и не заменяет медицинскую консультацию.

К основным ограничениям дополнительно относятся: доступ к персональным данным разрешается только владельцу профиля и связанному тренеру; пользователь управляет видимостью социальных показателей; для шагомера требуется разрешение на физическую активность; AI-помощник не принимает решения медицинского характера; материалы тренера доступны только связанным ученикам.

1.2 Проектирование бизнес-процессов предметной области

Таблица 1.Б – Бизнес-процессы предметной области

Процесс	Входное действие	Результат обработки
Регистрация и профиль	Пользователь вводит email, пароль и персональные данные	Создается учетная запись Firebase Auth и документ users
Настройка целей	Пользователь задает шаги, калории и целевой вес	Параметры используются на панели управления и в аналитике
Запись тренировки	Пользователь выбирает тип активности, запускает сессию и завершает ее	Сохраняются длительность, калории, дистанция и маршрут
Учет питания	Пользователь выбирает продукт вручную или по штрихкоду	Система рассчитывает КБЖУ по массе порции
Шаговая активность	Сервис получает показания датчика шагов	Локально сохраняется дневной итог шагов
Аналитика	Система агрегирует питание, тренировки, шаги и вес	Пользователь видит прогресс и может сформировать PDF-отчет
Социальные функции	Пользователь отправляет заявки, общается в чате и смотрит рейтинг	Доступ к данным ограничивается настройками приватности
Работа тренера	Тренер создает упражнения, рецепты и шаблоны	Материалы назначаются ученикам и доступны в приложении

Диаграмма прецедентов (или диаграмма вариантов использования, англ. use-case diagram) в UML – это графическое представление отношений между акторами (ролями пользователей или внешних систем) и прецедентами (функциями системы), показывающее, какие действия система предоставляет

Для проектирования программного продукта использован язык UML. На рисунке 1.1 представлена диаграмма вариантов использования, отражающая основные сценарии взаимодействия пользователя с приложением Liga Gym.



Входная информация – это данные, которые поступают в систему, приложение или алгоритм для обработки, анализа и последующего использования. Это исходные сведения, необходимые для ведения учёта тренировок, контроля питания, сохранения пользовательских данных и синхронизации информации между локальным и облачным хранилищем.

<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>

Входными данными для создания системы являются:

- анкета пользователя;
- данные тренировки;
- маршрут тренировки;
- запись дневника питания;
- каталог продуктов питания;
- штрихкод продукта.

Пользователь является основным источником входной информации и выполняет регистрацию, авторизацию, заполнение профиля, запуск тренировки и внесение записей о приёмах пищи.

Профиль пользователя характеризуется следующими параметрами:

- идентификатор пользователя;
- адрес электронной почты;
- имя;
- пол;
- дата рождения;
- дата создания профиля;
- дата обновления профиля.

Тренировка характеризуется следующими параметрами:

- идентификатор тренировки;
- идентификатор пользователя;
- тип тренировки;
- дата и время начала;
- дата и время окончания;
- продолжительность;
- количество затраченных калорий;
- пройденная дистанция;
- маршрут тренировки;

– признак синхронизации.

Запись дневника питания характеризуется следующими параметрами:

- идентификатор записи;
- идентификатор пользователя;
- тип приёма пищи;
- наименование продукта;
- штрихкод;
- масса порции;
- калорийность;
- белки;
- жиры;
- углеводы;
- дата и время добавления;
- способ ввода данных;
- признак синхронизации.

Описание входных документов отображено в таблице 1.1.

Таблица 1.1 – Описание входных документов

Наименование документа (шифр)	Периодичность поступления документа	Откуда поступает документ
Каталог продуктов питания	По мере обращения к функции поиска	Из встроенного каталога приложения
Настройки целей пользователя	При изменении профиля или целей	От пользователя
Показания шагомера	В течение дня	От датчика физической активности
История веса	При обновлении профиля	От пользователя
Заявка в друзья / код приглашения	По запросу пользователя	От пользователя
Сообщение чата	При отправке сообщения	От пользователя
Запрос к AI-помощнику	По запросу пользователя	От пользователя
Тренерский материал	При создании или назначении	От тренера

К новым входным данным также относятся: настройки целей пользователя, история веса, показания шагомера, код друга или ссылка-приглашение, текст

сообщения в чате, запрос к AI-помощнику, данные тренерского материала, медиафайлы упражнения или рецепта.

1.4 Описание выходной информации

Выходной документ – это итоговый файл, экранная форма или отчет, созданный в результате обработки данных приложением. Он содержит готовую информацию, предназначенную для просмотра пользователем, анализа результатов, принятия решений и контроля показателей активности и питания.

Описание выходных документов отображено в таблице 1.2.

Таблица 1.2 – Описание выходных документов

Наименование документа (шифр)	Периодичность выдачи документа	Кол-во экз.	Куда передаются
Карточка результата тренировки	По завершении тренировки	1	Пользователю
История тренировок	По запросу пользователя	1	Пользователю
Дневник питания за выбранную дату	По запросу пользователя	1	Пользователю
Сводная статистика по активности и питанию	По мере необходимости	1	Пользователю
Дневной прогресс пользователя	При открытии панели управления	1	Пользователю
Недельная аналитика	По запросу пользователя	1	Пользователю
PDF-отчет активности	По выбранному периоду	1	Пользователю
Ответ AI-помощника	После отправки вопроса	1	Пользователю
Список друзей и рейтинг	По запросу пользователя	1	Пользователю
Сообщения чата	При открытии диалога	1	Участникам чата
Материалы тренера	При открытии раздела тренера	1	Ученику

Шаблоны выходных документов представлены на рисунках 1.2 – 4.4.

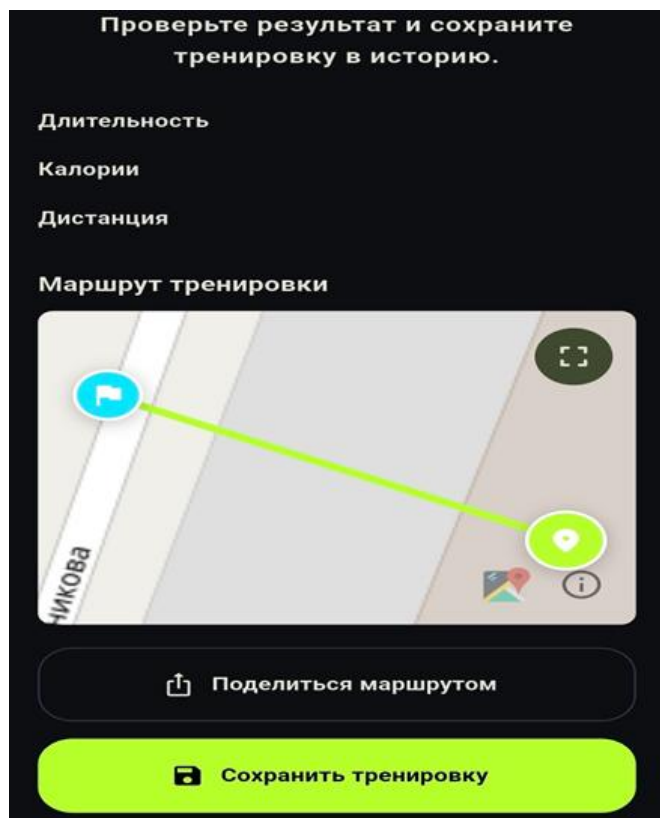


Рисунок 1.2 – Шаблон выходного документа «Карточка результата тренировки»

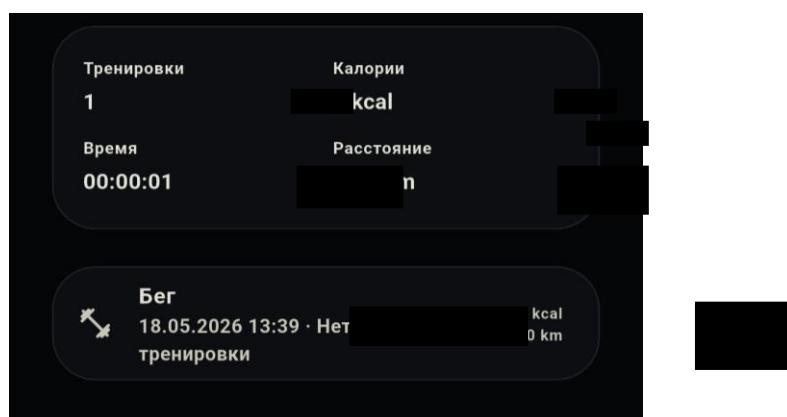


Рисунок 1.3 – Шаблон выходного документа «История тренировок»

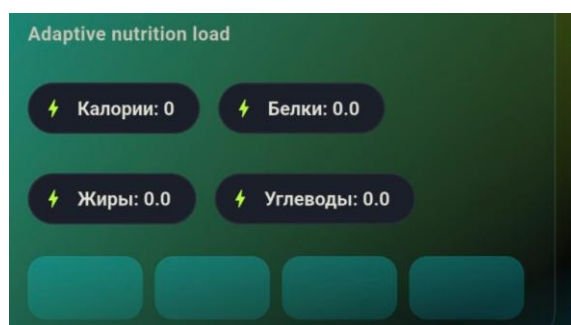


Рисунок 1.4 – Шаблон выходного документа «Дневник питания за выбранную дату»

Liga Gym — отчёт по аналитике

Период:

Сводка

- За неделю шагов
- За неделю ккал
- В среднем шагов в день
- В среднем ккал в день
- На тренировках сожжено ккал
- Выполнено тренировок:
- Текущий вес кг
- Прогресс по весу кг

Результаты по дням

Дата	Шаги	Калории	Прогресс

Рисунок 1.5 – Шаблон выходного документа «Сводная статистика по активности и питанию»

К выходным данным дополнительно относятся: карточка дневного прогресса, недельная аналитика активности и питания, PDF-отчет за выбранный период, результат работы AI-помощника, список друзей, сообщения чата, рейтинг активности, список материалов тренера и назначенные тренировки или рецепты.

1.5 Общие требования к программному продукту

Таблица 1.В – Общие требования к программному продукту Liga Gym

Группа требований	Содержание требования
Функциональные	Регистрация, вход, профиль, цели, тренировки, питание, шаги, аналитика, PDF, AI, друзья, чат, лидерборд, тренерские материалы
Надежность	Обработка ошибок Firebase Auth, сетевых тайм-аутов, локальное сохранение записей и последующая синхронизация
Данные	Хранение пользовательских данных в Cloud Firestore, локальных записей в SQLite, настроек в Shared Preferences
Безопасность	Разграничение доступа правилами Firestore, учет владельца документа userId и настройка приватности социальных данных
Интерфейс	Русская и английская локализация, адаптивные экраны Flutter, единая навигация через AppRoutes
Производительность	Локальные выборки SQLite, ограничение сетевых запросов по тайм-ауту, фоновая обработка шагомера

Группа требований	Содержание требования
Переносимость	Проект содержит конфигурации Android, iOS, Web, Windows, macOS и Linux, основная целевая среда - мобильное приложение

К программному продукту предъявляются требования надежности, удобства использования, конфиденциальности пользовательских данных и сохранности информации при работе без постоянного подключения к сети.

Приложение должно обеспечивать регистрацию и авторизацию пользователя, заполнение персонального профиля, запуск и завершение тренировки, сохранение маршрута, ведение дневника питания, расчет калорий и БЖУ, а также синхронизацию данных с облачным хранилищем.

Таблица 1.3 – Компоненты и технологии программного продукта

Компонент	Технология
Клиентское приложение	Flutter, Dart
Облачные сервисы	Firebase Authentication, Cloud Firestore
Локальное хранение	SQLite, Sqflite
Целевые платформы	Android, iOS
Управление состоянием	Riverpod, Riverpod Generator
Шагомер	Pedometer, Flutter Background Service
Геолокация и карта	Geolocator, Flutter Map, LatLong2
Аналитика и графики	fl_chart, PDF, Share Plus
AI-помощник	HTTP API Pollinations AI
Социальные функции	Cloud Firestore, QR Flutter
Медиа тренера	Supabase Storage, Image Picker, Video Player
Локальные настройки	Shared Preferences

Требования к программному продукту: надежность работы; удобный интерфейс; корректная обработка некорректных данных; разграничение доступа к персональной информации; сохранность локальных и облачных данных.

1.6 Описание структуры базы данных

Таблица 1.Г – Коллекция users

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор пользователя	userId	String	Ключ документа, соответствует Firebase Auth uid
Электронная почта	email	String	Используется при входе и отображении профиля
Имя	name	String	Отображаемое имя пользователя
Пол	gender	String	male/female из перечисления Gender

Содержание поля	Имя поля	Тип, длина	Примечания
Дата рождения	birthDate	Timestamp	Используется при заполнении профиля
Город	city	String	Дополнительный параметр профиля
Код друга	friendCode	String	Используется для приглашений
Рост	heightCm	Number	Рост пользователя в сантиметрах
Начальный вес	startWeightKg	Number	Исходная масса тела
Текущий вес	currentWeightKg	Number	Актуальный показатель
Целевой вес	targetWeightKg	Number	Используется в аналитике веса
Тип цели	goalType	String	loseWeight, maintainWeight, gainWeight
Цель по шагам	dailyStepGoal	Number	По умолчанию 10000
Цель по калориям	dailyCalorieGoal	Number	По умолчанию 2200
Роль	role	String	student или trainer

Таблица 1.Д – Подколлекция users/{userId}/weight_history

Содержание поля	Имя поля	Тип, длина	Примечания
Дата записи	recordedAt	Timestamp	Время фиксации веса
Вес	weightKg	Number	Масса тела в килограммах
Дата обновления	updatedAt	Timestamp	Служебная отметка изменения

Таблица 1.Е – Подколлекция users/{userId}/food_entries

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор	id	String	Ключ записи питания
Прием пищи	meal_type	String	breakfast, lunch, dinner, snack
Дата	date_key	String	Ключ дня в формате YYYY-MM-DD
Название EN	product_name_en	String	Английское название продукта
Название RU	product_name_ru	String	Русское название продукта
Штрихкод	barcode	String	Заполняется при поиске по штрихкоду
Масса	grams	Number	Размер порции в граммах
Калории	calories	Number	Расчетная калорийность порции
Белки	proteins	Number	Расчетное количество белков
Жиры	fats	Number	Расчетное количество жиров
Углеводы	carbs	Number	Расчетное количество углеводов
Дата ввода	logged_at	Timestamp	Время добавления записи
Способ ввода	input_method	String	manual или barcode

Таблица 1.Ж – Подколлекция users/{userId}/workouts

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор	id	String	Ключ тренировки
Пользователь	userId	String	Владелец записи

Содержание поля	Имя поля	Тип, длина	Примечания
Тип тренировки	type	String	Тип из перечисления WorkoutType
Начало	startedAt	String	Дата и время начала ISO-8601
Окончание	endedAt	String	Дата и время окончания ISO-8601
Длительность	durationSeconds	Number	Продолжительность в секундах
Калории	calories	Number	Расчетный расход калорий
Дистанция	distanceMeters	Number	Расстояние в метрах
Маршрут	routeJson	String	JSON-массив точек маршрута
Название	title	String	Пользовательское название тренировки
Заметка	note	String	Комментарий пользователя
Место	place	String	Место выполнения тренировки
Упражнения	exerciseEntriesJson	String	JSON-массив упражнений
Ручной ввод	isManual	Boolean	Признак ручного добавления

Таблица 1.И – Локальные таблицы SQLite

Содержание поля	Имя поля	Тип, длина	Примечания
Записи питания	liga_gym_nutrition.db / food_entries	SQLite	Локальный дневник питания и флаг is_synced
Тренировки	liga_gym_workouts.db / workouts	SQLite	Кэш тренировок, маршрута и упражнений
Дневные шаги	liga_gym_steps.db / daily_steps	SQLite	Итоги шагов по user_id и date_key
Состояние датчика	liga_gym_steps.db / step_sensor_state	SQLite	Последние показания датчика шагов

Таблица 1.К – Социальные коллекции Firestore

Содержание поля	Имя поля	Тип, длина	Примечания
Лидерборд	leaderboard_entries	Collection	score, workoutsCount, caloriesBurned, stepsCount
Заявки в друзья	friend_requests	Collection	fromUserId, toUserId, status, createdAt
Приглашения	friend_invites	Collection	Код друга и владелец приглашения
Друзья	users/{userId}/friends	Subcollection	Профиль друга и доступные категории
Настройки приватности	users/{userId}/social_settings/privacy	Document	allowedCategories, видимость рейтинга
Чаты	interest_chats	Collection	title, description, memberCount, createdAt
Сообщения	interest_chats/{chatId}/messages	Subcollection	senderId, message, sentAt, type

Таблица 1.Л – Тренерские коллекции Firestore и Supabase Storage

Содержание поля	Имя поля	Тип, длина	Примечания
Тренеры	trainers	Collection	Документ тренера и его материалы
Запросы тренеру	coach_requests	Collection	trainerId, studentId, status, createdAt
Связи ученика	users/{studentId}/coach_links	Subcollection	Связь ученика с тренером
Упражнения	trainers/{trainerId}/exercises	Subcollection	title, description, media, muscleGroups
Рецепты	trainers/{trainerId}/recipes	Subcollection	ingredientsText, calories, proteins, fats, carbs
Шаблоны тренировок	trainers/{trainerId}/workout_templates	Subcollection	goal, instructions, exerciseIds
Назначенные рецепты	users/{studentId}/recipe_assignments	Subcollection	recipeId, trainerId, status
Назначенные тренировки	users/{studentId}/workout_assignments	Subcollection	templateId, scheduledAt, status
Медиафайлы	coach-media	Supabase Storage	Изображения и видео тренерских материалов

При проектировании структуры хранения данных приложения Liga Gym была выбрана гибридная модель хранения, включающая облачное хранилище Firebase Firestore и локальную базу данных SQLite, используемую через библиотеку sqflite. Firebase Firestore обеспечивает хранение профиля пользователя, записей дневника питания и тренировок с возможностью синхронизации между устройствами, а SQLite применяется для автономной работы приложения при отсутствии сетевого соединения.

Для управления аутентификацией используется сервис Firebase Authentication. Учетные данные авторизации хранятся в отдельном сервисе Firebase и не входят в состав описываемой структуры Firestore и SQLite. В рассматриваемой схеме приводятся только те данные, с которыми непосредственно работает клиентское приложение.

Подробная информация о полях представлена в таблицах 1.4 – 1.9.

Таблица 1.4 – Users (Пользователи)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор пользователя	user_id	string	Первичный ключ, идентификатор документа
Адрес электронной почты	email	string	Обязательное поле
Имя	name	string	Обязательное поле
Пол	gender	string	Обязательное поле
Дата рождения	birthDate	timestamp	Обязательное поле
Дата создания	createdAt	timestamp	Серверное время создания
Дата обновления	updatedAt	timestamp	Серверное время обновления

Таблица 1.5 – Food_Entries (Записи дневника питания в облаке)

Содержание поля	Имя поля	Тип, длина	Примечания
1	2	3	4
Идентификатор записи	id	string	Идентификатор документа
Тип приема пищи	meal_type	string	Обязательное поле
Ключ даты	date_key	string	Используется для выборки записей за дату
Название продукта (EN)	product_name_en	string	Обязательное поле
Название продукта (RU)	product_name_ru	string	Обязательное поле
Штрихкод	barcode	string	Необязательное поле
Вес порции	grams	double	Обязательное поле
Калорийность	calories	double	Обязательное поле
Белки	proteins	double	Обязательное поле
Жиры	fats	double	Обязательное поле
Углеводы	carbs	double	Обязательное поле
Дата и время записи	logged_at	timestamp	Обязательное поле
Способ ввода	input_method	string	manual / barcode

Таблица 1.6 – Workouts (Тренировки)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор тренировки	id	string	Идентификатор документа
Пользователь	userId	string	Идентификатор владельца тренировки
Тип тренировки	type	string	running / cycling / walking / strength / cardio
Дата и время начала	startedAt	string	Формат ISO 8601
Дата и время окончания	endedAt	string	Формат ISO 8601

Содержание поля	Имя поля	Тип, длина	Примечания
Продолжительность	durationSeconds	integer	В секундах
Калории	calories	double	Обязательное поле
Дистанция	distanceMeters	double	В метрах
Маршрут	routeJson	text	JSON-массив точек маршрута
Признак синхронизации	isSynced	boolean	Для синхронизированных записей принимает значение true

Таблица 1.7 – Food_Entries_Local (Сохранённая еда)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор записи	id	TEXT	Первичный ключ
Пользователь	user_id	TEXT	Обязательное поле
Тип приема пищи	meal_type	TEXT	Обязательное поле
Ключ даты	date_key	TEXT	Используется для выборки по дате
Название продукта (EN)	product_name_en	TEXT	Обязательное поле
Название продукта (RU)	product_name_ru	TEXT	Обязательное поле
Штрихкод	barcode	TEXT	Необязательное поле
Вес порции	grams	REAL	Обязательное поле
Калорийность	calories	REAL	Обязательное поле
Белки	proteins	REAL	Обязательное поле
Жиры	fats	REAL	Обязательное поле
Углеводы	carbs	REAL	Обязательное поле
Дата и время записи	logged_at	INTEGER	Хранится в миллисекундах Unix time
Способ ввода	input_method	TEXT	manual / barcode
Признак синхронизации	is_synced	INTEGER	0 - не синхронизировано, 1 - синхронизировано

Таблица 1.8 – Workouts_Local (Локальные тренировки)

Содержание поля	Имя поля	Тип, длина	Примечания
1	2	3	4
Идентификатор тренировки	id	TEXT	Первичный ключ
Пользователь	user_id	TEXT	Обязательное поле
Тип тренировки	type	TEXT	Обязательное поле

Содержание поля	Имя поля	Тип, длина	Примечания
Дата и время начала	started_at	INTEGER	Хранится в миллисекундах Unix time
Дата и время окончания	ended_at	INTEGER	Хранится в миллисекундах Unix time

Продолжение таблицы 1.8

1	2	3	4
Продолжительность	duration_seconds	INTEGER	В секундах
Калории	calories	REAL	Обязательное поле
Дистанция	distance_meters	REAL	В метрах
Маршрут	route_json	TEXT	JSON-массив точек маршрута
Признак синхронизации	is_synced	INTEGER	0 - не синхронизировано, 1 - синхронизировано

На рисунке 1.6 изображена схема хранения данных приложения.



Рисунок 1.6 – Схема структуры данных приложения

Схема базы данных представляет собой формализованное описание структуры хранимой информации, включающее облачные коллекции, локальные таблицы, их поля и взаимосвязи. В гибридных системах хранения она определяет состав данных, правила синхронизации и способы обеспечения целостности информации.

В связи с расширением функциональности структура хранения дополнена коллекциями и локальными таблицами, обеспечивающими цели пользователя, шаговую активность, социальные функции и тренерский режим.

Таблица 1.9 – Дополнительные коллекции и локальные структуры данных

Наименование	Назначение	Основные поля
daily_metrics	Дневные показатели активности	userId, date, steps, calories, updatedAt
weight_history	История веса пользователя	weightKg, recordedAt
friend_invites	Приглашения в друзья	inviteCode, ownerUserId, createdAt
friend_requests	Заявки в друзья	fromUserId, toUserId, status
interest_chats/messages	Персональные чаты и сообщения	participantIds, senderId, message, sentAt
trainers/exercises	Библиотека упражнений тренера	title, muscleGroup, guideText, mediaUrls
trainers/recipes	Библиотека рецептов тренера	nameRu, macros, guideText, mediaUrls
workout_assignments	Назначенные тренировки	trainerId, studentId, scheduledAt, templateId
step_counts	Локальный учет шагов	user_id, date_key, sensor_steps, daily_steps

1.7 Контрольный пример

Таблица 1.М – Сводный контрольный пример работы Liga Gym

№	Этап	Входные данные	Ожидаемый результат
1	Регистрация	email, пароль, имя	Создан пользователь Firebase Auth и документ users
2	Заполнение профиля	пол, дата рождения, рост, вес, цель	Профиль доступен на экране профиля и панели управления
3	Настройка цели	10000 шагов, 2200 ккал, целевой вес	Показатели используются в DashboardGoalProgress
4	Запуск тренировки	тип тренировки, разрешение геолокации	Создана активная сессия WorkoutSessionController
5	Завершение тренировки	время, дистанция, маршрут	Запись сохранена в SQLite и синхронизирована в Firestore
6	Добавление питания	продукт, масса, прием пищи	Рассчитаны calories, proteins, fats, carbs
7	Шагомер	показания pedometer	В таблице daily_steps обновлен итог за день
8	Аналитика	тренировки, питание, шаги, вес	Отображены недельная статистика, прогресс и динамика веса
9	PDF-отчет	диапазон аналитики	Сформирован отчет и передан через share_plus
10	AI-помощник	вопрос о питании или тренировке	Ответ сформирован с учетом дневника, целей и контекста
11	Социальный сценарий	код друга, заявка, сообщение	Создана заявка, связь друзей и сообщение в чате
12	Тренерский сценарий	упражнение, рецепт, шаблон	Материал сохранен в Firestore, медиа - в Supabase Storage

Контрольный пример представляет собой заранее подготовленный набор данных, позволяющий проверить корректность работы мобильного приложения

Liga Gym, его вычислительных алгоритмов и сценариев сохранения данных.

Для приложения Liga Gym контрольные примеры используются при проверке авторизации, расчета параметров тренировки, ведения дневника питания, синхронизации с Cloud Firestore и записи данных в локальную SQLite-базу.

В таблице 1.10 представлены входные данные «Статус аутентификации» для контрольного примера.

Таблица 1.10 – Контрольный пример «Статус аутентификации»

Номер статуса аутентификации	Наименование
1	Не авторизован
2	Профиль не заполнен
3	Авторизован

В таблице 1.11 представлены входные данные «Тип тренировки» для контрольного примера.

Таблица 1.11 – Контрольный пример «Тип тренировки»

Номер типа тренировки	Наименование
1	Бег
2	Велоспорт
3	Пешком
4	Сила
5	Кардио

В таблице 1.12 представлены входные данные «Тип приема пищи» для контрольного примера.

Таблица 1.12 – Контрольный пример «Тип приема пищи»

Номер приема пищи	Наименование
1	Завтрак
2	Обед
3	Ужин

В таблице 1.13 представлены входные данные «Пользовательский профиль» для контрольного примера.

Таблица 1.13 – Контрольный пример «Пользовательский профиль»

Номер	Электронная почта	Имя	Пол	Дата рождения	Способ входа
1	2	3	4	5	6
1	ivan.petrov@gmail.com	Иван Петров	Мужской	14.08.2002	По электронной почте и паролю

Номер	Электронная почта	Имя	Пол	Дата рождения	Способ входа
2	alina.smirnova@gmail.com	Алина Смирнова	Женский	03.11.2000	По аккаунту гугл
3	oleg.runner@mail.ru	Олег Крылов	Мужской	21.02.1998	По электронной почте и паролю
4	maria.cardio@yandex.ru	Мария Белова	Женский	09.07.2004	По аккаунту гугл
5	timur.volkov@mail.ru	Тимур Волков	Мужской	17.01.2001	По электронной почте и паролю

Продолжение таблицы 1.12

Кэш тренировок	Кэш питания	Последняя активность	Примечание
7	8	9	10
3	5	25.05.2026 20:30	Пользователь с полной историей активности
1	4	25.05.2026 18:40	Чаще использует модуль питания
7	2	25.05.2026 07:55	Пользователь английской локали
2	6	25.05.2026 21:15	Регулярно сохраняет cardio-сессии
4	3	25.05.2026 19:05	Комбинированный сценарий тренировок и питания

В таблице 1.14 представлены входные данные «Маршрут приложения» для контрольного примера.

Таблица 1.14 – Контрольный пример «Маршрут приложения»

Номер маршрута	Маршрут	Назначение
1	/	Splash-экран и определение auth-состояния
2	вход	Вход по email и паролю
3	/регистрация	Создание учетной записи
4	/настройка профиля	Первичное заполнение профиля
5	/панель управления	Главный экран приложения
6	/питание/дневник	Дневник питания за выбранную дату

В таблице 1.15 представлены входные данные «Способ ввода записи питания» для контрольного примера.

Таблица 1.15 – Контрольный пример «Способ ввода записи питания»

Номер способа	Наименование
1	Ручной
2	Штрих-код

В таблице 1.16 представлены входные данные «Запись дневника питания» для контрольного примера.

Таблица 1.16 – Контрольный пример «Запись дневника питания»

Номер записи	Пользователь	Прием пищи	Продукт (RU)	Штрихкод
1	2	3	4	5
1	Иван Петров	завтрак	Овсяные хлопья	4601234567890
2	Иван Петров	завтрак	Греческий йогурт	4607002010012
3	Иван Петров	обед	Куриная грудка	4601111111111
4	Иван Петров	перекус	Протеиновый батончик	4820001234567
5	Иван Петров	ужин	Банан	1232131231231

Продолжение таблицы 1.15

Масса, г	Калории	Белки	Жиры	Углеводы	Дата и время
6	7	8	9	10	11
80	281.6	9.8	4.9	47.6	25.05.2026 08:10
150	109.5	15.0	3.0	5.7	25.05.2026 08:15
180	297.0	55.8	6.5	0.0	25.05.2026 13:05
60	216.0	15.0	7.2	21.0	25.05.2026 17:40
120	106.8	1.3	0.4	27.4	25.05.2026 20:25

В таблице 1.17 представлены входные данные «Поиск продукта по штрихкоду» для контрольного примера.

Таблица 1.17 – Контрольный пример «Поиск продукта по штрихкоду»

Номер запроса	Штрихкод	Пользователь
1	2	3
1	4601234567890	Иван Петров
2	4607002010012	Иван Петров
3	9999999999999	Иван Петров

Продолжение таблицы 1.16

Дата поиска	Результат
4	5
25.05.2026 08:09	Овсяные хлопья найдены
25.05.2026 08:14	Греческий йогурт найден
25.05.2026 20:24	Продукт не найден

В таблице 1.18 представлены входные данные «Способ входа» для контрольного примера.

Таблица 1.18 – Контрольный пример «Способ входа»

Номер способа	Наименование
1	По электронной почте и паролю
2	По аккаунту гугл

В таблице 1.19 представлены входные данные «Продукт» для контрольного примера

Таблица 1.19 – Контрольный пример «Продукт»

Номер продукта	Номер	Штрихкод	Название (RU)	Ккал и БЖУ / 100 г
1	греческий йогурт	4607002010012	Греческий йогурт	73 / 10.0 / 2.0 / 3.8
2	протеиновый батончик	4820001234567	Протеиновый батончик	360 / 25.0 / 12.0 / 35.0
3	овсянка	4601234567890	Овсяные хлопья	352 / 12.3 / 6.1 / 59.5
4	куриная грудка	4601111111111	Куриная грудка	165 / 31.0 / 3.6 / 0.0
5	банан, приготовленный вручную	4607002010014	Банан	89 / 1.1 / 0.3 / 22.8

В таблице 1.20 представлены входные данные «Продукт по приему пищи» для контрольного примера.

Таблица 1.20 – Контрольный пример «Продукт по приему пищи»

Номер записи	Прием пищи	Продукт
1	завтрак	Овсяные хлопья
2	завтрак	Греческий йогурт
3	обед	Куриная грудка
4	перекус	Протеиновый батончик
5	ужин	Банан

В таблице 1.21 представлены входные данные «Статус тренировочной сессии» для контрольного примера.

Таблица 1.21 – Контрольный пример «Статус тренировочной сессии»

Номер статуса	Наименование
1	бездействие
2	в процессе работы
3	приостановлено
4	завершено
5	сохранение

В таблице 1.22 представлены входные данные «Сессия тренировки» для контрольного примера.

Таблица 1.22 – Контрольный пример «Сессия тренировки»

Номер сессии	Пользователь	Тип тренировки
1	2	3
1	Иван Петров	бег
2	Иван Петров	ходьба
3	Тимур Волков	сила
4	Олег Крылов	велоспорт
5	Мария Белова	кардио

Продолжение таблицы 1.22

Дата и время старта	Длительность, мин	Дистанция, м	Статус
4	5	6	7
25.05.2026 18:00	32	4600	завершено

Дата и время старта	Длительность, мин	Дистанция, м	Статус
25.05.2026 19:10	45	5300	завершено
25.05.2026 08:30	12	0	приостановлено
25.05.2026 07:15	40	12500	завершено
25.05.2026 18:45	28	3100	завершено

В таблице 1.23 представлены выходные данные «Результат тренировки» для контрольного примера.

Таблица 1.23 – Контрольный пример «Результат тренировки»

Номер	Пользователь	Тип	Начало	Окончание	Длительность
1	2	3	4	5	6
1	Иван Петров	бег	25.05.2026 18:00	25.05.2026 18:32	0:32:00
2	Алина Смирнова	ходьба	25.05.2026 19:10	25.05.2026 19:55	0:45:00
3	Олег Крылов	езда на велосипеде	25.05.2026 07:15	25.05.2026 07:55	0:40:00
4	Мария Белова	кардио	25.05.2026 18:45	25.05.2026 19:13	0:28:00

Продолжение таблицы 1.22

Начало	Окончание	Длительность
7	8	9
25.05.2026 18:00	25.05.2026 18:32	0:32:00
25.05.2026 19:10	25.05.2026 19:55	0:45:00
25.05.2026 07:15	25.05.2026 07:55	0:40:00
25.05.2026 18:45	25.05.2026 19:13	0:28:00

В таблице 1.24 представлены выходные данные «Профиль пользователя» для контрольного примера.

Таблица 1.24 – Контрольный пример «Профиль пользователя»

Номер профиля	Пользователь
1	2
1	Иван Петров
2	Алина Смирнова
3	Мария Белова

Продолжение таблицы 1.23

Электронная почта	Пол	Статус профиля
3	4	5
ivan.petrov@gmail.com	мужчина	заполнен
alina.smirnova@gmail.com	женщина	заполнен
maria.cardio@yandex.ru	женщина	заполнен

В таблице 1.25 представлены выходные данные «История тренировок» для контрольного примера.

Таблица 1.25 – Контрольный пример «История тренировок»

Номер записи	Пользователь	Тип тренировки	Дата	Калории	Дистанция, м
1	Иван Петров	бег	25.05.2026	318.7	4600
2	Иван Петров	ходьба	25.05.2026	219.2	5300
3	Олег Крылов	езда на велосипеде	25.05.2026	305.0	12500
4	Мария Белова	кардио	25.05.2026	230.5	3100

В таблице 1.26 представлены выходные данные «Документ Записи о еде» для контрольного примера.

Таблица 1.26 – Контрольный пример «Документ Записи о еде»

Номер	Тип еды	Дата
1	2	3
food_1775981400000000_breakfast	Завтрак	25.05.2026
food_1775981700000000_breakfast	Завтрак	25.05.2026
food_1775999100000000_lunch	Ланч	25.05.2026
food_1776015600000000_snack	Закуска	25.05.2026
food_1776025500000000_dinner	Ужин	25.05.2026

Продолжение таблицы 1.25

Наименование продукта	Граммы	Входной метод	Регистрация
4	5	6	5
Овсяные хлопья	80	штрих-код	25.05.2026 08:10
Греческий йогурт	150	штрих-код	25.05.2026 08:15
Куриная грудка	180	штрих-код	25.05.2026 13:05
Протеиновый батончик	60	штрих-код	25.05.2026 17:40
Банан	120	руководство	25.05.2026 20:25

В таблице 1.27 представлены выходные данные «Сводка Панель управления за выбранную дату» для контрольного примера.

Таблица 1.27 – Контрольный пример «Сводка Панель управления за выбранную дату»

Номер сводки	Дата	Электронная почта пользователя	Калории, ккал	Белки, г	Жиры, г	Углеводы, г	Комментарий
1	25.05.2026	ivan.petrov@gmail.com	1010.9	96.9	22.0	101.7	Сводка дневника питания за выбранную дату

Номер сводки	Дата	Электронная почта пользователя	Калории, ккал	Белки, г	Жиры, г	Углеводы, г	Комментарий
2	25.05.2026	ivan.petrov@gmail.com	742.3	68.4	18.2	80.1	Сводка дневника питания за выбранную дату

2 Экспериментальный раздел

2.1 Описание программы

Таблица 2.А – Модульная структура проекта Liga Gym

Модуль	Назначение	Ключевые файлы и классы
core	Общие сервисы, тема, навигация, Firebase, Supabase, offline-sync	main.dart, app.dart, app_router.dart
auth	Регистрация, вход, профиль, история веса	FirebaseAuthRemoteDataSource, ProfileRemoteDataSource
dashboard	Панель управления, цели, аналитика, PDF-отчет	DashboardAnalyticsCalculator, DashboardScreen
workout	Тренировки, маршруты, история, сохранение	WorkoutSessionController, WorkoutModel
nutrition	Дневник питания, продукты, штрихкод, КБЖУ	FoodDiaryController, NutritionMacroCalculator
steps	Шагомер и дневная цель	StepTrackingService, SqfliteStepLocalDataSource
social	Друзья, заявки, чат, лидерборд, приватность	SocialRemoteDataSource, FriendVisibilityService
coach	Режим тренера, упражнения, рецепты, шаблоны, медиа	CoachRepositoryImpl, CoachDashboardScreen
ai_coach	AI-консультации в рамках фитнеса и питания	FitnessAiAgent, PollinationsAiClient
notifications	Настройки напоминаний	ReminderSettingsController, AppNotificationService

Управление состоянием реализовано через flutter_riverpod и riverpod_annotation. Для каждого крупного функционального блока выделены провайдеры, репозитории, источники данных и контроллеры экранов, что отделяет интерфейс от бизнес-логики и хранения данных.

В рамках разработки программного продукта реализовано единое мобильное приложение Liga Gym, предназначенное для авторизации пользователя, учета тренировок, маршрутов, питания, шаговой активности, целей, аналитики прогресса, общения с друзьями, работы с тренером и получения AI-рекомендаций по фитнесу.

Клиентская часть реализована на языке Dart с использованием фреймворка Flutter и библиотеки Material 3. Интерфейс приложения включает splash-экран с Lottie-анимацией, экраны входа и регистрации, анкету профиля, главный экран, тренировочные экраны и формы работы с дневником питания. Для локализации используются ресурсы на русском и английском языках.

Для удаленного хранения данных и аутентификации используются Firebase Authentication, Google Sign-In и Cloud Firestore. Локальное хранение тренировок и записей питания организовано на базе SQLite через библиотеку sqflite. Маршрут тренировки формируется на основе данных Geolocator, а поиск продукта по штрихкоду выполняется по встроенному in-memory каталогу.

Архитектура приложения построена по модульному принципу с выделением функциональных областей auth, dashboard, workout, nutrition, steps, ai_coach, social и coach. Внутри модулей код разделен на уровни presentation, domain и data, а состояние управляется через Riverpod.

Главный экран приложения объединяет быстрые действия, карточку дневного прогресса, недельные показатели и переходы к тренировкам, дневнику питания, шагомеру, друзьям и AI-помощнику.

Подсистема аналитики рассчитывает шаги, калории, прогресс по целям, изменение веса, средние показатели за период и позволяет сформировать PDF-отчет для передачи или хранения.

Подсистема шагомера использует фоновый сервис Android и датчик физической активности. Данные сохраняются локально и обновляют интерфейс при достижении дневной цели.

Социальный модуль реализует приглашения по коду друга, заявки, список друзей, персональные чаты, обмен результатами и настройки приватности.

Тренерский модуль предназначен для пользователей с ролью trainer. Тренер создает упражнения, рецепты, шаблоны тренировок, загружает изображения или видео и назначает материалы ученикам.

AI-помощник формирует ответы на вопросы о питании и тренировках с учетом дневника питания, цели пользователя, дневной нормы калорий и шагов.

На рисунке 2.1 изображена модульная схема мобильного приложения.

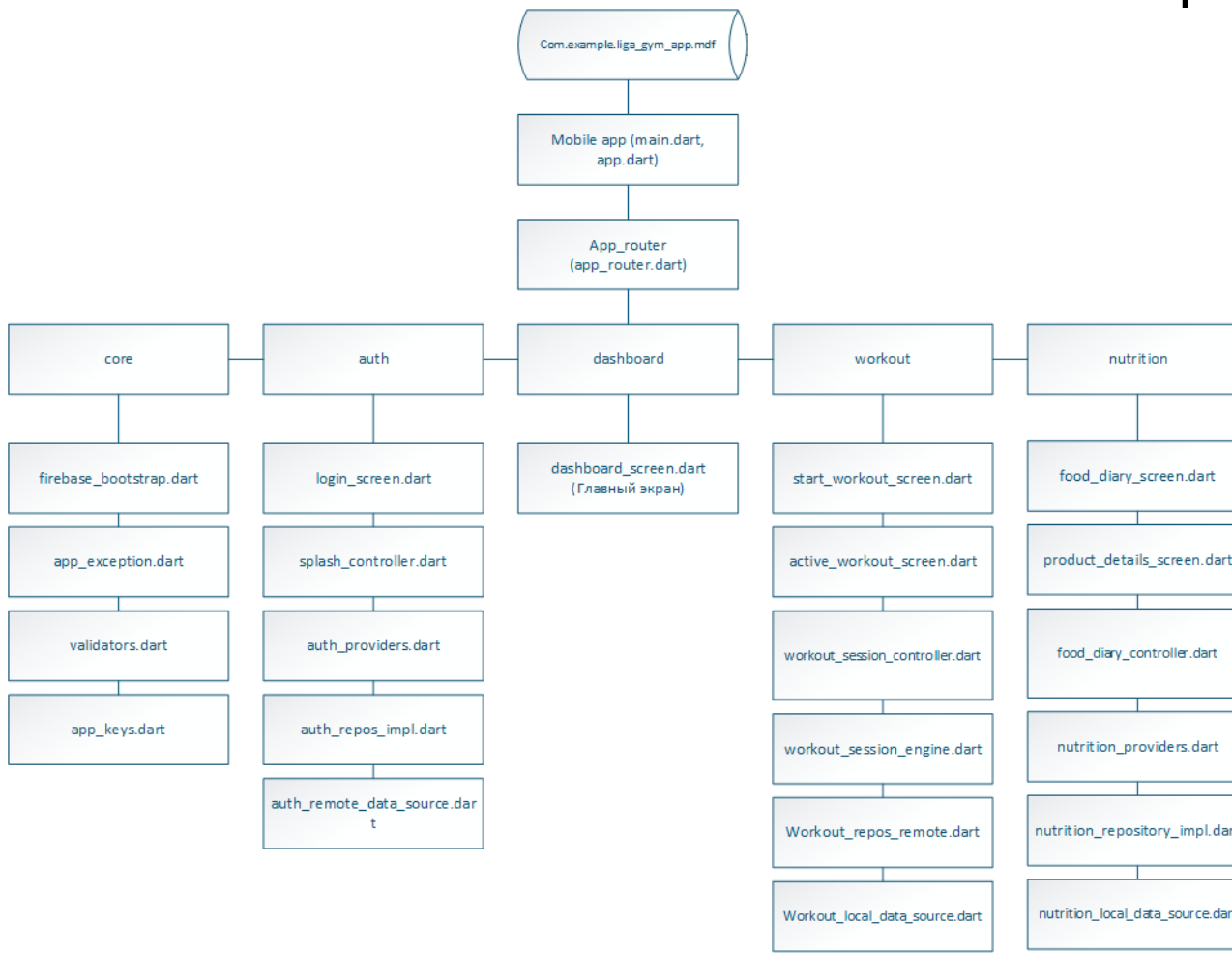


Рисунок 2.1 – Модульная схема мобильного приложения

В таблице 2.1 описывается каждый модуль мобильного приложения, перечисляются процедуры пользовательского уровня и описываются действия, выполняемые в рамках интерфейсных сценариев.

Таблица 2.1 – Описание процедур мобильного приложения

Процедуры	Название
1	2
app.dart	
Liga GymApp	Класс модуля приложения, участвующий в общей архитектуре приложения.

Процедуры	Название
Liga GymApp.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
core/navigation/app_router.dart	Формирует экраны по именам маршрутов и передает аргументы навигации.
onGenerateRoute()	Создает и возвращает экран приложения по имени маршрута.
core/navigation/app_routes.dart	Содержит константы именованных маршрутов мобильного приложения.
features/auth/presentation/controllers/auth_action_controller.dart	Контроллер состояния и действий модуля авторизации.
AuthActionController	Класс контроллера, управляющий состоянием и вызовами пользовательских сценариев.
AuthActionController.loginWithEmail()	Выполняет вход пользователя по email и паролю.
AuthActionController.signInWithGoogle()	Запускает сценарий входа через учетную запись Google.
AuthActionController.registerUser()	Создает новую учетную запись пользователя и возвращает результат авторизации.
AuthActionController.saveUserProfile()	Сохраняет заполненный профиль пользователя после регистрации.
AuthActionController.signOut()	Завершает пользовательскую сессию и очищает auth-состояние.
features/auth/presentation/controllers/splash_controller.dart	
SplashController	Класс контроллера, управляющий состоянием и вызовами пользовательских сценариев.
SplashController.checkUserAuthState()	
features/auth/presentation/providers/auth_providers.dart	Провайдеры зависимостей и состояния модуля авторизации.
authStateChanges()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.

Процедуры	Название
currentAuthUser()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
features/auth/presentation/screens/login_screen.dart	Экран пользовательского интерфейса модуля авторизации.
LoginScreen	Класс экрана пользовательского интерфейса модуля авторизации.
_LoginScreenState.dispose()	
_LoginScreenState.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/auth/presentation/screens/profile_setup_screen.dart	Экран пользовательского интерфейса модуля авторизации.
ProfileSetupScreen	Класс экрана пользовательского интерфейса модуля авторизации.
_ProfileSetupScreenState.dispose()	
_ProfileSetupScreenState.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
Features/auth/presentation/screens/register_screen.dart	Экран пользовательского интерфейса модуля авторизации.
RegisterScreen	Класс экрана пользовательского интерфейса модуля авторизации.
_RegisterScreenState.dispose()	
_RegisterScreenState.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
Features/auth/presentation/screens/splash_screen.dart	
SplashScreen	Класс экрана пользовательского интерфейса модуля авторизации.

Процедуры	Название
_SplashScreenState.initState()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
_SplashScreenState.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
Features/auth/presentation/utils/auth_status_route_mapper.dart	
mapAuthStatusToRoute()	Преобразует auth-состояние в маршрут приложения.
Features/auth/presentation/widgets/auth_page_scaffold.dart	
AuthPageScaffold	Переиспользуемый виджет модуля авторизации.
AuthPageScaffold.build()	Класс модуля авторизации, участвующий в общей архитектуре приложения.
AuthPageScaffold.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
Features/dashboard/presentation/screens/dashboard_screen.dart	
DashboardScreen	Экран пользовательского интерфейса модуля главного экрана.
DashboardScreen.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/nutrition/presentation/controllers/add_food_controller.dart	
AddFoodState	Контроллер состояния и действий модуля питания.
AddFoodState	Класс состояния, хранящий актуальные данные пользовательского сценария.
AddFoodController	Класс контроллера, управляющий состоянием и вызовами пользовательских сценариев.
AddFoodController.setInputMethod()	Переключает режим добавления записи питания.
AddFoodController.findProductByBarcode()	Ищет продукт по штрихкоду в доступном каталоге.
AddFoodController.calculateMacros()	Рассчитывает калории и БЖУ для выбранной порции.
AddFoodController.addFoodEntry()	Добавляет запись в дневник питания и инициирует сохранение.
features/nutrition/presentation/controllers/food_diary_controller.dart	
FoodDiaryState	Контроллер состояния и действий модуля питания.
FoodDiaryState	Класс состояния, хранящий актуальные данные пользовательского сценария.

Процедуры	Название
FoodDiaryController	Класс контроллера, управляющий состоянием и вызовами пользовательских сценариев.
FoodDiaryController.loadDailyFoodEntries()	Загружает записи дневника питания за указанную дату.
features/nutrition/presentation/providers/nutrition_providers.dart	Провайдеры зависимостей и состояния модуля питания.
todayNutritionDiary()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
features/nutrition/presentation/screens/add_food_screen.dart	Экран пользовательского интерфейса модуля питания.
AddFoodScreen	Класс экрана пользовательского интерфейса модуля питания.
_AddFoodScreenState.initState()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
_AddFoodScreenState.dispose()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
_AddFoodScreenState.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
_NutritionNumberField.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/nutrition/presentation/screens/food_diary_screen.dart	Экран пользовательского интерфейса модуля питания.
FoodDiaryScreen	
_FoodDiaryScreenState.initState()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
_FoodDiaryScreenState.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
_MealSectionCard.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
_MacrosSummary.build()	Формирует интерфейс или начальное состояние соответствующего модуля.

Процедуры	Название
_MacroChip.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/nutrition/presentation/screens/product_details_screen.dart	
ProductDetailsScreen	Класс экрана пользовательского интерфейса модуля питания.
ProductDetailsScreen.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
_MacrosCard.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/nutrition/presentation/utils/nutrition_route_arguments.dart	Файл nutrition_route_arguments.dart, поддерживающий работу модуля питания.
AddFoodRouteArguments	Класс модуля питания, участвующий в общей архитектуре приложения.
ProductDetailsRouteArguments	Класс модуля питания, участвующий в общей архитектуре приложения.
features/workout/presentation/controllers/workout_list_controller.dart	Контроллер состояния и действий модуля тренировок.
WorkoutListState	Класс состояния, хранящий актуальные данные пользовательского сценария.
WorkoutListController	Класс контроллера, управляющий состоянием и вызовами пользовательских сценариев.
WorkoutListController.loadUserWorkouts()	Загружает историю тренировок пользователя.
WorkoutListController.filterWorkouts()	
features/workout/presentation/controllers/workout_session_controller.dart	Контроллер состояния и действий модуля тренировок.
WorkoutSessionState	Класс состояния, хранящий актуальные данные пользовательского сценария.
WorkoutSessionController	
WorkoutSessionController.startWorkoutTimer()	Запускает тренировочную сессию, таймер и сбор маршрута.
WorkoutSessionController.pauseWorkout()	
WorkoutSessionController.resumeWorkout()	Возобновляет ранее приостановленную тренировку.

Процедуры	Название
WorkoutSessionController.stopWorkout()	Завершает тренировочную сессию и формирует итоговый результат.
WorkoutSessionController.saveWorkoutToDatabase()	Сохраняет завершённую тренировку в локальное и/или облачное хранилище.
WorkoutSessionController.reset()	Сбрасывает состояние модуля тренировок к начальному значению.
features/workout/presentation/providers/workout_providers.dart	Провайдеры зависимостей и состояния модуля тренировок.
features/workout/presentation/screens/active_workout_screen.dart	Экран пользовательского интерфейса модуля тренировок.
ActiveWorkoutScreen	Класс экрана пользовательского интерфейса модуля тренировок.
ActiveWorkoutScreen.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
_MetricTile.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/workout/presentation/screens/start_workout_screen.dart	Экран пользовательского интерфейса модуля тренировок.
StartWorkoutScreen	
_StartWorkoutScreenState.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/workout/presentation/screens/workout_list_screen.dart	Экран пользовательского интерфейса модуля тренировок.
WorkoutListScreen	
_WorkoutListScreenState.initState()	Служебная процедура модуля тренировок, поддерживающая основной пользовательский сценарий.
_WorkoutListScreenState.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/workout/presentation/screens/workout_result_screen.dart	Экран пользовательского интерфейса модуля тренировок.
WorkoutResultScreen	Класс экрана пользовательского

Процедуры	Название
	интерфейса модуля тренировок.
WorkoutResultScreen.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
_ResultRow.build()	Формирует интерфейс или начальное состояние соответствующего модуля.
features/workout/presentation/utils/workout_formatters.dart	Файл workout_formatters.dart, поддерживающий работу модуля тренировок.
formatWorkoutDuration()	Форматирует значение для отображения или сохранения в модуле тренировок.
formatWorkoutDistance()	Форматирует значение для отображения или сохранения в модуле тренировок.
formatWorkoutCalories()	Форматирует значение для отображения или сохранения в модуле тренировок.
formatWorkoutTimestamp()	Форматирует значение для отображения или сохранения в модуле тренировок.
localizeWorkoutType()	Служебная процедура модуля тренировок, поддерживающая основной пользовательский сценарий.
l10n/app_localizations.dart	Файл app_localizations.dart, поддерживающий работу модуля локализации.
dashboardNutritionCalories()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
dashboardNutritionProteins()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
dashboardNutritionFats()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
dashboardNutritionCarbs()	
dashboardSignedInAs()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.

Процедуры	Название
foodDiaryEntrySubtitle()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
foodDiaryInlineMacros()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
productDetailsMeal()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
productDetailsPortion()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
_AppLocalizationsDelegate.load()	Загружает данные модуля локализации из доступного источника.
_AppLocalizationsDelegate.isSupported()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
_AppLocalizationsDelegate.shouldReload()	
n/app_localizations_en.dart	Файл app_localizations_en.dart, поддерживающий работу модуля локализации.
AppLocalizationsEn	Класс модуля локализации, участвующий в общей архитектуре приложения.
AppLocalizationsEn.dashboardNutritionCalories()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
AppLocalizationsEn.dashboardNutritionProteins()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
AppLocalizationsEn.dashboardNutritionFats()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
AppLocalizationsEn.dashboardNutritionCarbs()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.

Процедуры	Название
AppLocalizationsEn.dashboardSignedInAs()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
AppLocalizationsEn.foodDiaryEntrySubtitle()	
AppLocalizationsEn.foodDiaryInlineMacros()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.
AppLocalizationsEn.productDetailsMeal()	Служебная процедура модуля локализации, поддерживающая основной пользовательский сценарий.

В таблице 2.2 описываются сервисы, сущности и процедуры хранения данных, перечисляются алгоритмы предметной области и операции, выполняемые репозиториями, источниками данных и вспомогательными модулями приложения.

Таблица 2.2 – Описание процедур сервисов, домена и хранения данных

Процедуры	Название
1	2
core/constants/app_keys.dart	
core/errors/app_exception.dart	Инфраструктурный модуль обработки ошибок, используемый несколькими частями приложения.
core/firebase/firebase_bootstrap.dart	Инфраструктурный модуль инициализации Firebase, используемый несколькими частями приложения.
FirebaseBootstrapResult	Класс модуля инициализации Firebase, участвующий в общей архитектуре приложения.
FirebaseBootstrapResult.initialize()	Служебная процедура модуля инициализации Firebase, поддерживающая основной пользовательский сценарий.
FirebaseUnavailableBootstrapResult	
core/utils/date_formatter.dart	Инфраструктурный модуль вспомогательных вычислений, используемый несколькими частями приложения.
formatLocalizedDate()	Форматирует значение для отображения или сохранения в модуле вспомогательных вычислений.
core/utils/input_validators.dart	Инфраструктурный модуль вспомогательных вычислений,

Процедуры	Название
	используемый несколькими частями приложения.
core/utills/localization_extensions.dart	Инфраструктурный модуль вспомогательных вычислений, используемый несколькими частями приложения.
localize()	Служебная процедура модуля вспомогательных вычислений, поддерживающая основной пользовательский сценарий.
localize()	Служебная процедура модуля вспомогательных вычислений, поддерживающая основной пользовательский сценарий.
localize()	Служебная процедура модуля вспомогательных вычислений, поддерживающая основной пользовательский сценарий.
features/auth/data/datasources/auth_remote_data_source.dart	Источник данных, работающий с внешним или локальным хранилищем модуля авторизации.
watchAuthState()	Подписывается на поток изменений данных модуля авторизации.
getCurrentUser()	Возвращает вычисленное или подготовленное значение для модуля авторизации.
loginWithEmail()	Выполняет вход пользователя по email и паролю.
registerUser()	Создает новую учетную запись пользователя и возвращает результат авторизации.
signInWithGoogle()	Запускает сценарий входа через учетную запись Google.
signOut()	Завершает пользовательскую сессию и очищает auth-состояние.
UnavailableAuthRemoteDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
UnavailableAuthRemoteDataSource.watchAuthState()	Подписывается на поток изменений данных модуля авторизации.
UnavailableAuthRemoteDataSource.getCurrentUser()	Возвращает вычисленное или подготовленное значение для модуля авторизации.
UnavailableAuthRemoteDataSource.loginWithEmail()	Выполняет вход пользователя по email и паролю.
UnavailableAuthRemoteDataSource.registerUser()	Создает новую учетную запись пользователя и возвращает результат авторизации.
UnavailableAuthRemoteDataSource.signInWithGoogle()	Запускает сценарий входа через учетную запись Google.
UnavailableAuthRemoteDataSource.signOut()	Завершает пользовательскую сессию и очищает auth-состояние.

Процедуры	Название
FirebaseAuthRemoteDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
FirebaseAuthRemoteDataSource.watchAuthState()	
FirebaseAuthRemoteDataSource.getCurrentUser()	Возвращает вычисленное или подготовленное значение для модуля авторизации.
FirebaseAuthRemoteDataSource.loginWithEmail()	Выполняет вход пользователя по email и паролю.
FirebaseAuthRemoteDataSource.registerUser()	Создает новую учетную запись пользователя и возвращает результат авторизации.
FirebaseAuthRemoteDataSource.signInWithGoogle()	Запускает сценарий входа через учетную запись Google.
FirebaseAuthRemoteDataSource.signOut()	Завершает пользовательскую сессию и очищает auth-состояние.
features/auth/data/datasources/profile_remote_data_source.dart	Источник данных, работающий с внешним или локальным хранилищем модуля авторизации.
hasUserProfile()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
saveUserProfile()	Сохраняет заполненный профиль пользователя после регистрации.
UnavailableProfileRemoteDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
UnavailableProfileRemoteDataSource.hasUserProfile()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
UnavailableProfileRemoteDataSource.saveUserProfile()	Сохраняет заполненный профиль пользователя после регистрации.
FirestoreProfileRemoteDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
FirestoreProfileRemoteDataSource.hasUserProfile()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
FirestoreProfileRemoteDataSource.saveUserProfile()	Сохраняет заполненный профиль пользователя после регистрации.
features/auth/data/models/auth_user_model.dart	Модель сериализации и преобразования данных модуля авторизации.
AuthUserModel	Класс модели данных для сериализации и преобразования значений.
UserProfileModel	Класс модели данных для сериализации и преобразования значений.

Процедуры	Название
UserProfileModel.toFirestore()	Преобразует объект в структуру документа Cloud Firestore.
features/auth/data/repositories/auth_repository_impl.dart	Репозиторий, объединяющий источники данных модуля авторизации.
AuthRepositoryImpl	Реализация репозитория, координирующая обмен данными между слоями.
AuthRepositoryImpl.watchAuthState()	Подписывается на поток изменений данных модуля авторизации.
AuthRepositoryImpl.getCurrentUser()	Возвращает вычисленное или подготовленное значение для модуля авторизации.
AuthRepositoryImpl.checkUserAuthState()	Проверяет текущее состояние авторизации и определяет целевой экран.
AuthRepositoryImpl.loginWithEmail()	Выполняет вход пользователя по email и паролю.
AuthRepositoryImpl.signInWithGoogle()	Запускает сценарий входа через учетную запись Google.
AuthRepositoryImpl.registerUser()	Создает новую учетную запись пользователя и возвращает результат авторизации.
AuthRepositoryImpl.saveUserProfile()	Сохраняет заполненный профиль пользователя после регистрации.
AuthRepositoryImpl.signOut()	Завершает пользовательскую сессию и очищает auth-состояние.
features/auth/domain/entities/auth_status.dart	
features/auth/domain/entities/auth_user.dart	Сущность предметной области модуля авторизации.
AuthUser	Класс модуля авторизации, участвующий в общей архитектуре приложения.
features/auth/domain/entities/gender.dart	Сущность предметной области модуля авторизации.
features/auth/domain/entities/profile_setup_data.dart	Сущность предметной области модуля авторизации.
ProfileSetupData	Класс модуля авторизации, участвующий в общей архитектуре приложения.
features/auth/domain/entities/user_profile.dart	Сущность предметной области модуля авторизации.
UserProfile	Класс модуля авторизации, участвующий в общей архитектуре приложения.
features/auth/domain/repositories/auth_repository.dart	Файл auth_repository.dart, поддерживающий работу модуля авторизации.
watchAuthState()	Подписывается на поток изменений данных модуля авторизации.
getCurrentUser()	Возвращает вычисленное или подготовленное значение для модуля авторизации.

Процедуры	Название
checkUserAuthState()	Проверяет текущее состояние авторизации и определяет целевой экран.
loginWithEmail()	Выполняет вход пользователя по email и паролю.
signInWithGoogle()	
registerUser()	Создает новую учетную запись пользователя и возвращает результат авторизации.
saveUserProfile()	Сохраняет заполненный профиль пользователя после регистрации.
signOut()	Завершает пользовательскую сессию и очищает auth-состояние.
features/auth/domain/usecases/check_user_auth_state_use_case.dart	Сценарий прикладной логики модуля авторизации.
CheckUserAuthStateUseCase	Класс прикладного сценария, реализующего отдельную бизнес-операцию.
CheckUserAuthStateUseCase.call()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
features/auth/domain/usecases/login_with_email_use_case.dart	
LoginWithEmailUseCase	Класс прикладного сценария, реализующего отдельную бизнес-операцию.
LoginWithEmailUseCase.call()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
features/auth/domain/usecases/register_user_use_case.dart	Сценарий прикладной логики модуля авторизации.
RegisterUserUseCase	Класс прикладного сценария, реализующего отдельную бизнес-операцию.
RegisterUserUseCase.call()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
features/auth/domain/usecases/save_user_profile_use_case.dart	Сценарий прикладной логики модуля авторизации.
SaveUserProfileUseCase	Класс прикладного сценария, реализующего отдельную бизнес-операцию.
SaveUserProfileUseCase.call()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
features/auth/domain/usecases/sign_in_with_google_use_case.dart	Сценарий прикладной логики модуля авторизации.
SignInWithGoogleUseCase	Класс прикладного сценария, реализующего отдельную бизнес-операцию.

Процедуры	Название
SignInWithGoogleUseCase.call()	Служебная процедура модуля авторизации, поддерживающая основной пользовательский сценарий.
features/nutrition/data/datasources/nutrition_local_data_source.dart	
saveFoodEntry()	Сохраняет данные модуля питания в целевое хранилище.
saveFoodEntries()	Сохраняет данные модуля питания в целевое хранилище.
markFoodEntrySynced()	
SqfliteNutritionLocalDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
SqfliteNutritionLocalDataSource.saveFoodEntry()	Сохраняет данные модуля питания в целевое хранилище.
SqfliteNutritionLocalDataSource.saveFoodEntries()	Сохраняет данные модуля питания в целевое хранилище.
SqfliteNutritionLocalDataSource.markFoodEntrySynced()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
features/nutrition/data/datasources/nutrition_remote_data_source.dart	
saveFoodEntry()	Сохраняет данные модуля питания в целевое хранилище.
UnavailableNutritionRemoteDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
UnavailableNutritionRemoteDataSource.saveFoodEntry()	Сохраняет данные модуля питания в целевое хранилище.
FirestoreNutritionRemoteDataSource	
FirestoreNutritionRemoteDataSource.saveFoodEntry()	Сохраняет данные модуля питания в целевое хранилище.
features/nutrition/data/datasources/product_catalog_data_source.dart	
findByBarcode()	Выполняет поиск данных модуля питания по заданным параметрам.
InMemoryProductCatalogDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
InMemoryProductCatalogDataSource.findByBarcode()	Выполняет поиск данных модуля питания по заданным параметрам.
features/nutrition/data/models/food_entry_model.dart	
FoodEntryModel	Класс модели данных для сериализации и преобразования значений.

Процедуры	Название
FoodEntryModel.toLocalMap()	Преобразует объект в формат локальной SQLite-таблицы.
FoodEntryModel.toFirestore()	Преобразует объект в структуру документа Cloud Firestore.
FoodEntryModel.withSyncStatus()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
FoodEntryModel.fromDraft()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
FoodEntryModel.buildDateKey()	
features/nutrition/data/models/food_product_model.dart	Модель сериализации и преобразования данных модуля питания.
FoodProductModel	Класс модели данных для сериализации и преобразования значений.
features/nutrition/data/repositories/nutrition_repository_impl.dart	Репозиторий, объединяющий источники данных модуля питания.
NutritionRepositoryImpl	Реализация репозитория, координирующая обмен данными между слоями.
NutritionRepositoryImpl.loadDailyFoodEntries()	Загружает записи дневника питания за указанную дату.
NutritionRepositoryImpl.addFoodEntry()	Добавляет запись в дневник питания и инициирует сохранение.
NutritionRepositoryImpl.findProductByBarcode()	Ищет продукт по штрихкоду в доступном каталоге.
NutritionRepositoryImpl.calculateMacros()	Рассчитывает калории и БЖУ для выбранной порции.
features/nutrition/domain/entities/daily_food_diary.dart	Сущность предметной области модуля питания.
DailyFoodDiary	Класс модуля питания, участвующий в общей архитектуре приложения.
DailyFoodDiary.entriesForMeal()	
DailyFoodDiary.totalMacros()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
DailyFoodDiary.mealMacros()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
features/nutrition/domain/entities/food_entry.dart	Сущность предметной области модуля питания.
FoodEntry	Класс модуля питания, участвующий в общей архитектуре приложения.
FoodEntry.localizedName()	Возвращает локализованное отображаемое название объекта.
features/nutrition/domain/entities/food_entry_draft.dart	Сущность предметной области модуля питания.

Процедуры		Название
FoodEntryDraft	Класс модуля питания, участвующий в общей архитектуре приложения.	
features/nutrition/domain/entities/food_input_method.dart		
features/nutrition/domain/entities/food_macros.dart	Сущность предметной области модуля питания.	
FoodMacros	Класс модуля питания, участвующий в общей архитектуре приложения.	
features/nutrition/domain/entities/food_product.dart	Сущность предметной области модуля питания.	
FoodProduct	Класс модуля питания, участвующий в общей архитектуре приложения.	
FoodProduct.localizedName()		
features/nutrition/domain/entities/meal_type.dart	Сущность предметной области модуля питания.	
features/nutrition/domain/repositories/nutrition_repository.dart	Файл nutrition_repository.dart, поддерживающий работу модуля питания.	
loadDailyFoodEntries()	Загружает записи дневника питания за указанную дату.	
addFoodEntry()	Добавляет запись в дневник питания и инициирует сохранение.	
findProductByBarcode()	Ищет продукт по штрихкоду в доступном каталоге.	
calculateMacros()		Рассчитывает калории и БЖУ для выбранной порции.
features/nutrition/domain/services/nutrition_macro_calculator.dart		Сервис вычислений и бизнес-правил модуля питания.
NutritionMacroCalculator		Класс сервисной логики, выполняющий вычисления и обработку состояния.
NutritionMacroCalculator.calculate()		Выполняет вычисление значений предметной области для текущего сценария.
features/nutrition/domain/usecases/add_food_entry_use_case.dart		Сценарий прикладной логики модуля питания.
AddFoodEntryUseCase		Класс прикладного сценария, реализующего отдельную бизнес-операцию.
AddFoodEntryUseCase.call()		Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
features/nutrition/domain/usecases/calculate_macros_use_case.dart		Сценарий прикладной логики модуля питания.
CalculateMacrosUseCase		Класс прикладного сценария, реализующего отдельную бизнес-операцию.
CalculateMacrosUseCase.call()		Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.

Процедуры	Название
features/nutrition/domain/usecases/find_product_by_barcode_use_case.dart	Сценарий прикладной логики модуля питания.
FindProductByBarcodeUseCase	
FindProductByBarcodeUseCase.call()	Служебная процедура модуля питания, поддерживающая основной пользовательский сценарий.
features/nutrition/domain/usecases/load_daily_food_entries_use_case.dart	Сценарий прикладной логики модуля питания.
LoadDailyFoodEntriesUseCase	Класс прикладного сценария, реализующего отдельную бизнес-операцию.
LoadDailyFoodEntriesUseCase.call()	
features/workout/data/datasources/workout_local_data_source.dart	Источник данных, работающий с внешним или локальным хранилищем модуля тренировок.
saveWorkout()	Сохраняет данные модуля тренировок в целевое хранилище.
markWorkoutSynced()	
SqliteWorkoutLocalDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
SqliteWorkoutLocalDataSource.saveWorkout()	
SqliteWorkoutLocalDataSource.markWorkoutSynced()	Служебная процедура модуля тренировок, поддерживающая основной пользовательский сценарий.
features/workout/data/datasources/workout_location_data_source.dart	Источник данных, работающий с внешним или локальным хранилищем модуля тренировок.
prepareTracking()	
watchRoute()	Подписывается на поток изменений данных модуля тренировок.
GeolocatorWorkoutLocationDataSource	
GeolocatorWorkoutLocationDataSource.prepareTracking()	Подготавливает инфраструктуру или зависимости модуля тренировок к работе.
GeolocatorWorkoutLocationDataSource.watchRoute()	Подписывается на поток изменений данных модуля тренировок.
features/workout/data/datasources/workout_remote_data_source.dart	Источник данных, работающий с внешним или локальным хранилищем модуля тренировок.
saveWorkout()	
UnavailableWorkoutRemoteDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.

Процедуры	Название
UnavailableWorkoutRemoteDataSource.saveWorkout()	
FirestoreWorkoutRemoteDataSource	Класс источника данных, инкапсулирующий работу с конкретным хранилищем.
FirestoreWorkoutRemoteDataSource.saveWorkout()	Сохраняет данные модуля тренировок в целевое хранилище.
features/workout/data/models/workout_model.dart	Модель сериализации и преобразования данных модуля тренировок.
WorkoutModel	Класс модели данных для сериализации и преобразования значений.
WorkoutModel.toLocalMap()	
WorkoutModel.toFirestore()	Преобразует объект в структуру документа Cloud Firestore.
WorkoutModel.withSyncStatus()	Служебная процедура модуля тренировок, поддерживающая основной пользовательский сценарий.
features/workout/data/repositories/workout_repository_impl.dart	Репозиторий, объединяющий источники данных модуля тренировок.
WorkoutRepositoryImpl	Реализация репозитория, координирующая обмен данными между слоями.

2.2 Протокол тестирования программного продукта

Таблица 2.Б – Сводный протокол тестирования основных функций

№	Функция	Входные данные	Ожидаемый результат	Факт	Статус
1	Запуск приложения	Открыть приложение	Splash-экран и переход по статусу входа	Да	Пройден
2	Регистрация	Корректные email и пароль	Создана учетная запись, открыт профиль	Да	Пройден
3	Авторизация	Существующие учетные данные	Переход пользователя на dashboard	Да	Пройден
4	Профиль	Рост, вес, цель, город	Документ users обновлен	Да	Пройден
5	История веса	Новая запись веса	Создан документ weight_history	Да	Пройден
6	Тренировка	Тип, старт, финиш	Сохранены duration, calories, distanceMeters	Да	Пройден
7	Маршрут	Разрешение геолокации	routeJson содержит точки маршрута	Да	Пройден
8	Питание вручную	Продукт и граммы	Рассчитаны КБЖУ	Да	Пройден
9	Штрихкод	barcode продукта	Открыт продукт или сообщение об отсутствии	Да	Пройден
10	Шагомер	Данные датчика pedometer	Обновлена daily_steps	Да	Пройден
11	Аналитика	Диапазон дат	Показаны шаги, калории и прогресс	Да	Пройден

№	Функция	Входные данные	Ожидаемый результат	Факт	Статус
12	PDF-экспорт	Команда экспорта	Создан PDF и открыт системный шаринг	Да	Пройден
13	AI-помощник	Вопрос о питании	Ответ ограничен темой фитнеса и питания	Да	Пройден
14	Друзья	Код друга	Создана заявка friend_requests	Да	Пройден
15	Чат	Текст сообщения	Сообщение сохранено в messages	Да	Пройден
16	Лидерборд	Открыть рейтинг	Отображаются score, workoutsCount, stepsCount	Да	Пройден
17	Приватность	Скрыть категорию	FriendVisibilityService ограничивает показ	Да	Пройден
18	Тренер	Создать материал	Материал сохранен в коллекциях trainers	Да	Пройден
19	Медиа	Загрузить файл	Файл отправлен в bucket coach-media	Да	Пройден
20	Офлайн-сценарий	Создать запись без сети	is_synced = 0, затем синхронизация	Да	Пройден

Процесс исследования и испытания программы, целью которого является проверка соответствия между её реальным поведением и поведением, зафиксированным в требованиях.

Эта проверка осуществляется путем прогона программы на конечном наборе сценариев, подобранных таким образом, чтобы максимизировать вероятность обнаружения несоответствий.

В таблице 2.3 представлен протокол тестирования «Проверка регистрации пользователя при вводе корректных данных».

Таблица 2.3 – Протокол тестирования «Проверка регистрации пользователя при вводе корректных данных»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка регистрации пользователя при вводе корректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при регистрации пользователя и первичном заполнении профиля
Этапы теста	1 Перейти на экран регистрации; 2 Ввести корректные данные; 3 Нажать на кнопку «Зарегистрироваться»; 4 Заполнить профиль; 5 Сохранить профиль.

Поле	Описание
Тестовые данные	Электронная почта: «newuser@ligagym.dev»; Пароль: «password123»; Подтверждение пароля: «password123»; Имя: «Alex»; Пол: «Male»; Дата рождения: «08.04.2004».
Ожидаемый результат	Успешная регистрация, открытие экрана заполнения профиля, сохранение профиля и переход на главный экран приложения
Фактический результат	Успешная регистрация, открытие экрана заполнения профиля, сохранение профиля и переход на главный экран приложения

На рисунках 2.2 – 2.3 изображены результаты тестирования на проверку регистрации пользователя при вводе корректных данных.

Рисунок 2.2 – Заполненная форма регистрации с корректными данными

Настройка профиля

Расскажите немного о себе, чтобы мы персонализировали приложение.

Имя
Alex

Пол
Мужской

Дата рождения
8 апреля 2004 г.

Сохранить профиль

Рисунок 2.3 – Главный экран приложения после успешной регистрации

В таблице 2.4 представлен протокол тестирования «Проверка добавления записи о питании при вводе корректных данных».

Таблица 2.4 – Протокол тестирования «Проверка добавления записи о питании при вводе корректных данных»

Поле	Описание
1	2
Дата теста	25.05.2026
Заголовок/название теста	Проверка добавления записи о питании при вводе корректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при добавлении записи о питании вручную
Этапы теста	1 Открыть дневник питания; 2 Нажать на кнопку добавления записи; 3 Заполнить данные продукта; 4 Нажать «Продолжить»; 5 Сохранить запись в дневник.
Тестовые данные	Прием пищи: «Завтрак»; Продукт: «Oatmeal»; Калории на 100 г: «352»; Белки: «12»; Жиры: «6»; Углеводы: «60»; Вес порции: «50».

Поле	Описание
Ожидаемый результат	Корректный расчет калорий и БЖУ для порции, сохранение записи в дневник питания и отображение новой записи в выбранном разделе
Фактический результат	Корректный расчет калорий и БЖУ для порции, сохранение записи в дневник питания и отображение новой записи в выбранном разделе

На рисунках 2.4 – 2.6 изображены результаты тестирования на проверку добавления записи о питании при вводе корректных данных.

Рисунок 2.4 – Ввод корректных данных о продукте в ручном режиме



Рисунок 2.5 – Корректно рассчитанные калории и БЖУ для выбранной порции

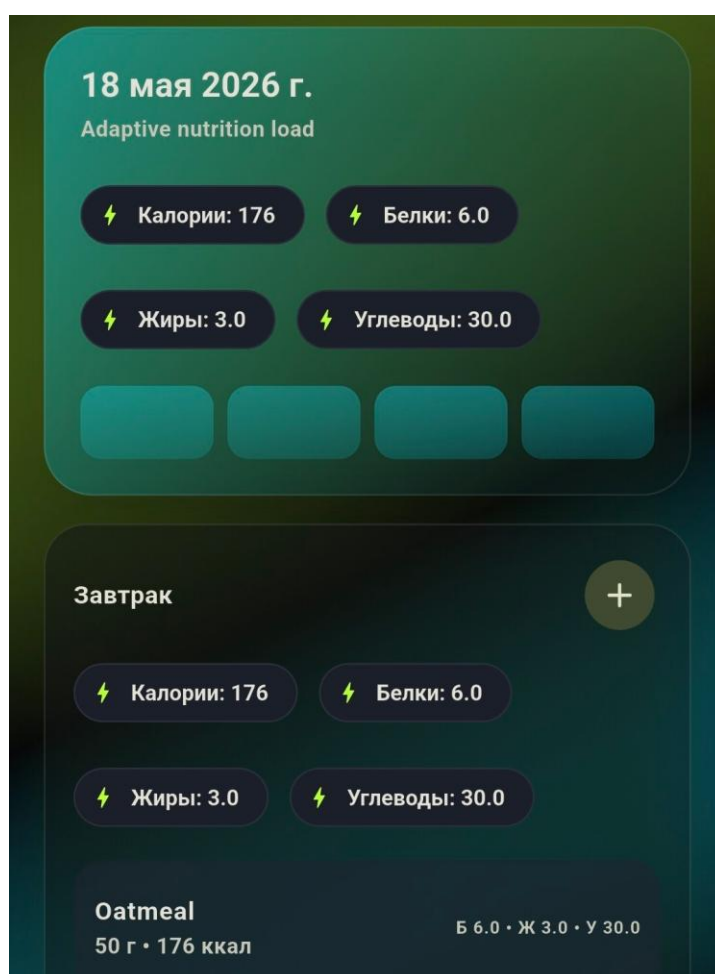


Рисунок 2.6 – Добавленная запись в дневнике питания

В таблице 2.5 представлен протокол тестирования «Проверка авторизации при вводе некорректных данных».

Таблица 2.5 – Протокол тестирования «Проверка авторизации при вводе некорректных данных»

Поле	Описание
1	2
Дата теста	25.05.2026
Заголовок/название теста	Проверка авторизации при вводе некорректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при входе пользователя с некорректными значениями полей
Этапы теста	1 Открыть экран входа; 2 Ввести некорректные данные; 3 Нажать на кнопку «Войти».

Продолжение таблицы 2.5

1	2
Тестовые данные	Электронная почта: «wrong-email»; Пароль: «12345»; Дополнительный сценарий: оставить оба поля пустыми при повторной проверке.
Тестовые данные	Пустой email: «»; Пустой пароль: «»; Некорректный email: «wrong-email»; Слишком короткий пароль: «12345».
Ожидаемый результат	Вывод сообщений о необходимости заполнения полей, ошибке формата email и недостаточной длине пароля
Фактический результат	Вывод сообщений о необходимости заполнения полей, ошибке формата email и недостаточной длине пароля

На рисунке 2.7 изображен результат тестирования на авторизацию при вводе некорректных данных.

Рисунок 2.7 – Заполненная форма входа с некорректными данными

В таблице 2.6 представлен протокол тестирования «Проверка заполнения профиля пользователя при вводе некорректных данных».

Таблица 2.6 – Протокол тестирования «Проверка заполнения профиля пользователя при вводе некорректных данных»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка заполнения профиля пользователя при вводе некорректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при сохранении профиля без обязательных полей
Этапы теста	1 Открыть экран настройки профиля; 2 Не заполнять обязательные поля; 3 Нажать на кнопку «Сохранить профиль».
Тестовые данные	Имя: «»; Пол: не выбран; Дата рождения: не выбрана.
Ожидаемый результат	Вывод сообщений об обязательности ввода имени, выбора пола и даты рождения

Поле	Описание
Фактический результат	Вывод сообщений об обязательности ввода имени, выбора пола и даты рождения

На рисунке 2.8 изображен результат тестирования на проверку заполнения профиля пользователя при вводе некорректных данных.

Рисунок 2.8 – Форма заполнения профиля без обязательных данных

В таблице 2.7 представлен протокол тестирования «Проверка завершения и сохранения тренировки при вводе корректных данных».

Таблица 2.7 – Протокол тестирования «Проверка завершения и сохранения тренировки при вводе корректных данных»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка завершения и сохранения тренировки при вводе корректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при завершении тренировки и сохранении результата
Этапы теста	1 Выбрать тип тренировки; 2 Запустить тренировочную сессию; 3 Завершить тренировку; 4 Нажать на кнопку сохранения результата.
Тестовые данные	Тип активности: «Running»; Длительность: «15 секунд»; Маршрут: две и более точки геолокации на 1.48 км; Пользователь: «mewkfely@gmail.com».

Поле	Описание
Ожидаемый результат	Формирование экрана результата, сохранение записи в историю тренировок и отображение новой записи в списке тренировок
Фактический результат	Формирование экрана результата, сохранение записи в историю тренировок и отображение новой записи в списке тренировок

На рисунках 2.9 – 2.11 изображены результаты тестирования на завершение и сохранение тренировки при вводе корректных данных.

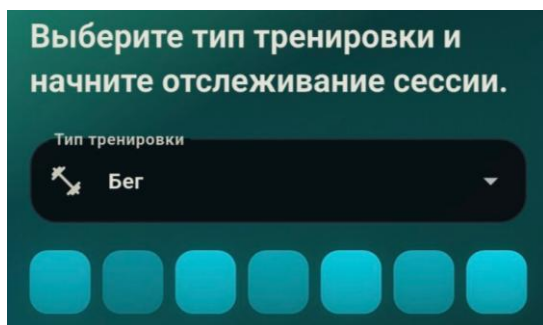


Рисунок 2.9 – Выбор типа тренировки перед запуском сессии

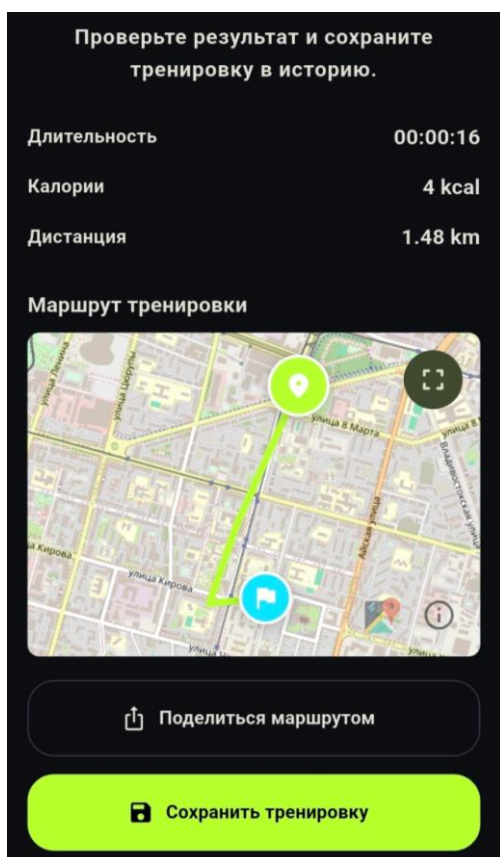


Рисунок 2.10 – Экран результата завершенной тренировки

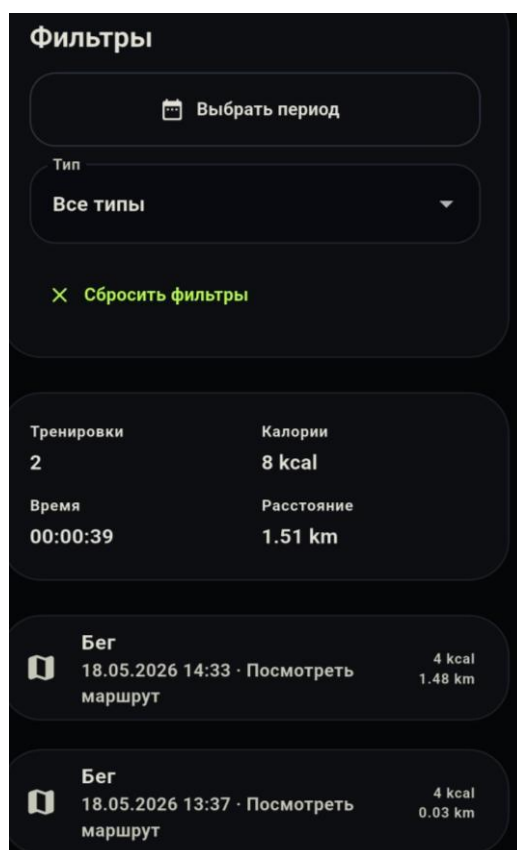


Рисунок 2.11 – Сохраненная запись в истории тренировок

Дополнительно проведено тестирование функций, появившихся в новой версии приложения.

Таблица 2.8 – Протокол тестирования «Настройка целей пользователя»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка изменения целей пользователя
Резюме испытания	Необходимо проверить сохранение дневной цели по шагам, калориям и целевого веса.
Предусловие	Пользователь зарегистрирован, авторизован и имеет заполненный профиль.
Этапы теста	1. Открыть экран целей. 2. Указать цель 12000 шагов, 2300 ккал и текущий вес. 3. Сохранить изменения. 4. Вернуться на главную панель.
Ожидаемый результат	Данные сохраняются в профиле, а карточки прогресса на главном экране пересчитываются по новым значениям.
Фактический результат	Программа выполнила сценарий корректно, критических ошибок не выявлено.
Статус	Пройден

Таблица 2.9 – Протокол тестирования «Работа шагомера»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка включения шагомера и отображения прогресса
Резюме испытания	Необходимо проверить запрос разрешений, запуск фонового сервиса и вывод количества шагов.
Предусловие	Пользователь зарегистрирован, авторизован и имеет заполненный профиль.
Этапы теста	1. Открыть раздел «Шаги». 2. Перейти в настройки. 3. Нажать кнопку включения отслеживания. 4. Вернуться на экран шагомера.
Ожидаемый результат	Приложение запрашивает разрешение на физическую активность, запускает сервис и отображает прогресс по дневной цели.
Фактический результат	Программа выполнила сценарий корректно, критических ошибок не выявлено.
Статус	Пройден

Таблица 2.10 – Протокол тестирования «Экспорт аналитики в PDF»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка формирования PDF-отчета по аналитике
Резюме испытания	Необходимо проверить выбор периода и создание PDF-файла с показателями активности.
Предусловие	Пользователь зарегистрирован, авторизован и имеет заполненный профиль.
Этапы теста	1. Открыть подробную аналитику. 2. Выбрать период. 3. Нажать кнопку экспорта PDF. 4. Открыть системное окно передачи файла.
Ожидаемый результат	Формируется PDF-отчет с шагами, калориями, тренировками и средними показателями за выбранный период.
Фактический результат	Программа выполнила сценарий корректно, критических ошибок не выявлено.
Статус	Пройден

Таблица 2.11 – Протокол тестирования «Работа AI-помощника»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка ответа AI-помощника по питанию
Резюме испытания	Необходимо проверить, что AI-помощник отвечает только на вопросы о питании и тренировках.
Предусловие	Пользователь зарегистрирован, авторизован и имеет заполненный профиль.

Поле	Описание
Этапы теста	1. Открыть AI-помощника. 2. Отправить вопрос о дневном рационе. 3. Отправить вопрос не по теме фитнеса.
Ожидаемый результат	На фитнес-вопрос формируется практический ответ, а на посторонний вопрос выводится отказ с указанием тематического ограничения.
Фактический результат	Программа выполнила сценарий корректно, критических ошибок не выявлено.
Статус	Пройден

Таблица 2.12 – Протокол тестирования «Социальные функции»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка добавления друга и открытия чата
Резюме испытания	Необходимо проверить создание приглашения, отправку заявки и переход в персональный чат.
Предусловие	Пользователь зарегистрирован, авторизован и имеет заполненный профиль.
Этапы теста	1. Открыть раздел друзей. 2. Создать ссылку-приглашение. 3. Отправить заявку по коду. 4. Принять заявку. 5. Открыть чат и отправить сообщение.
Ожидаемый результат	Друг появляется в списке, чат открывается, сообщение сохраняется и отображается участникам диалога.
Фактический результат	Программа выполнила сценарий корректно, критических ошибок не выявлено.
Статус	Пройден

Таблица 2.13 – Протокол тестирования «Тренерские материалы»

Поле	Описание
Дата теста	25.05.2026
Заголовок/название теста	Проверка назначения тренером упражнения или рецепта
Резюме испытания	Необходимо проверить создание материала тренером и доступ ученика к назначению.
Предусловие	Пользователь зарегистрирован, авторизован и имеет заполненный профиль.
Этапы теста	1. Авторизоваться под тренером. 2. Создать упражнение или рецепт. 3. Прикрепить медиафайл. 4. Назначить материал ученику. 5. Открыть раздел материалов у ученика.
Ожидаемый результат	Материал сохраняется в библиотеке тренера, назначается ученику и отображается в разделе тренерских материалов.
Фактический результат	Программа выполнила сценарий корректно, критических ошибок не выявлено.

Поле	Описание
Статус	Пройден

2.3 Руководство пользователя

Таблица 2.В – Основные действия пользователя в приложении

Операция	Действия пользователя	Результат
Регистрация	Открыть экран регистрации, ввести email, пароль и подтвердить действие	Система создает учетную запись и открывает заполнение профиля
Профиль	Указать имя, пол, дату рождения, город, рост, текущий и целевой вес	Данные сохраняются и используются в расчетах
Цели	Открыть настройки целей и задать шаги, калории и тип цели	Панель управления отображает процент выполнения
Тренировка	Выбрать тип тренировки, нажать запуск, после завершения сохранить результат	В истории появляется запись с временем, калориями и маршрутом
Питание	Открыть дневник питания, выбрать прием пищи, продукт и массу порции	Система показывает калории, белки, жиры и углеводы
Штрихкод	Открыть добавление продукта и ввести или отсканировать штрихкод	Приложение подставляет найденный продукт
Шаги	Открыть экран шагомера и разрешить доступ к датчику активности	Отображается количество шагов за день
Аналитика и PDF	Открыть аналитику, выбрать период и нажать экспорт	Формируется PDF-отчет для передачи или сохранения
AI-помощник	Открыть экран фитнес-помощника и задать вопрос о питании или тренировке	Пользователь получает краткую рекомендацию по контексту дневника
Друзья	Ввести код друга или принять входящую заявку	Друг появляется в списке и доступен для чата
Приватность	Открыть настройки лидерборда и выбрать видимые категории	Скрытые показатели не передаются друзьям
Тренер	Перейти в режим тренера, создать материал и назначить ученику	Ученик видит назначенный рецепт или тренировку

Руководство пользователя – это документ обзорного и одновременно справочного характера, основная цель которого помочь пользователям понять программу.

Руководство пользователя предназначено для пользователей мобильного приложения Liga Gym и тренеров, работающих с материалами и назначениями. В документе приведен порядок запуска приложения, регистрации, заполнения профиля и использования основных разделов системы.

Цель создания документа заключается в том, чтобы предоставить пользователю возможность самостоятельно решать свои прикладные задачи с помощью системы.

Руководство пользователя мобильного приложения

На рисунке 2.12 изображен splash-экран приложения «Liga Gym».

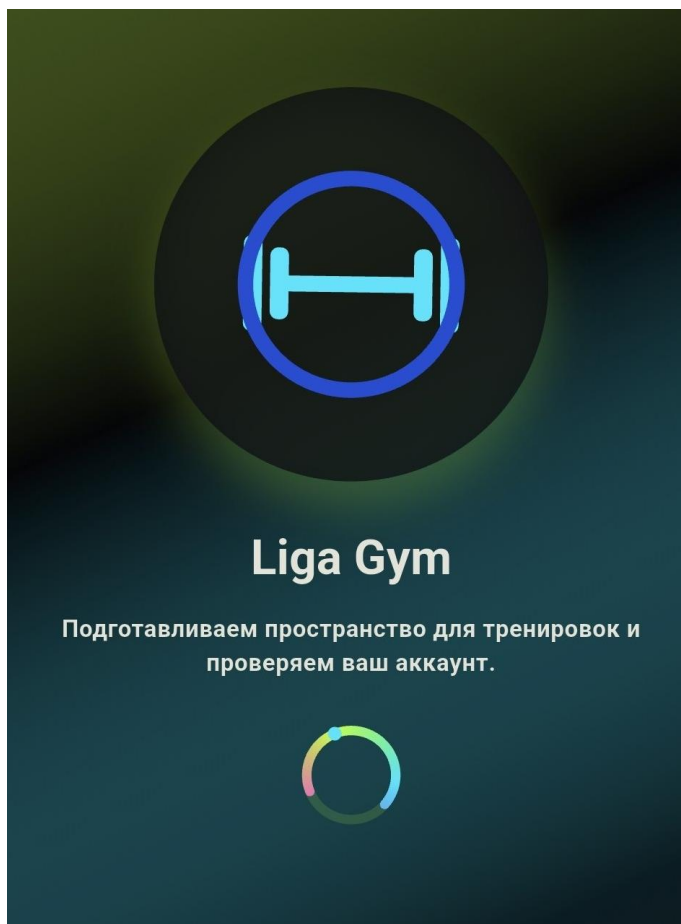


Рисунок 2.12 – Splash-экран приложения «Liga Gym»

После завершения загрузки пользователь автоматически переходит к экрану авторизации. Если пользователь еще не зарегистрирован, он может создать учетную запись, иначе достаточно выполнить вход в систему.

На рисунке 2.13 изображена страница «Авторизация».

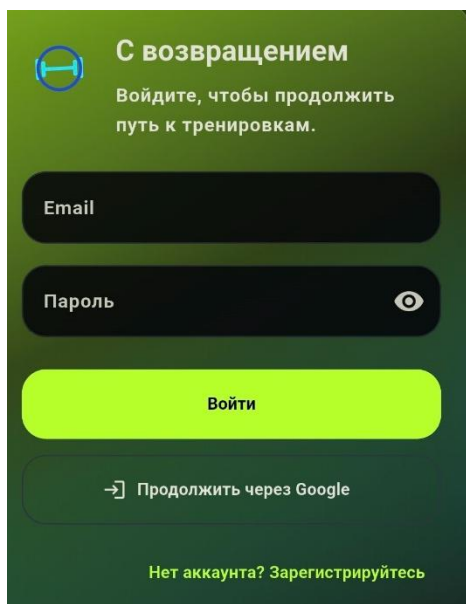


Рисунок 2.13 – Страница «Авторизация»

При нажатии на кнопку перехода к регистрации откроется страница создания учетной записи. После заполнения регистрационной формы и отправки данных приложение переводит пользователя к экрану заполнения профиля.

На рисунке 2.14 изображена страница «Регистрация».

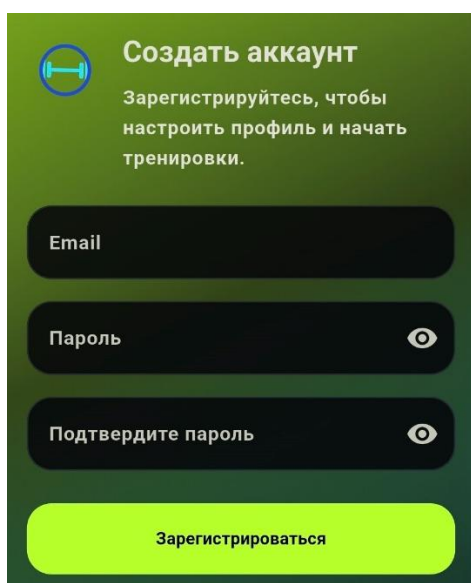


Рисунок 2.14 – Страница «Регистрация»

После успешной регистрации открывается экран первичной настройки профиля пользователя.

На рисунке 2.15 изображена страница «Настройка профиля».

Рисунок 2.15 – Страница «Настройка профиля»

На экране настройки профиля необходимо указать имя, выбрать пол и дату рождения.

На рисунке 2.16 изображена заполненная форма профиля пользователя.

Рисунок 2.16 – Заполненная форма профиля пользователя

После сохранения профиля открывается главный экран приложения, с которого доступны основные разделы системы.

На рисунке 2.17 изображена страница «Dashboard».

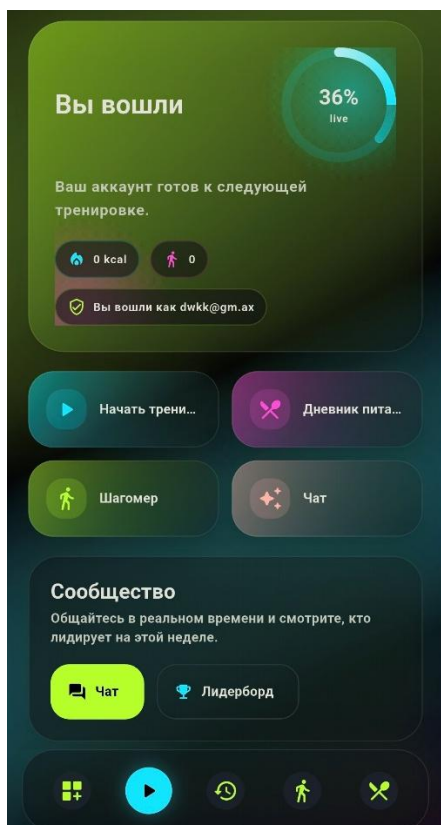


Рисунок 2.17 – Страница «Dashboard»

Для запуска тренировки пользователь нажимает кнопку начала тренировки на главном экране. Откроется страница выбора типа активности.

На рисунке 2.18 изображена страница «Старт тренировки».

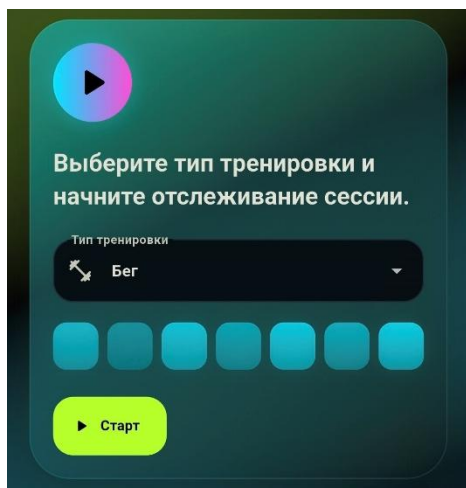


Рисунок 2.18 – Страница «Старт тренировки»

После выбора типа активности и нажатия кнопки запуска открывается экран активной тренировки.

На рисунке 2.19 изображена страница «Активная тренировка».

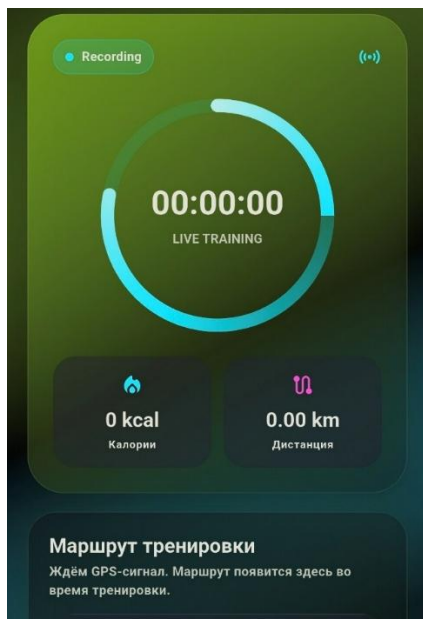


Рисунок 2.19 – Страница «Активная тренировка»

На рисунке 2.20 изображено состояние паузы тренировки на странице «Активная тренировка».

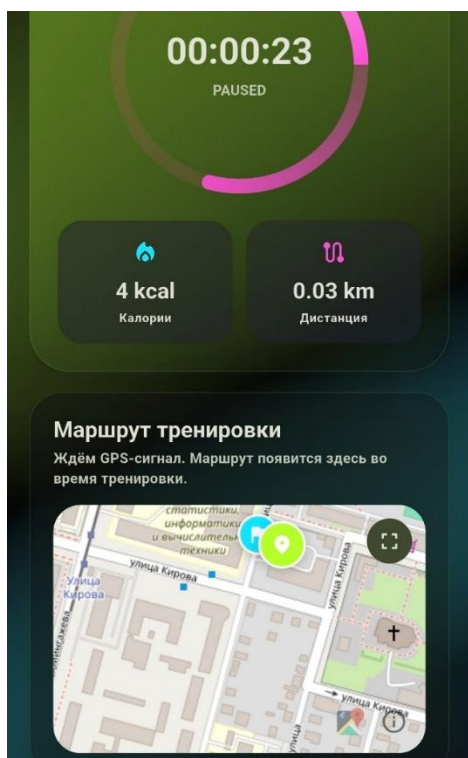


Рисунок 2.20 – Пауза тренировки на странице «Активная тренировка»

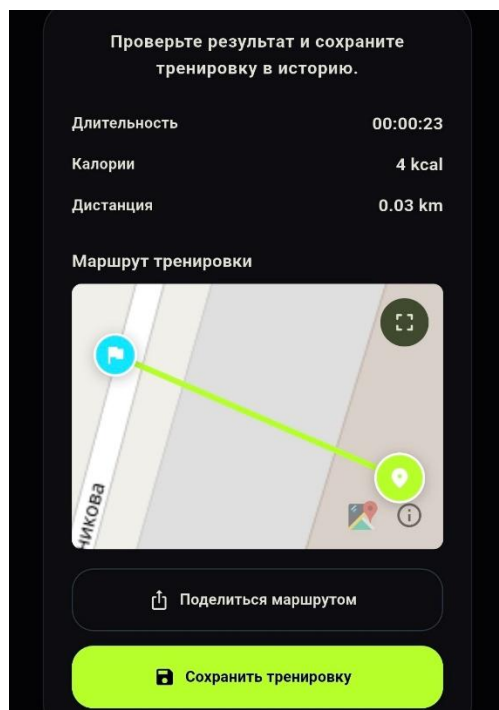


Рисунок 2.21 – Раздел «Результат тренировки»

На рисунке 2.22 изображен раздел «История тренировок».

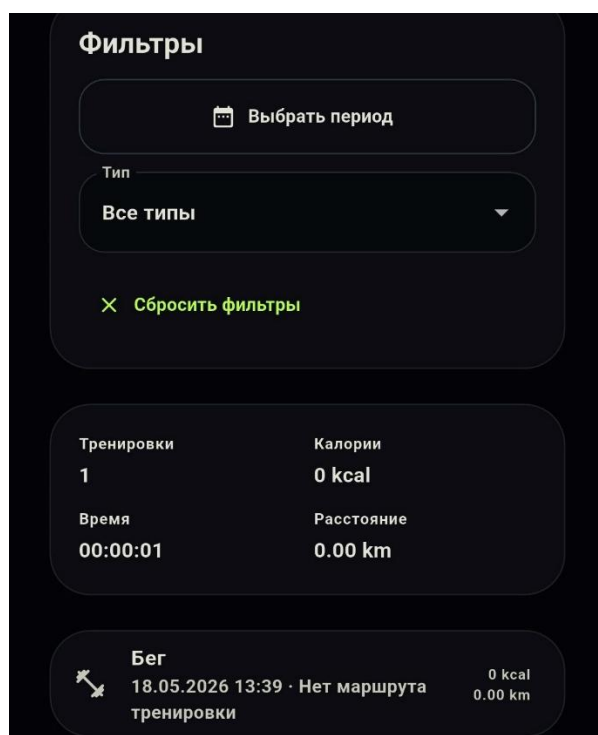


Рисунок 2.22 – Раздел «История тренировок»

Для фильтрации истории тренировок по дате необходимо нажать кнопку выбора даты в верхней части страницы. После выбора дня приложение отображает тренировки, относящиеся к указанному периоду.

На рисунке 2.23 изображено окно выбора даты для фильтрации истории тренировок.



Рисунок 2.23 – Окно выбора даты для истории тренировок

После выбора даты можно дополнительно использовать фильтрацию по типу тренировки через выпадающий список.

Чтобы включить фильтр по типу тренировки, нужно выбрать необходимый тип активности в выпадающем списке. Система отобразит только соответствующие записи.

На рисунке 2.24 изображен фильтр по типу активности на странице истории тренировок.

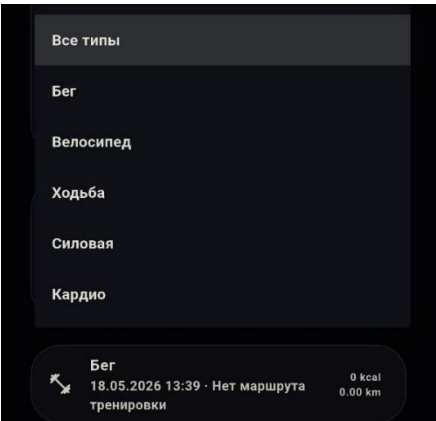


Рисунок 2.24 – Фильтрация истории тренировок по типу активности

При переходе в дневник питания в верхней части страницы отображается выбранная дата. Рядом с датой расположена кнопка добавления новой записи, с помощью которой пользователь вносит сведения о приеме пищи, порции и пищевой ценности продукта.

На рисунке 2.25 изображена страница «Дневник питания».

Рисунок 2.25 – Страница «Дневник питания»

Для выбора другой даты дневника необходимо нажать на кнопку календаря. После этого откроется стандартное окно выбора даты.

На рисунке 2.26 изображено окно выбора даты для дневника питания.

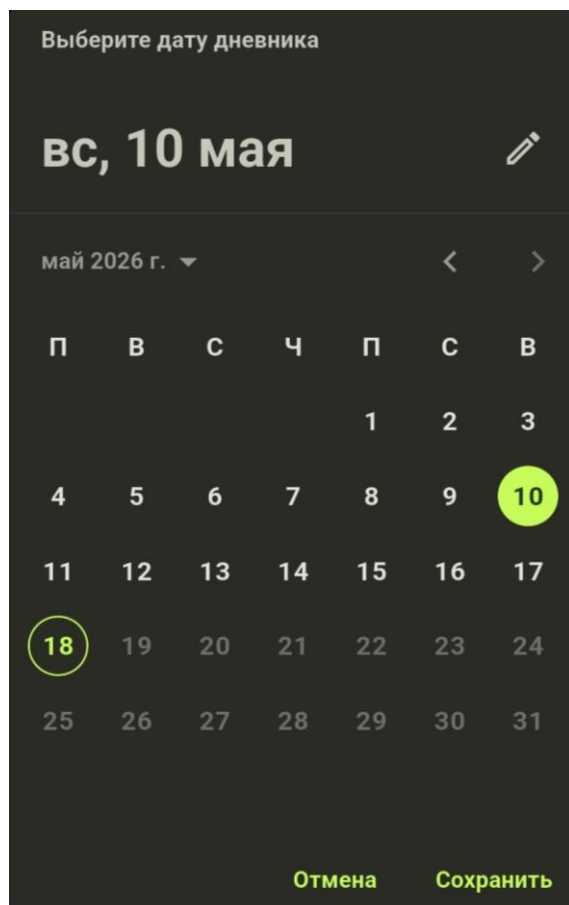


Рисунок 2.26 – Окно выбора даты для дневника питания

Ниже на этой же странице расположен раздел приема пищи «Завтрак» со списком записей и суммарными значениями БЖУ.

На рисунке 2.27 изображен раздел «Завтрак» страницы «Дневник питания».

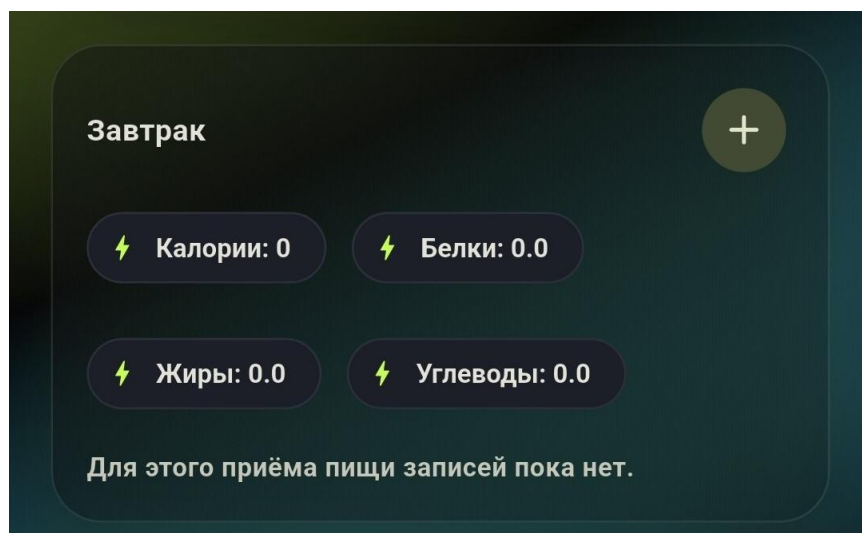


Рисунок 2.27 – Раздел «Завтрак» страницы «Дневник питания»

При нажатии на кнопку добавления записи открывается экран создания продукта в ручном режиме.

На рисунке 2.28 изображена страница «Добавление продукта» в ручном режиме.

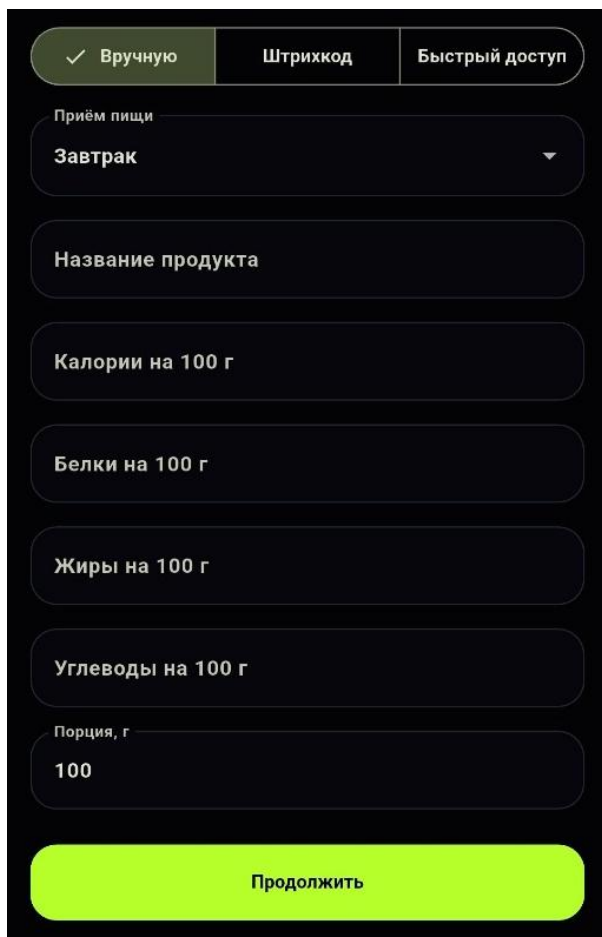


Рисунок 2.28 – Страница «Добавление продукта» в ручном режиме

В ручном режиме пользователь вводит данные о продукте вручную, указывая название продукта, его энергетическую ценность в килокалориях, а также содержание белков, жиров и углеводов на порцию. Дополнительно пользователь задаёт массу порции, что позволяет системе корректно пересчитывать и отображать пищевую ценность продукта в соответствии с введённым весом.

На рисунке 2.29 изображена заполненная форма ручного добавления продукта.

Рисунок 2.29 – Заполненная форма ручного добавления продукта

При переходе в режим добавления по штрихкоду отображается поле для ввода значения штрихкода.

На рисунке 2.30 изображен экран «Добавление продукта по штрихкоду».

Рисунок 2.30 – Экран «Добавление продукта по штрихкоду»

После ввода исходных данных и нажатия кнопки продолжения открывается страница карточки продукта, где рассчитываются калории и БЖУ для выбранной порции.

На рисунке 2.31 изображен экран карточки продукта.

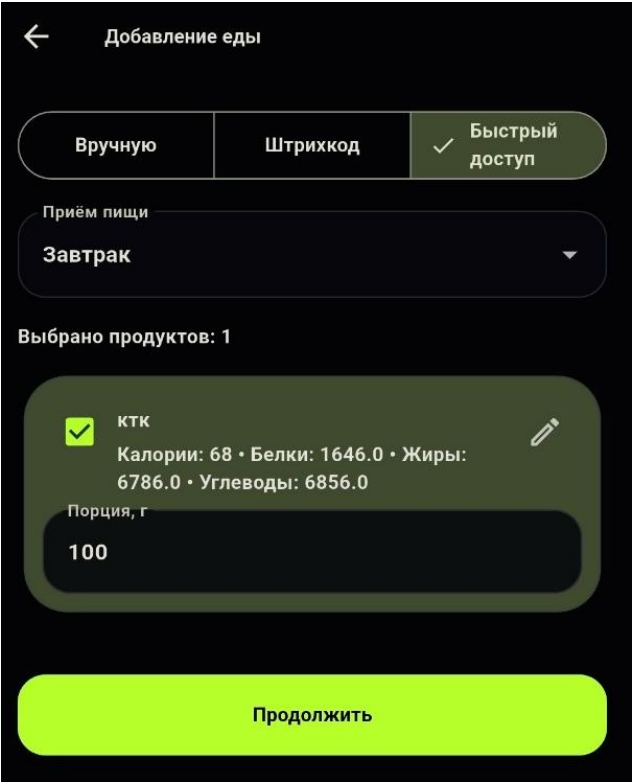


Рисунок 2.31 – Экран карточки продукта

После сохранения записи продукт отображается в разделе приема пищи выбранного дня.

На рисунке 2.32 изображена добавленная запись в дневнике питания.

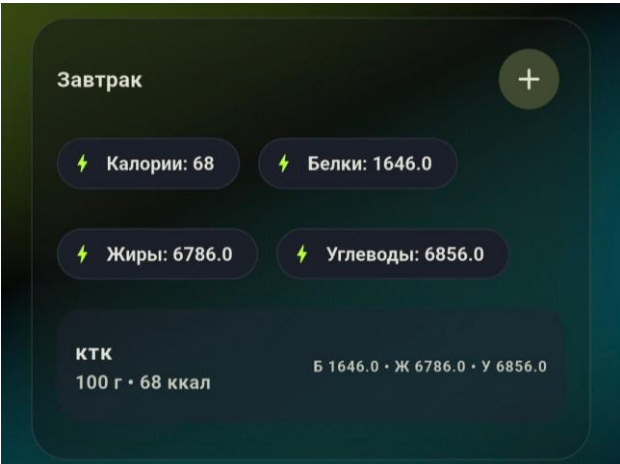


Рисунок 2.32 – Добавленная запись в дневнике питания

После добавления завтрака аналогичным образом можно заполнить данные для раздела «Обед».

Если при входе пользователь вводит некорректные данные, система отображает сообщения о нарушении правил валидации.

На рисунке 2.33 изображена страница «Авторизация» с сообщениями об ошибках.

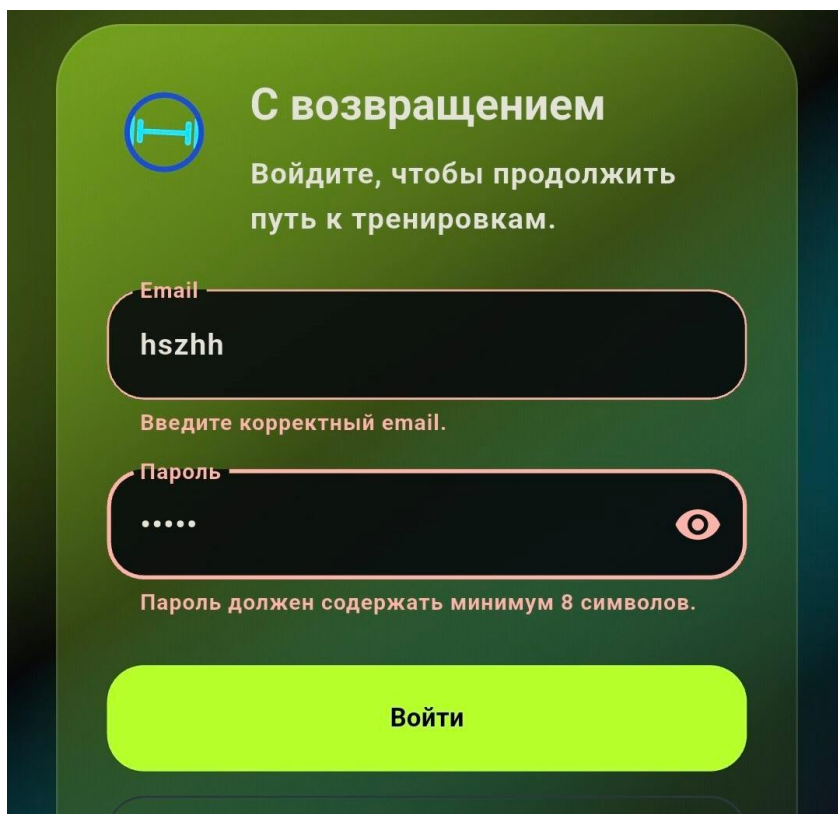
The image shows a login interface with a green background. At the top, there is a blue circular icon with a white dumbbell and the text "С возвращением" (Welcome back). Below this, it says "Войдите, чтобы продолжить путь к тренировкам." (Log in to continue your path to workouts). There are two input fields: "Email" and "Пароль" (Password). The "Email" field contains the text "hszhh" and has a red error message below it: "Введите корректный email." (Enter a correct email). The "Пароль" field contains five dots and has a red error message below it: "Пароль должен содержать минимум 8 символов." (Password must contain at least 8 characters). There is a red eye icon to the right of the password field. At the bottom, there is a large red button labeled "Войти" (Log in).

Рисунок 2.33 – Страница «Авторизация» с ошибками валидации

Если при регистрации пользователь вводит некорректные данные, приложение проверяет формат электронной почты, длину пароля и совпадение подтверждения пароля. При обнаружении ошибки система отображает сообщение под соответствующим полем или во всплывающем уведомлении.

На рисунке 2.34 изображена страница «Регистрация» с ошибками валидации.

Создать аккаунт

Зарегистрируйтесь, чтобы настроить профиль и начать тренировки.

Email

Введите корректный email.

Пароль

Пароль должен содержать минимум 8 символов.

Подтвердите пароль

Пароли не совпадают.

Зарегистрироваться

Рисунок 2.34 – Страница «Регистрация» с ошибками валидации

При попытке сохранить незаполненный профиль система проверяет обязательные поля: имя, пол и дату рождения. Если часть данных отсутствует, приложение не выполняет переход к следующему экрану и отображает сообщение о необходимости заполнения поля.

На рисунке 2.35 изображена форма профиля с ошибками заполнения.

Рисунок 2.35 – Ошибки заполнения формы профиля

Для завершения работы с приложением пользователь нажимает кнопку выхода на главной панели. После подтверждения действия приложение завершает текущую сессию и возвращает пользователя на экран авторизации.

На рисунке 2.36 изображена кнопка выхода на странице «Dashboard».



Рисунок 2.36 – Кнопка выхода на странице «Dashboard»

Дополнительные возможности приложения. Настройка целей и просмотр дневного прогресса

После входа пользователь попадает на главную панель. В верхней части отображается краткая сводка за текущий день: шаги, калории, общий прогресс и быстрые действия.

Для изменения целей необходимо открыть раздел целей, указать дневную норму шагов, дневную норму калорий, текущий вес и при необходимости целевой вес. После сохранения приложение пересчитывает прогресс на главной панели.

На рисунке 2.37 изображен экран настройки целей пользователя.



Рисунок 2.37 – Экран настройки целей пользователя

На рисунке 2.38 изображена панель дневного прогресса после сохранения целей.

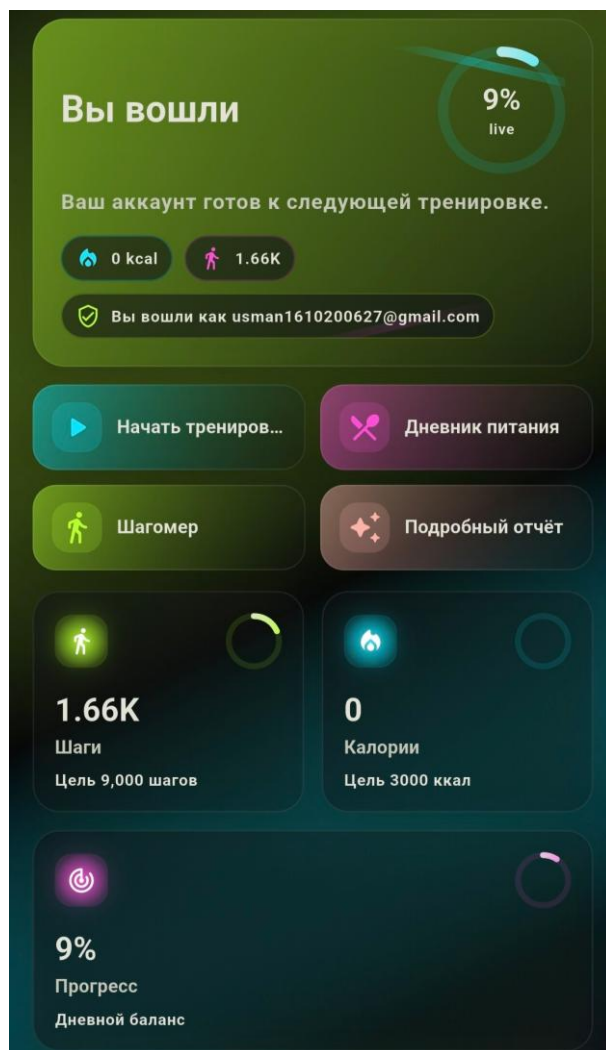


Рисунок 2.38 – Панель дневного прогресса пользователя

Раздел шагомера предназначен для контроля дневной активности. При первом запуске необходимо открыть настройки шагомера и разрешить приложению доступ к физической активности.

После включения отслеживания приложение запускает фоновый сервис и отображает текущее количество шагов, дневную цель, остаток до цели и визуальное кольцо прогресса.

На рисунке 2.39 изображен экран шагомера.



Рисунок 2.39 – Экран шагомера

На рисунке 2.40 изображены настройки отслеживания шагов.

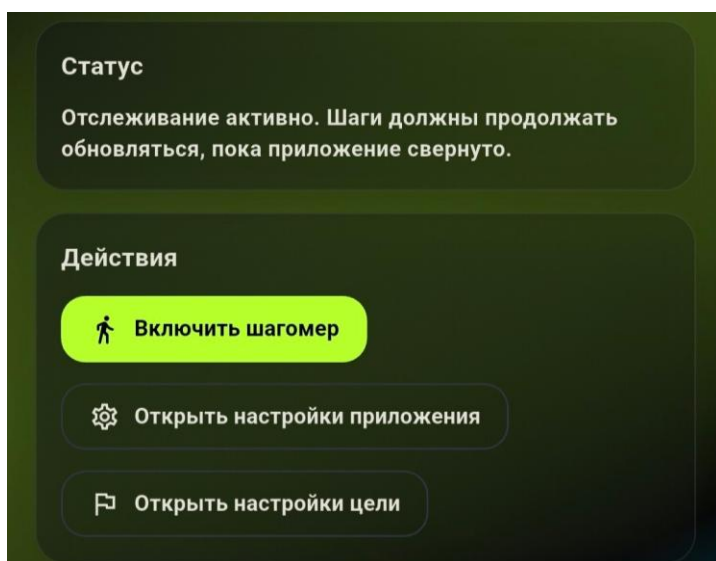


Рисунок 2.40 – Настройки шагомера

Подробная аналитика и экспорт отчета

Подробная аналитика открывается с главной панели. Пользователь выбирает начальную и конечную дату периода, после чего приложение рассчитывает суммарные шаги, калории, тренировки, средние дневные показатели и динамику веса.

При необходимости пользователь нажимает кнопку экспорта. Приложение формирует PDF-отчет и открывает системное окно передачи файла.

На рисунке 2.41 изображен экран подробной аналитики.

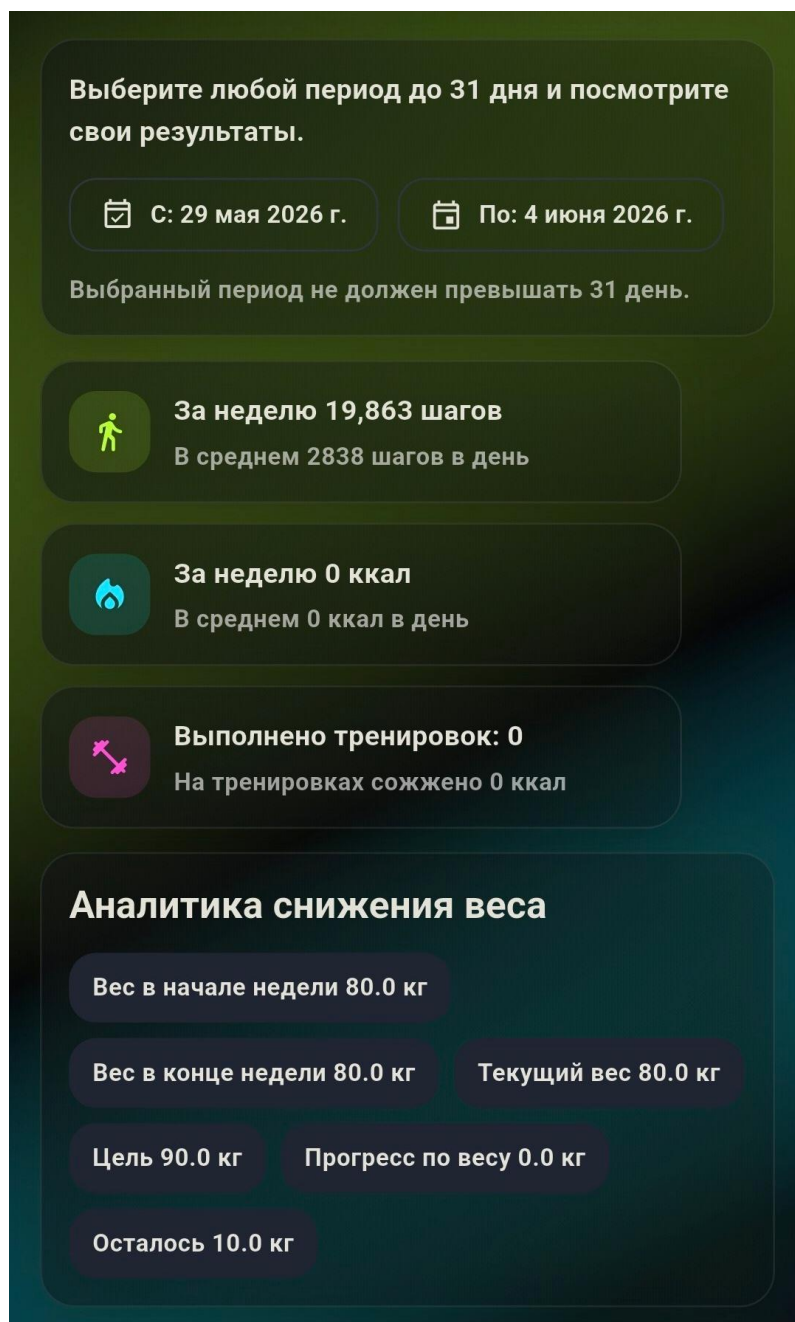


Рисунок 2.41 – Подробная аналитика за выбранный период

На рисунке 2.42 изображен сформированный PDF-отчет.

Liga Gym — отчёт по аналитике

Период: 29.05.2026 - 04.06.2026

Сводка

- За неделю 19,863 шагов
- За неделю 0 ккал
- В среднем 2838 шагов в день
- В среднем 0 ккал в день
- На тренировках сожжено 0 ккал
- Выполнено тренировок: 0
- Текущий вес 80.0 кг
- Прогресс по весу 0.0 кг

Результаты по дням

Дата	Шаги	Калории	Прогресс
29.05.2026	0	0	0%
30.05.2026	0	0	0%
31.05.2026	11999	0	50%
01.06.2026	1186	0	7%
02.06.2026	0	0	0%
03.06.2026	5016	0	28%
04.06.2026	1662	0	9%

Рисунок 2.42 – Экспорт аналитики в PDF

Использование AI-помощника

AI-помощник открывается с нижней панели навигации. Пользователь может спросить о рационе, тренировках, восстановлении или выборе нагрузки. При формировании ответа учитываются дневник питания, цель пользователя, дневная норма калорий и шагов.

Если вопрос не относится к питанию или тренировкам, помощник сообщает, что не может отвечать вне своей предметной области.

На рисунке 2.43 изображен экран AI-помощника.

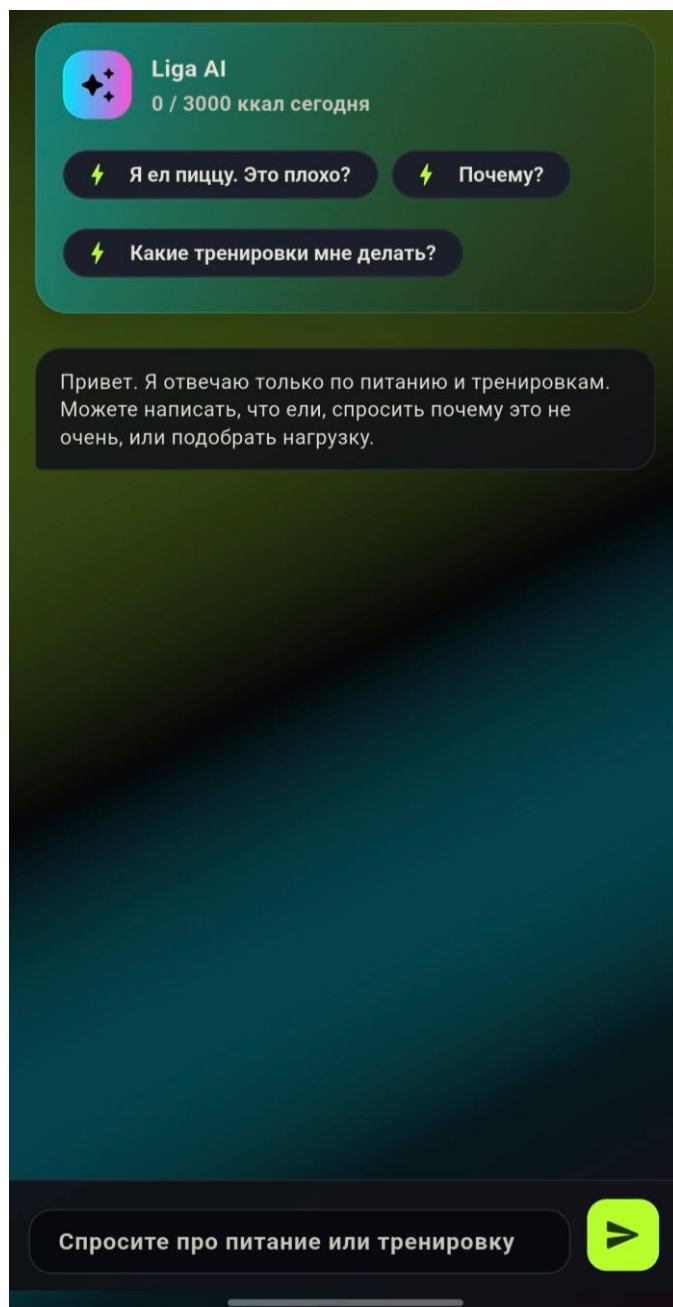


Рисунок 2.43 – AI-помощник Liga Gym

Друзья, рейтинг и чат

В разделе друзей пользователь создает ссылку-приглашение или вводит код другого пользователя. После принятия заявки друг появляется в списке, а его показатели могут отображаться в рейтинге активности.

Настройки приватности позволяют определить, какие категории данных доступны друзьям. Из карточки друга можно открыть персональный чат и отправить сообщение.

На рисунке 2.44 изображен раздел друзей и рейтинга.

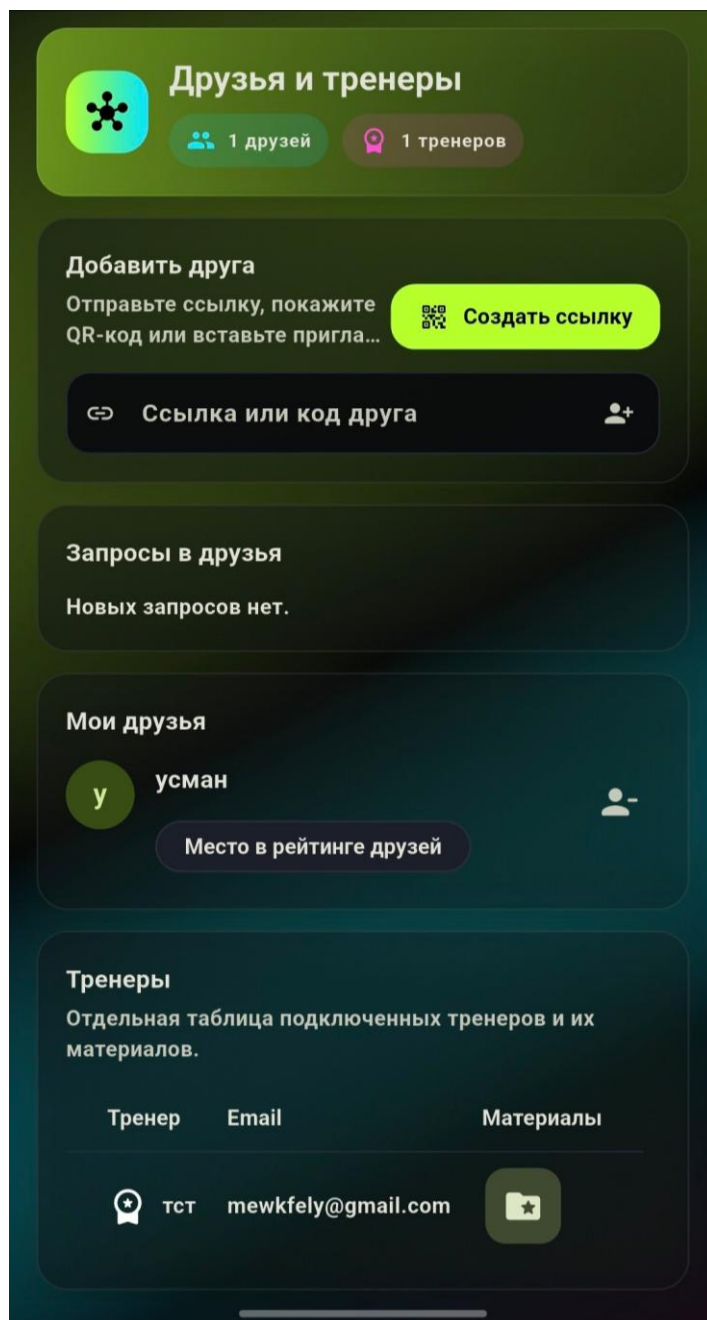


Рисунок 2.44 – Раздел друзей и рейтинга активности

На рисунке 2.45 изображен персональный чат пользователя.

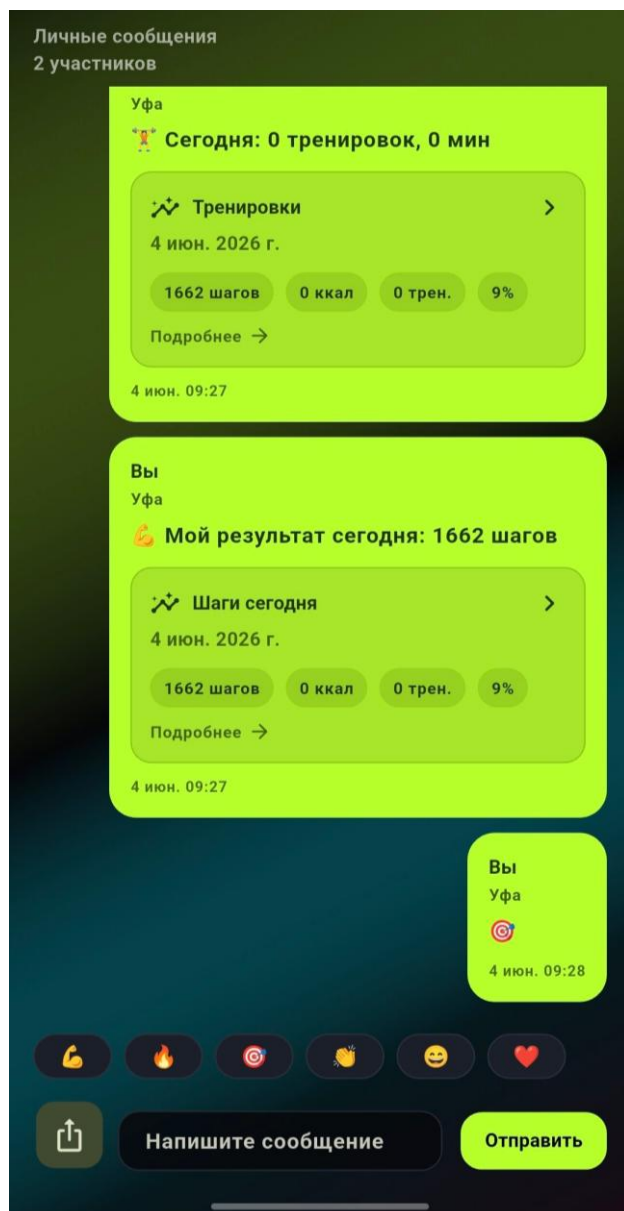


Рисунок 2.45 – Персональный чат с другом

Материалы тренера

Если пользователь связан с тренером, в разделе друзей отображается карточка тренера. При открытии карточки пользователь видит назначенные тренировки, рецепты, библиотеку упражнений и дополнительные материалы.

Тренер в своем режиме может создавать упражнения, рецепты, шаблоны тренировок, прикреплять изображения или видео и назначать материалы конкретным ученикам.

На рисунке 2.46 изображен раздел материалов тренера у ученика.

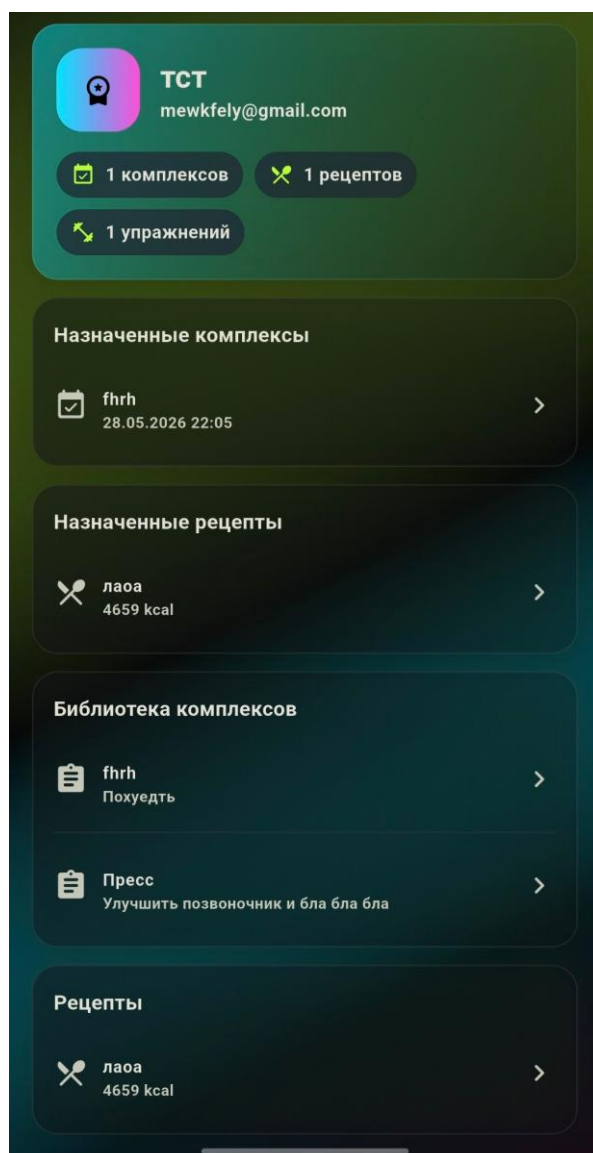


Рисунок 2.46 – Материалы тренера для ученика

На рисунке 2.47 изображена панель трен`ера.

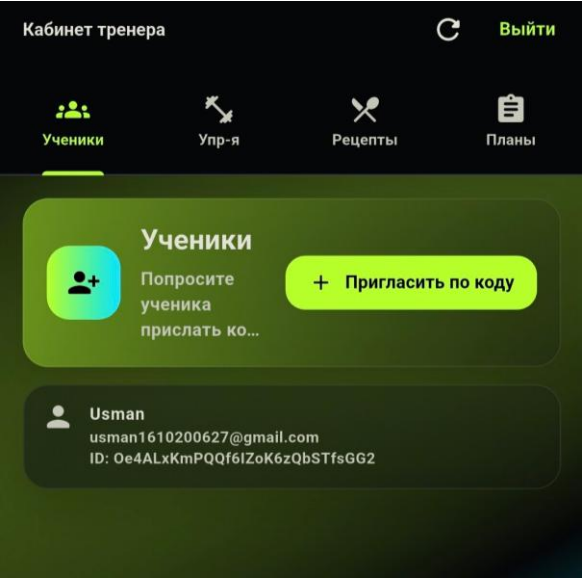


Рисунок 2.47 – Панель тренера

Основные сообщения, которые приложение выводит пользователю при проверке данных и обработке ошибок, приведены в таблице 2.14.

Таблица 2.14 – Сообщения пользователю

Сообщение	Причина	Действия пользователя
Введите корректный email.	Адрес электронной почты введен в неверном формате.	Проверить адрес электронной почты и повторить ввод.
Пароль должен содержать минимум 8 символов.	Введенный пароль не соответствует минимальной длине.	Указать пароль длиной не менее 8 символов.
Пароли не совпадают.	Пароль и подтверждение пароля различаются.	Повторно ввести пароль и подтверждение пароля.
Выберите дату рождения.	В профиле не указана дата рождения.	Выбрать дату рождения в форме настройки профиля.
Введите штрихкод.	Поиск продукта по штрихкоду запущен без значения кода.	Ввести штрихкод продукта и повторить поиск.
Продукт с таким штрихкодом не найден.	В каталоге отсутствует продукт с указанным штрихкодом.	Проверить штрихкод или добавить продукт вручную.
Сетевая ошибка.	Приложение не получило ответ от облачного сервиса.	Проверить подключение к сети и повторить операцию.
Не удалось отправить сообщение.	Сообщение чата не было передано в облачное хранилище.	Проверить сеть и отправить сообщение повторно.

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломного проекта цель разработки информационной системы мониторинга физической активности и питания пользователей достигнута. Разработано мобильное приложение Liga Gym, объединяющее учет профиля пользователя, целей, тренировок, маршрутов, питания, шаговой активности, аналитики, социальных функций, тренерских материалов и AI-помощника.

В ходе работы выполнено проектирование структуры данных, реализованы облачные коллекции Cloud Firestore и локальные таблицы SQLite, обеспечена авторизация через Firebase Authentication, хранение тренерских медиафайлов через Supabase Storage, управление состоянием через Riverpod и локальная работа с последующей синхронизацией данных.

Практическая значимость разработанной системы заключается в том, что пользователь получает единый мобильный фитнес-дневник: может контролировать вес и цели, фиксировать тренировки и питание, отслеживать шаги, анализировать прогресс, формировать отчет, общаться с друзьями и получать материалы от тренера. Система может использоваться как учебный программный продукт и как основа для дальнейшего развития мобильного сервиса персонального фитнес-мониторинга.

В ходе выполнения дипломного проекта была разработана и протестирована информационная система Liga Gym, предназначенная для мониторинга физической активности, питания, шаговой активности, персональных целей и взаимодействия пользователя с тренером.

В процессе работы выполнены основные задачи проекта: проведен анализ предметной области, определены входные и выходные данные, спроектирована структура базы данных, реализованы программные модули, подготовлен контрольный пример и расширенный протокол тестирования.

Программный продукт реализует регистрацию и авторизацию, заполнение профиля, настройку целей, запуск и завершение тренировок, сохранение

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		95

маршрута, ведение дневника питания, расчет калорий и БЖУ, учет шагов, аналитику прогресса, экспорт PDF-отчета, AI-помощника, друзей, чат и тренерские материалы.

Проведенное тестирование подтвердило корректность ключевых сценариев: регистрации, авторизации, добавления записи питания, заполнения профиля, сохранения тренировки, настройки целей, работы шагомера, экспорта аналитики, AI-помощника, социальных функций и назначения тренерских материалов.

Практическая значимость проекта заключается в том, что разработанное приложение может использоваться как персональный фитнес-дневник, учебный программный продукт или основа для дальнейшего развития сервиса сопровождения тренировочного процесса.

Таким образом, цель дипломного проекта достигнута: создано работоспособное мобильное приложение, объединяющее учет тренировок, питание, шаговую активность, аналитику, социальное взаимодействие и тренерское сопровождение в единой информационной системе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. – Москва : Стандартинформ, 2010. – Текст : непосредственный.

2 ГОСТ Р 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. – Москва : Стандартинформ, 2021. – Текст : непосредственный.

3 ГОСТ Р 59793-2021. Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – Москва : Российский институт стандартизации, 2021. – Текст : непосредственный.

4 Галиаскаров, Э. Г. Анализ и проектирование систем с использованием UML : учебник для вузов / Э. Г. Галиаскаров, А. С. Воробьев. – Москва : Юрайт, 2025. – 125 с. – URL: <https://urait.ru/bcode/568178> (дата обращения: 25.05.2026). – Текст : электронный.

5 Полуэктова, Н. Р. Разработка веб-приложений : учебник для среднего профессионального образования / Н. Р. Полуэктова. – 2-е изд. – Москва : Юрайт, 2025. – 204 с. – URL: <https://urait.ru/bcode/567621> (дата обращения: 25.05.2026). – Текст : электронный.

6 Стружкин, Н. П. Базы данных: проектирование : учебник для среднего профессионального образования / Н. П. Стружкин, В. В. Годин. – Москва : Юрайт, 2025. – 477 с. – URL: <https://urait.ru/bcode/566509> (дата обращения: 25.05.2026). – Текст : электронный.

7 Трофимов, В. В. Основы алгоритмизации и программирования : учебник для среднего профессионального образования / В. В. Трофимов, Т. А. Павловская. – 4-е изд. – Москва : Юрайт, 2025. – 108 с. – URL: <https://urait.ru/bcode/563861> (дата обращения: 25.05.2026). – Текст : электронный.

8 Федорова, А. Н. Основы информационной безопасности : учебное пособие для СПО / А. Н. Федорова, В. И. Симонов. – 2-е изд. – Москва : Юрайт, 2022. – 245 с.

9 Flutter Documentation : [официальная документация]. – URL: <https://docs.flutter.dev/> (дата обращения: 25.05.2026). – Текст : электронный.

10 Dart Documentation : [официальная документация]. – URL: <https://dart.dev/guides> (дата обращения: 25.05.2026). – Текст : электронный.

11 Firebase Authentication Documentation : [официальная документация]. – URL: <https://firebase.google.com/docs/auth> (дата обращения: 25.05.2026). – Текст : электронный.

12 Cloud Firestore Documentation : [официальная документация]. – URL: <https://firebase.google.com/docs/firestore> (дата обращения: 25.05.2026). – Текст : электронный.

13 Firebase Security Rules : [официальная документация]. – URL:
<https://firebase.google.com/docs/rules> (дата обращения: 25.05.2026). – Текст :
электронный.

14 FlutterFire Documentation : [официальная документация]. – URL:
<https://firebase.flutter.dev/> (дата обращения: 25.05.2026). – Текст :
электронный.

15 SQLite Documentation : [официальная документация]. – URL:
<https://www.sqlite.org/docs.html> (дата обращения: 25.05.2026). – Текст :
электронный.

16 Sqflite package : [документация библиотеки]. – URL:
<https://pub.dev/packages/sqflite> (дата обращения: 25.05.2026). – Текст :
электронный.

17 Riverpod Documentation : [официальная документация]. – URL:
<https://riverpod.dev/> (дата обращения: 25.05.2026). – Текст : электронный.

18 flutter_background_service package : [документация библиотеки]. – URL:
https://pub.dev/packages/flutter_background_service (дата обращения: 25.05.2026). – Текст : электронный.

19 Pedometer package : [документация библиотеки]. – URL:
<https://pub.dev/packages/pedometer> (дата обращения: 25.05.2026). – Текст :
электронный.

20 Geolocator package : [документация библиотеки]. – URL:
<https://pub.dev/packages/geolocator> (дата обращения: 25.05.2026). – Текст :
электронный.

21 Flutter Map Documentation : [документация библиотеки]. – URL:
<https://docs.fleaflet.dev/> (дата обращения: 25.05.2026). – Текст : электронный.

22 fl_chart package : [документация библиотеки]. – URL:
https://pub.dev/packages/fl_chart (дата обращения: 25.05.2026). – Текст :
электронный.

23 Supabase Flutter Documentation : [официальная документация]. – URL: <https://supabase.com/docs/reference/dart/introduction> (дата обращения: 25.05.2026). – Текст : электронный.

24 Pollinations AI Documentation : [документация сервиса]. – URL: <https://github.com/pollinations/pollinations> (дата обращения: 25.05.2026). – Текст : электронный.

25 Material Design 3 : [руководство по проектированию интерфейсов]. – URL: <https://m3.material.io/> (дата обращения: 25.05.2026). – Текст : электронный.

26 Android Developers Documentation : [официальная документация]. – URL: <https://developer.android.com/> (дата обращения: 25.05.2026). – Текст : электронный.

27 Apple Developer Documentation : [официальная документация]. – URL: <https://developer.apple.com/documentation/> (дата обращения: 25.05.2026). – Текст : электронный.

28 PDF package : [документация библиотеки]. – URL: <https://pub.dev/packages/pdf> (дата обращения: 25.05.2026). – Текст : электронный.

29 Share Plus package : [документация библиотеки]. – URL: https://pub.dev/packages/share_plus (дата обращения: 25.05.2026). – Текст : электронный.

30 Pub.dev : каталог пакетов Flutter и Dart. – URL: <https://pub.dev/> (дата обращения: 25.05.2026). – Текст : электронный.

ПРИЛОЖЕНИЕ А

Исходный код основных модулей программы

В приложении приведены основные фрагменты исходного кода мобильного приложения Liga Gym, подтверждающие реализацию авторизации, маршрутизации, тренировок, питания, аналитики, шагомера, AI-помощника, социальных функций и тренерского раздела.

А.1 Инициализация приложения

Файл: lib/main.dart

```
import 'package:flutter/widgets.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'app.dart';
import 'core/firebase/firebase_bootstrap.dart';
import 'core/providers/app_theme_provider.dart';
import 'core/notifications/app_notification_service.dart';
import 'core/providers/locale_provider.dart';
import 'core/providers/shared_preferences_provider.dart';
import 'core/supabase/supabase_bootstrap.dart';
import 'core/theme/app_theme.dart';
import 'features/steps/data/services/step_tracking_service.dart';
```

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
```

```
  // Инициализируем Firebase до старта приложения, чтобы Splash сразу
  видел auth-состояние.
```

```
  final firebaseBootstrap = await FirebaseBootstrapResult.initialize();
```

```

final supabaseBootstrap = await SupabaseBootstrapResult.initialize();
final sharedPreferences = await SharedPreferences.getInstance();
await AppNotificationService.initialize();
await StepTrackingService.configureBackgroundTracking();
final initialLocale = parseSavedLocale(
  sharedPreferences.getString('app_locale'),
);
final initialTheme = AppThemePalettes.byId(
  sharedPreferences.getString(appThemePreferenceKey),
);

runApp(
  ProviderScope(
    overrides: [
      firebaseBootstrapProvider.overrideWithValue(firebaseBootstrap),
      supabaseBootstrapProvider.overrideWithValue(supabaseBootstrap),
      sharedPreferencesProvider.overrideWithValue(sharedPreferences),
      initialAppLocaleProvider.overrideWithValue(initialLocale),
      initialAppThemeProvider.overrideWithValue(initialTheme),
    ],
    child: const LigaGymApp(),
  ),
);
}

```

A.2 Основной каркас приложения

Файл: lib/app.dart

```
import 'dart:async';
```

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		101

```
import 'dart:math' as math;
```

```
import 'package:confetti/confetti.dart';
```

```
import 'package:flutter/material.dart';
```

```
import 'package:flutter_background_service/flutter_background_service.dart';
```

```
import 'package:flutter_localizations/flutter_localizations.dart';
```

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
```

```
import 'package:permission_handler/permission_handler.dart';
```

```
import 'package:pedometer/pedometer.dart';
```

```
import 'core/firebase/firebase_bootstrap.dart';
```

```
import 'core/notifications/app_notification_service.dart';
```

```
import 'core/navigation/app_router.dart';
```

```
import 'core/navigation/app_routes.dart';
```

```
import 'core/offline/offline_sync_providers.dart';
```

```
import 'core/providers/app_theme_provider.dart';
```

```
import 'core/providers/locale_provider.dart';
```

```
import 'core/providers/shared_preferences_provider.dart';
```

```
import 'core/theme/app_theme.dart';
```

```
import 'features/auth/domain/entities/user_profile.dart';
```

```
import 'features/auth/presentation/providers/auth_providers.dart';
```

```
import 'features/dashboard/presentation/providers/dashboard_providers.dart';
```

```
import
```

```
'features/notifications/presentation/providers/reminder_settings_providers.dart';
```

```
import 'features/social/domain/entities/friend_request.dart';
```

```
import 'features/social/presentation/providers/social_providers.dart';
```

```
import 'features/steps/data/datasources/step_local_data_source.dart';
```

```
import 'features/steps/data/models/daily_step_count_model.dart';
```

```
import 'features/steps/data/services/step_tracking_service.dart';
```

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						102
Изм.	Лист	№ докум.	Подпись	Дата		

```

import 'features/steps/presentation/providers/step_providers.dart';
import
'features/water_tracker/presentation/providers/water_tracker_providers.dart';
import
'features/workout_completion/presentation/providers/workout_completion_providers.
dart';
import 'lib/app_localizations.dart';

class LigaGymApp extends ConsumerStatefulWidget {
  const LigaGymApp({super.key, this.locale});

  final Locale? locale;

  @override
  ConsumerState<LigaGymApp> createState() => _LigaGymAppState();
}

class _LigaGymAppState extends ConsumerState<LigaGymApp>
  with WidgetsBindingObserver {
  final GlobalKey<NavigatorState> _navigatorKey =
GlobalKey<NavigatorState>();
  StreamSubscription<Object?>? _authSubscription;
  StreamSubscription<Map<String, dynamic>?>? _stepUpdateSubscription;
  StreamSubscription<StepCount>? _foregroundPedometerSubscription;
  ProviderSubscription<AsyncValue<UserProfile?>>? _profileSubscription;
  ProviderSubscription<AsyncValue<List<FriendRequest>>>?
    _friendRequestsSubscription;
  late final ConfettiController _confettiController;

```

```

final StepLocalDataSource _stepLocalDataSource =
SqliteStepLocalDataSource();

bool _isCelebrationOpen = false;
String? _activeUserId;
Set<String> _knownIncomingFriendRequestIds = const <String>{};
bool _hasLoadedIncomingFriendRequests = false;

@override
void initState() {
  super.initState();
  _confettiController = ConfettiController(
    duration: const Duration(seconds: 3),
  );
  WidgetsBinding.instance.addObserver(this);
  WidgetsBinding.instance.addPostFrameCallback((_) {
    unawaited(
      ref.read(reminderSettingsControllerProvider.notifier).reschedule(),
    );
  });

  if (ref.read(firebaseBootstrapProvider).isConfigured) {
    _authSubscription = ref
      .read(firebaseAuthProvider)
      .authStateChanges()
      .listen((user) async {
        final previousUserId = _activeUserId;
        _activeUserId = user?.uid;
        _knownIncomingFriendRequestIds = const <String>{};
        _hasLoadedIncomingFriendRequests = false;
      });
  }
}

```

```

await ref

    .read(stepTrackingServiceProvider)
    .resetUserSession(
        previousUserId: previousUserId,
        nextUserId: user?.uid,
    );
ref.invalidate(currentAuthProvider);
ref.invalidate(currentUserProfileProvider);
ref.invalidate(dashboardAnalyticsProvider);
ref.invalidate(todayStepCountProvider);
ref.invalidate(stepGoalProvider);
ref.invalidate(stepGoalCelebrationPendingProvider);
ref.invalidate(stepTrackingStatusProvider);
ref.invalidate(incomingFriendRequestsProvider);
ref.invalidate(reminderSettingsControllerProvider);
ref.invalidate(waterTrackerControllerProvider);
ref.invalidate(workoutCompletionControllerProvider);

if (user == null) {
    unawaited(ref.read(stepTrackingServiceProvider).stopTracking());
    unawaited(_stopForegroundStepSync());
    unawaited(AppNotificationService.cancelDailyReminders());
    return;
}

unawaited(
    ref
        .read(stepTrackingServiceProvider)
        .ensureTracking(userId: user.uid),

```

```

);
final email = user.email ?? "";
unawaited(
  ref
    .read(ensureLeaderboardEntryUseCaseProvider)
    .call(
      userId: user.uid,
      fallbackName: user.displayName ?? email.split('@').first,
      fallbackEmail: email,
    ),
);
unawaited(_startForegroundStepSync(user.uid));
unawaited(
  ref
    .read(appOfflineSyncCoordinatorProvider)
    .syncDataWithServer(userId: user.uid),
);
unawaited(
  ref
    .read(reminderSettingsControllerProvider.notifier)
    .reschedule(),
);
unawaited(_refreshUserScopedState(userId: user.uid));
});
}

_profileSubscription = ref.listenManual<AsyncValue<UserProfile?>>(
  currentUserProfileProvider,
  (_, next) {

```

```

next.whenData((profile) {
  if (profile == null) {
    return;
  }

  unawaited(
    ref
      .read(stepTrackingServiceProvider)
      .saveDailyGoal(
        userId: profile.userId,
        goal: profile.dailyStepGoal,
      ),
  );
});
},
);

_friendRequestsSubscription = ref
  .listenManual<AsyncValue<List<FriendRequest>>>(
    incomingFriendRequestsProvider,
    (_, next) {
      next.whenData(_handleIncomingFriendRequests);
    },
  );

if (isStepTrackingSupportedPlatform) {
  _stepUpdateSubscription = FlutterBackgroundService()
    .on(stepTrackingUpdateEvent)
    .listen((payload) {

```

```

        ref.invalidate(dashboardAnalyticsProvider);
        ref.invalidate(todayStepCountProvider);
        ref.invalidate(stepGoalCelebrationPendingProvider);
        final goalReachedNow = payload?['goalReachedNow'] == true;
        if (goalReachedNow) {
            final l10n = _navigatorKey.currentContext == null
                ? null
                : AppLocalizations.of(_navigatorKey.currentContext!);
            if (l10n != null) {
                unawaited(
                    AppNotificationService.showStepGoalReached(
                        title: 'Liga Gym',
                        body: l10n.stepGoalReachedMessage,
                    ),
                );
            }
            unawaited(_showPendingStepCelebration());
        });
    }
}

```

```

@override
void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    _authSubscription?.cancel();
    _stepUpdateSubscription?.cancel();
    _foregroundPedometerSubscription?.cancel();
    _profileSubscription?.close();
}

```

```

    _friendRequestsSubscription?.close();
    _confettiController.dispose();
    super.dispose();
}

```

```

@override

```

```

void didChangeAppLifecycleState(AppLifecycleState state) {
    if (state == AppLifecycleState.resumed) {
        ref.invalidate(dashboardAnalyticsProvider);
        ref.invalidate(stepGoalCelebrationPendingProvider);
        unawaited(_showPendingStepCelebration());
    }
}

```

```

Future<void> _showPendingStepCelebration() async {
    if (_isCelebrationOpen) {
        return;
    }
}

```

```

final sharedPreferences = ref.read(sharedPreferencesProvider);
final userId = _activeUserId;
if (userId == null) {
    return;
}
final pendingDate = sharedPreferences?.getString(
    stepGoalCelebrationPendingDateKey(userId),
);
final todayKey = buildStepDateKey(DateTime.now());
if (pendingDate != todayKey) {

```

```

        return;
    }

    final dialogContext = _navigatorKey.currentContext;
    if (dialogContext == null) {
        return;
    }

    _isCelebrationOpen = true;
    _confettiController.play();
    final l10n = AppLocalizations.of(dialogContext!);

    await showDialog<void>(
        context: dialogContext,
        barrierDismissible: true,
        builder: (context) {
            return Stack(
                alignment: Alignment.topCenter,
                children: [
                    AlertDialog(
                        title: Text(l10n.stepGoalReachedTitle),
                        content: Text(l10n.stepGoalReachedMessage),
                        actions: [
                            TextButton(
                                onPressed: () => Navigator.of(context).pop(),
                                child: Text(l10n.commonContinue),
                            ),
                        ],
                    ),
                ],
            ),
        ),
    );

```

```

IgnorePointer(
    child: ConfettiWidget(
        confettiController: _confettiController,
        blastDirection: math.pi / 2,
        emissionFrequency: 0.05,
        numberOfParticles: 24,
        maxBlastForce: 18,
        minBlastForce: 8,
        gravity: 0.3,
        shouldLoop: false,
    ),
),
],
);
},
);

```

```

await ref
    .read(stepScreenControllerProvider.notifier)
    .markGoalCelebrationSeen();
_isCelebrationOpen = false;
}

```

```

void _handleIncomingFriendRequests(List<FriendRequest> requests) {
    if (_activeUserId == null) {
        _knownIncomingFriendRequestIds = const <String>{};
        _hasLoadedIncomingFriendRequests = false;
        return;
    }
}

```

```

final requestIds = requests.map((request) => request.id).toSet();
if (!_hasLoadedIncomingFriendRequests) {
    _knownIncomingFriendRequestIds = requestIds;
    _hasLoadedIncomingFriendRequests = true;
    return;
}

```

```

final newRequests = requests
    .where(
        (request) => !_knownIncomingFriendRequestIds.contains(request.id),
    )
    .toList(growable: false);
_knownIncomingFriendRequestIds = requestIds;
if (newRequests.isEmpty) {
    return;
}

```

```

final context = _navigatorKey.currentContext;

```

```

final isRu = context == null

```

```

    ? false

```

```

    : Localizations.localeOf(context).languageCode == 'ru';

```

```

final newestRequest = newRequests.first;

```

```

final baseBody = isRu

```

```

    ? 'Вам пришел запрос в друзья'

```

```

    : 'You received a friend request';

```

```

final requesterName = newestRequest.fromDisplayName.trim();

```

```

final body = requesterName.isEmpty ? baseBody : '$baseBody:
$requesterName';

```

```

unawaited(
    AppNotificationService.showFriendRequestReceived(
        title: 'Liga Gym',
        body: body,
    ),
);
}

```

```

Future<void> _startForegroundStepSync(String userId) async {
    if (!isStepTrackingSupportedPlatform) {
        return;
    }

```

```

    final permission = await Permission.activityRecognition.status;
    if (!permission.isGranted) {
        return;
    }

```

```

    final todaySteps = await _stepLocalDataSource.loadStepsForDate(
        userId: userId,
        date: DateTime.now(),
    );
    await _syncFriendSteps(userId: userId, stepsCount: todaySteps);

```

```

    await _foregroundPedometerSubscription?.cancel();
    _foregroundPedometerSubscription = Pedometer.stepCountStream.listen((
        event,
    ) async {

```

```

        await _stepLocalDataSource.recordSensorReading(
            userId: userId,
            sensorSteps: event.steps,
            recordedAt: event.timeStamp,
        );

        final todaySteps = await _stepLocalDataSource.loadStepsForDate(
            userId: userId,
            date: event.timeStamp,
        );

        await _syncFriendSteps(userId: userId, stepsCount: todaySteps);
        await _handleGoalReachedOnMain(todaySteps);

        ref.invalidate(todayStepCountProvider);
        ref.invalidate(dashboardAnalyticsProvider);
    });
}

Future<void> _syncFriendSteps({
    required String userId,
    required int stepsCount,
}) async {
    try {
        await ref
            .read(updateLeaderboardStepsUseCaseProvider)
            .call(userId: userId, stepsCount: stepsCount);
    } on Object {
        // Social step sharing should not interrupt local step tracking.
    }
}

```

```
}
```

```
Future<void> _stopForegroundStepSync() async {  
    await _foregroundPedometerSubscription?.cancel();  
    _foregroundPedometerSubscription = null;  
}
```

```
Future<void> _handleGoalReachedOnMain(int todaySteps) async {  
    final sharedPreferences = ref.read(sharedPreferencesProvider);  
    final userId = _activeUserId;  
    if (sharedPreferences == null || userId == null) {  
        return;  
    }
```

```
    final stepGoal =  
        sharedPreferences.getInt(stepTrackingGoalKey(userId)) ?? 10000;  
    if (todaySteps < stepGoal) {  
        return;  
    }
```

```
    final todayKey = buildStepDateKey(DateTime.now());  
    final lastCelebratedDate = sharedPreferences.getString(  
        stepGoalCelebratedDateKey(userId),  
    );  
    if (lastCelebratedDate == todayKey) {  
        return;  
    }
```

```
    await sharedPreferences.setString(  

```

```

        stepGoalCelebratedDateKey(userId),
        todayKey,
    );
    await sharedPreferences.setString(
        stepGoalCelebrationPendingDateKey(userId),
        todayKey,
    );
    final l10n = _navigatorKey.currentContext == null
        ? null
        : AppLocalizations.of(_navigatorKey.currentContext!);
    if (l10n != null) {
        await AppNotificationService.showStepGoalReached(
            title: 'Liga Gym',
            body: l10n.stepGoalReachedMessage,
        );
    }
}

@override
Widget build(BuildContext context) {
    final locale = ref.watch(appLocaleProvider) ?? widget.locale;
    final appPalette = ref.watch(appThemeProvider);

    return MaterialApp(
        navigatorKey: _navigatorKey,
        debugShowCheckedModeBanner: false,
        locale: locale,
        onGenerateTitle: (context) => AppLocalizations.of(context)!.appTitle,
        theme: buildLigaGymTheme(appPalette),
    );
}

```

```

themeAnimationDuration: const Duration(milliseconds: 450),
themeAnimationCurve: Curves.easeOutCubic,
localizationsDelegates: const [
  AppLocalizations.delegate,
  GlobalMaterialLocalizations.delegate,
  GlobalWidgetsLocalizations.delegate,
  GlobalCupertinoLocalizations.delegate,
],
supportedLocales: AppLocalizations.supportedLocales,
initialRoute: AppRoutes.splash,
onGenerateRoute: AppRouter.onGenerateRoute,
);
}

```

```

Future<void> _refreshUserScopedState({required String userId}) async {
  ref.invalidate(currentAuthUserProvider);
  ref.invalidate(currentUserProfileProvider);
  ref.invalidate/dashboardAnalyticsProvider);
  ref.invalidate(todayStepCountProvider);
  ref.invalidate(stepGoalProvider);
  ref.invalidate(stepGoalCelebrationPendingProvider);
  ref.invalidate(stepTrackingStatusProvider);
  ref.invalidate(incomingFriendRequestsProvider);

  final profile = await ref.read(loadUserProfileUseCaseProvider).call(userId);
  await ref
    .read(stepTrackingServiceProvider)
    .saveDailyGoal(userId: profile.userId, goal: profile.dailyStepGoal);
}

```

```

final currentUser = ref.read(firebaseAuthProvider).currentUser;
if (currentUser != null) {
  final email = currentUser.email ?? "";
  await ref
    .read(ensureLeaderboardEntryUseCaseProvider)
    .call(
      userId: currentUser.uid,
      fallbackName: currentUser.displayName ?? email.split('@').first,
      fallbackEmail: email,
    );
}
}
}

```

А.3 Маршрутизация приложения

Файл: lib/core/navigation/app_routes.dart

```

abstract final class AppRoutes {
  static const splash = '/';
  static const login = '/login';
  static const register = '/register';
  static const profileSetup = '/profile-setup';
  static const profile = '/profile';
  static const dashboard = '/dashboard';
  static const aiCoach = '/ai/fitness-coach';
  static const coachDashboard = '/coach';
  static const todayOverview = '/dashboard/today-overview';
  static const goalSettings = '/dashboard/goal-settings';
  static const dashboardAnalyticsDetails = '/dashboard/analytics-details';
}

```

```

static const chat = '/social/chat';
static const chatRoom = '/social/chat-room';
static const friends = '/social/friends';
static const leaderboard = '/social/leaderboard';
static const leaderboardSettings = '/social/leaderboard/settings';
static const trainerMaterials = '/social/trainer-materials';
static const trainerMaterialDetails = '/social/trainer-material-details';
static const workoutList = '/workouts';
static const exerciseLibrary = '/workouts/exercises';
static const workoutHistory = '/workouts/history';
static const startWorkout = '/workouts/start';
static const activeWorkout = '/workouts/active';
static const workoutResult = '/workouts/result';
static const foodDiary = '/nutrition/diary';
static const addFood = '/nutrition/add';
static const productDetails = '/nutrition/product-details';
static const stepCounter = '/steps';
static const stepSettings = '/steps/settings';
}

```

А.4 Генератор маршрутов

Файл: lib/core/navigation/app_router.dart

```

import 'package:flutter/material.dart';

import '../features/auth/presentation/screens/login_screen.dart';
import '../features/auth/presentation/screens/profile_screen.dart';
import '../features/auth/presentation/screens/profile_setup_screen.dart';
import '../features/auth/presentation/screens/register_screen.dart';

```

```

import '../features/auth/presentation/screens/splash_screen.dart';
import '../features/ai_coach/presentation/screens/ai_coach_screen.dart';
import '../features/coach/presentation/screens/coach_dashboard_screen.dart';
import
'../features/dashboard/presentation/screens/dashboard_analytics_details_screen.dart'
;

import '../features/dashboard/presentation/screens/dashboard_screen.dart';
import '../features/dashboard/presentation/screens/goal_settings_screen.dart';
import
'../features/dashboard/presentation/screens/today_overview_screen.dart';
import
'../features/dashboard/presentation/utils/goal_settings_route_arguments.dart';
import
'../features/exercises/presentation/screens/exercise_library_screen.dart';
import '../features/nutrition/presentation/screens/add_food_screen.dart';
import '../features/nutrition/presentation/screens/food_diary_screen.dart';
import '../features/nutrition/presentation/screens/product_details_screen.dart';
import '../features/nutrition/presentation/utils/nutrition_route_arguments.dart';
import '../features/social/presentation/screens/chat_screen.dart';
import '../features/social/presentation/screens/chat_room_screen.dart';
import '../features/social/presentation/screens/leaderboard_screen.dart';
import '../features/social/presentation/screens/trainer_materials_screen.dart';
import '../features/social/presentation/utils/chat_room_route_arguments.dart';
import
'../features/social/presentation/utils/trainer_materials_route_arguments.dart';
import '../features/steps/presentation/screens/step_counter_screen.dart';
import '../features/steps/presentation/screens/step_settings_screen.dart';
import '../features/workout/presentation/screens/active_workout_screen.dart';
import '../features/workout/presentation/screens/start_workout_screen.dart';

```

```

import
'../../features/workout/presentation/screens/workout_history_screen.dart';
import ' ../../features/workout/presentation/screens/workout_list_screen.dart';
import ' ../../features/workout/presentation/screens/workout_result_screen.dart';
import 'app_routes.dart';

abstract final class AppRouter {
  static Route<dynamic> onGenerateRoute(RouteSettings settings) {
    switch (settings.name) {
      case AppRoutes.splash:
        return _buildRoute(const SplashScreen(), settings);
      case AppRoutes.login:
        return _buildRoute(const LoginScreen(), settings);
      case AppRoutes.register:
        return _buildRoute(const RegisterScreen(), settings);
      case AppRoutes.profileSetup:
        return _buildRoute(const ProfileSetupScreen(), settings);
      case AppRoutes.profile:
        return _buildRoute(const ProfileScreen(), settings);
      case AppRoutes.dashboard:
        return _buildRoute(const DashboardScreen(), settings);
      case AppRoutes.aiCoach:
        return _buildRoute(
          AiCoachScreen(
            arguments: settings.arguments as AiCoachRouteArguments?,
          ),
          settings,
        );
      case AppRoutes.coachDashboard:

```

```

        return _buildRoute(const CoachDashboardScreen(), settings);
    case AppRoutes.todayOverview:
        return _buildRoute(const TodayOverviewScreen(), settings);
    case AppRoutes.goalSettings:
        final arguments =
            settings.arguments as GoalSettingsRouteArguments? ??
            const GoalSettingsRouteArguments(
                section: GoalSettingsSection.progress,
            );
        return _buildRoute(GoalSettingsScreen(arguments: arguments), settings);
    case AppRoutes.dashboardAnalyticsDetails:
        return _buildRoute(const DashboardAnalyticsDetailsScreen(), settings);
    case AppRoutes.chat:
        return _buildRoute(const ChatScreen(), settings);
    case AppRoutes.chatRoom:
        final arguments = settings.arguments as ChatRoomRouteArguments;
        return _buildRoute(ChatRoomScreen(arguments: arguments), settings);
    case AppRoutes.friends:
        return _buildRoute(const SocialHubScreen(), settings);
    case AppRoutes.leaderboard:
        return _buildRoute(const LeaderboardScreen(), settings);
    case AppRoutes.leaderboardSettings:
        return _buildRoute(const LeaderboardSettingsScreen(), settings);
    case AppRoutes.trainerMaterials:
        final arguments = settings.arguments as TrainerMaterialsRouteArguments;
        return _buildRoute(
            TrainerMaterialsScreen(arguments: arguments),
            settings,
        );

```

```

case AppRoutes.trainerMaterialDetails:
    final arguments =
        settings.arguments as TrainerMaterialDetailsRouteArguments;
    return _buildRoute(
        TrainerMaterialDetailsScreen(arguments: arguments),
        settings,
    );
case AppRoutes.workoutList:
    return _buildRoute(const WorkoutListScreen(), settings);
case AppRoutes.exerciseLibrary:
    return _buildRoute(const ExerciseLibraryScreen(), settings);
case AppRoutes.workoutHistory:
    return _buildRoute(const WorkoutHistoryScreen(), settings);
case AppRoutes.startWorkout:
    return _buildRoute(const StartWorkoutScreen(), settings);
case AppRoutes.activeWorkout:
    return _buildRoute(const ActiveWorkoutScreen(), settings);
case AppRoutes.workoutResult:
    return _buildRoute(const WorkoutResultScreen(), settings);
case AppRoutes.foodDiary:
    return _buildRoute(const FoodDiaryScreen(), settings);
case AppRoutes.stepCounter:
    return _buildRoute(const StepCounterScreen(), settings);
case AppRoutes.stepSettings:
    return _buildRoute(const StepSettingsScreen(), settings);
case AppRoutes.addFood:
    final arguments =
        settings.arguments as AddFoodRouteArguments? ??
        AddFoodRouteArguments(date: DateTime.now());

```

```

        return _buildRoute(AddFoodScreen(arguments: arguments), settings);
    case AppRoutes.productDetails:
        final arguments = settings.arguments as ProductDetailsRouteArguments;
        return _buildRoute(
            ProductDetailsScreen(arguments: arguments),
            settings,
        );
    default:
        return _buildRoute(const SplashScreen(), settings);
    }
}

```

```

static PageRouteBuilder<void> _buildRoute(
    Widget page,
    RouteSettings settings,
) {
    return PageRouteBuilder<void>(
        settings: settings,
        transitionDuration: const Duration(milliseconds: 320),
        reverseTransitionDuration: const Duration(milliseconds: 240),
        pageBuilder: (_, __, _) => page,
        transitionsBuilder: (context, animation, secondaryAnimation, child) {
            final curved = CurvedAnimation(
                parent: animation,
                curve: Curves.easeOutCubic,
                reverseCurve: Curves.easeInCubic,
            );

            return FadeTransition(

```

```

        opacity: curved,
        child: SlideTransition(
          position: Tween<Offset>(
            begin: const Offset(0.025, 0.02),
            end: Offset.zero,
          ).animate(curved),
          child: child,
        ),
      );
    },
  );
}
}

```

А.5 Профиль пользователя

Файл: lib/features/auth/domain/entities/user_profile.dart

```

import 'gender.dart';
import 'user_goal.dart';

class UserProfile {
  const UserProfile({
    required this.userId,
    required this.email,
    required this.name,
    required this.gender,
    required this.birthDate,
    this.city,
    this.friendCode,

```

```

    this.heightCm,
    this.startWeightKg,
    this.currentWeightKg,
    this.targetWeightKg,
    this.goalType = UserGoalType.maintainWeight,
    this.dailyStepGoal = 10000,
    this.dailyCalorieGoal = 2200,
    this.role = 'student',
  });

```

```

final String userId;
final String email;
final String name;
final Gender gender;
final DateTime birthDate;
final String? city;
final String? friendCode;
final double? heightCm;
final double? startWeightKg;
final double? currentWeightKg;
final double? targetWeightKg;
final UserGoalType goalType;
final int dailyStepGoal;
final double dailyCalorieGoal;
final String role;

```

```

bool get isTrainer => role == 'trainer';
}

```

А.6 Сущность тренировки

Файл: lib/features/workout/domain/entities/workout.dart

```
import 'workout_route_point.dart';
import 'workout_type.dart';
import 'workout_exercise_entry.dart';

class Workout {
  const Workout({
    required this.id,
    required this.userId,
    required this.type,
    required this.startedAt,
    required this.endedAt,
    required this.duration,
    required this.calories,
    required this.distanceMeters,
    required this.route,
    required this.isSynced,
    this.title,
    this.note,
    this.place,
    this.exercises = const <WorkoutExerciseEntry>[],
    this.isManual = false,
  });

  final String id;
  final String userId;
  final WorkoutType type;
```

```

final DateTime startedAt;
final DateTime endedAt;
final Duration duration;
final double calories;
final double distanceMeters;
final List<WorkoutRoutePoint> route;
final bool isSynced;
final String? title;
final String? note;
final String? place;
final List<WorkoutExerciseEntry> exercises;
final bool isManual;

```

```

Workout copyWith({
  String? id,
  String? userId,
  WorkoutType? type,
  DateTime? startedAt,
  DateTime? endedAt,
  Duration? duration,
  double? calories,
  double? distanceMeters,
  List<WorkoutRoutePoint>? route,
  bool? isSynced,
  Object? title = _sentinel,
  Object? note = _sentinel,
  Object? place = _sentinel,
  List<WorkoutExerciseEntry>? exercises,
  bool? isManual,

```

```

    }) {
      return Workout(
        id: id ?? this.id,
        userId: userId ?? this.userId,
        type: type ?? this.type,
        startedAt: startedAt ?? this.startedAt,
        endedAt: endedAt ?? this.endedAt,
        duration: duration ?? this.duration,
        calories: calories ?? this.calories,
        distanceMeters: distanceMeters ?? this.distanceMeters,
        route: route ?? this.route,
        isSynced: isSynced ?? this.isSynced,
        title: title == _sentinel ? this.title : title as String?,
        note: note == _sentinel ? this.note : note as String?,
        place: place == _sentinel ? this.place : place as String?,
        exercises: exercises ?? this.exercises,
        isManual: isManual ?? this.isManual,
      );
    }
  }
}

```

```
const Object _sentinel = Object();
```

A.7 Расчет показателей тренировки

Файл: lib/features/workout/domain/services/workout_metrics_calculator.dart

```
import 'dart:math' as math;
```

```
import '../entities/workout_route_point.dart';
```

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		129

```

import '../entities/workout_type.dart';

abstract final class WorkoutMetricsCalculator {
  static double calculateCaloriesBurned({
    required WorkoutType type,
    required Duration duration,
    required double distanceMeters,
    List<WorkoutRoutePoint> route = const <WorkoutRoutePoint>[],
  }) {
    final durationMinutes = duration.inSeconds / 60;
    final distanceKm = distanceMeters / 1000;

    final calories = switch (type) {
      WorkoutType.running => (durationMinutes * 9.8) + (distanceKm * 1.1),
      WorkoutType.treadmillRunning =>
        (durationMinutes * 9.4) + (distanceKm * 0.9),
      WorkoutType.trailRunning => (durationMinutes * 11.2) + (distanceKm *
1.4),
      WorkoutType.intervalRunning =>
        (durationMinutes * 12.4) + (distanceKm * 1.2),
      WorkoutType.cycling => (durationMinutes * 7.5) + (distanceKm * 0.4),
      WorkoutType.walking => (durationMinutes * 4.8) + (distanceKm * 0.6),
      WorkoutType.strength => durationMinutes * 6.2,
      WorkoutType.cardio => (durationMinutes * 8.2) + (distanceKm * 0.3),
    };

    final routeTelemetry = analyzeRouteTelemetry(route);
    final elevationBonus = switch (type) {
      WorkoutType.running ||

```

```

WorkoutType.treadmillRunning ||
WorkoutType.trailRunning ||
WorkoutType.intervalRunning ||
WorkoutType.walking => routeTelemetry.elevationGainMeters * 0.7,
WorkoutType.cycling => routeTelemetry.elevationGainMeters * 0.42,
WorkoutType.strength || WorkoutType.cardio => 0.0,
};
final speedMultiplier = _speedEffortMultiplier(
    type,
    routeTelemetry.averageSpeedMetersPerSecond,
);
final adjustedCalories = (calories * speedMultiplier) + elevationBonus;

return double.parse(adjustedCalories.toStringAsFixed(1));
}

static double calculateDistanceBetween(
    WorkoutRoutePoint start,
    WorkoutRoutePoint end,
) {
    if (!start.isValidCoordinates || !end.isValidCoordinates) {
        return 0;
    }

    const earthRadiusMeters = 6371000.0;
    final latitudeDistance = _toRadians(end.latitude - start.latitude);
    final longitudeDistance = _toRadians(end.longitude - start.longitude);
    final startLatitude = _toRadians(start.latitude);
    final endLatitude = _toRadians(end.latitude);

```

```

final haversine =
    math.pow(math.sin(latitudeDistance / 2), 2) +
    math.cos(startLatitude) *
        math.cos(endLatitude) *
    math.pow(math.sin(longitudeDistance / 2), 2);
final normalizedHaversine = haversine.clamp(0, 1).toDouble();
final angularDistance =
    2 *
    math.atan2(
        math.sqrt(normalizedHaversine),
        math.sqrt(1 - normalizedHaversine),
    );

final distance = earthRadiusMeters * angularDistance;
return distance.isFinite ? distance : 0;
}

static WorkoutRouteTelemetry analyzeRouteTelemetry(
    List<WorkoutRoutePoint> route,
) {
    if (route.length < 2) {
        return const WorkoutRouteTelemetry();
    }

    var elevationGainMeters = 0.0;
    var previousAltitude = route.first.altitudeMeters;
    for (final point in route.skip(1)) {
        final altitude = point.altitudeMeters;

```

```

if (altitude == null || !altitude.isFinite) {
    continue;
}
if (previousAltitude == null || !previousAltitude.isFinite) {
    previousAltitude = altitude;
    continue;
}

final delta = altitude - previousAltitude;
if (delta > 1.5) {
    elevationGainMeters += delta;
}
previousAltitude = altitude;
}

final speedSamples = route
    .map((point) => point.speedMetersPerSecond)
    .whereType<double>()
    .where((speed) => speed.isFinite && speed > 0 && speed < 12)
    .toList(growable: false);
final averageSpeed = speedSamples.isEmpty
    ? 0.0
    : speedSamples.reduce((left, right) => left + right) /
      speedSamples.length;

final accuracySamples = route
    .map((point) => point.accuracyMeters)
    .whereType<double>()
    .where((accuracy) => accuracy.isFinite && accuracy > 0)

```

```

        .toList(growable: false);

final averageAccuracy = accuracySamples.isEmpty
    ? null
    : accuracySamples.reduce((left, right) => left + right) /
        accuracySamples.length;

return WorkoutRouteTelemetry(
    elevationGainMeters: elevationGainMeters,
    averageSpeedMetersPerSecond: averageSpeed,
    averageAccuracyMeters: averageAccuracy,
    pointCount: route.length,
);
}

static double _toRadians(double degrees) => degrees * math.pi / 180;

static double _speedEffortMultiplier(
    WorkoutType type,
    double averageSpeedMetersPerSecond,
) {
    if (averageSpeedMetersPerSecond <= 0) {
        return 1;
    }

    final speedKmH = averageSpeedMetersPerSecond * 3.6;
    return switch (type) {
        WorkoutType.running ||
        WorkoutType.treadmillRunning ||
        WorkoutType.trailRunning =>

```

```

        speedKmH >= 13
        ? 1.08
        : speedKmH >= 10
        ? 1.04
        : 1,
        WorkoutType.intervalRunning => speedKmH >= 14 ? 1.1 : 1.04,
        WorkoutType.walking => speedKmH >= 6.5 ? 1.05 : 1,
        WorkoutType.cycling =>
        speedKmH >= 28
        ? 1.07
        : speedKmH >= 22
        ? 1.03
        : 1,
        WorkoutType.strength || WorkoutType.cardio => 1,
    };
}
}

```

```

class WorkoutRouteTelemetry {
    const WorkoutRouteTelemetry({
        this.elevationGainMeters = 0,
        this.averageSpeedMetersPerSecond = 0,
        this.averageAccuracyMeters,
        this.pointCount = 0,
    });

    final double elevationGainMeters;
    final double averageSpeedMetersPerSecond;
    final double? averageAccuracyMeters;
}

```

```

final int pointCount;

bool get hasElevationData => elevationGainMeters > 0;
bool get hasSpeedData => averageSpeedMetersPerSecond > 0;
}

```

А.8 Контроллер тренировочной сессии

Файл:

lib/features/workout/presentation/controllers/workout_session_controller.dart

```

import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/errors/app_exception.dart';
import '../../domain/entities/workout.dart';
import '../../domain/entities/workout_route_point.dart';
import '../../domain/entities/workout_save_status.dart';
import '../../domain/entities/workout_type.dart';
import '../../domain/services/workout_metrics_calculator.dart';
import '../../domain/services/workout_session_engine.dart';
import '../providers/workout_providers.dart';

enum WorkoutSessionStatus { idle, running, paused, completed, saving }

@immutable
class WorkoutSessionState {
  const WorkoutSessionState({

```

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						136
Изм.	Лист	№ докум.	Подпись	Дата		

```

this.status = WorkoutSessionStatus.idle,
this.workoutType,
this.elapsed = Duration.zero,
this.calories = 0,
this.distanceMeters = 0,
this.route = const <WorkoutRoutePoint>[],
this.completedWorkout,
this.isLocationTrackingAvailable = true,
this.shouldAskForRouteMap = false,
this.shouldShowLocationEnableRequest = false,
});

```

```

final WorkoutSessionStatus status;
final WorkoutType? workoutType;
final Duration elapsed;
final double calories;
final double distanceMeters;
final List<WorkoutRoutePoint> route;
final Workout? completedWorkout;
final bool isLocationTrackingAvailable;
final bool shouldAskForRouteMap;
final bool shouldShowLocationEnableRequest;

```

```

WorkoutSessionState copyWith({
  WorkoutSessionStatus? status,
  Object? workoutType = _sentinel,
  Duration? elapsed,
  double? calories,
  double? distanceMeters,

```

```

List<WorkoutRoutePoint>? route,
Object? completedWorkout = _sentinel,
bool? isLocationTrackingAvailable,
bool? shouldAskForRouteMap,
bool? shouldShowLocationEnableRequest,
)) {
return WorkoutSessionState(
    status: status ?? this.status,
    workoutType: workoutType == _sentinel
        ? this.workoutType
        : workoutType as WorkoutType?,
    elapsed: elapsed ?? this.elapsed,
    calories: calories ?? this.calories,
    distanceMeters: distanceMeters ?? this.distanceMeters,
    route: route ?? this.route,
    completedWorkout: completedWorkout == _sentinel
        ? this.completedWorkout
        : completedWorkout as Workout?,
    isLocationTrackingAvailable:
        isLocationTrackingAvailable ?? this.isLocationTrackingAvailable,
    shouldAskForRouteMap:          shouldAskForRouteMap          ??
this.shouldAskForRouteMap,
    shouldShowLocationEnableRequest:
        shouldShowLocationEnableRequest ??
        this.shouldShowLocationEnableRequest,
);
}
}

```

```

const Object _sentinel = Object();

class WorkoutSessionController extends Notifier<WorkoutSessionState> {
  WorkoutSessionEngine? _engine;
  Timer? _ticker;
  StreamSubscription<WorkoutRoutePoint>? _locationSubscription;

  @override
  WorkoutSessionState build() {
    ref.onDispose(_disposeResources);
    return const WorkoutSessionState();
  }

  Future<void> startWorkoutTimer(WorkoutType type) async {
    _disposeResources();

    _engine = WorkoutSessionEngine(type: type);
    final locationEnabled = await ref
      .read(workoutLocationDataSourceProvider)
      .prepareTracking();

    state = WorkoutSessionState(
      status: WorkoutSessionStatus.running,
      workoutType: type,
      isLocationTrackingAvailable: locationEnabled,
      shouldAskForRouteMap: !locationEnabled,
    );

    _startTicker();
  }
}

```

```

    if (locationEnabled) {
        _startLocationSubscription();
    }
}

Future<void> requestRouteMap() async {
    if (state.status != WorkoutSessionStatus.running &&
        state.status != WorkoutSessionStatus.paused) {
        return;
    }

    final locationEnabled = await ref
        .read(workoutLocationDataSourceProvider)
        .prepareTracking();

    if (locationEnabled) {
        state = state.copyWith(
            isLocationTrackingAvailable: true,
            shouldAskForRouteMap: false,
            shouldShowLocationEnableRequest: false,
        );
        _startLocationSubscription();
        return;
    }

    state = state.copyWith(
        isLocationTrackingAvailable: false,
        shouldAskForRouteMap: false,
        shouldShowLocationEnableRequest: true,
    );
}

```

```
);  
}
```

```
void skipRouteMap() {  
    state = state.copyWith(  
        shouldAskForRouteMap: false,  
        shouldShowLocationEnableRequest: false,  
    );  
}
```

```
Future<void> openLocationSettings() {  
    return  
    ref.read(workoutLocationDataSourceProvider).openLocationSettings();  
}
```

```
void pauseWorkout() {  
    final engine = _engine;  
    if (engine == null || state.status != WorkoutSessionStatus.running) {  
        return;  
    }  
}
```

```
engine.pause(DateTime.now());  
_refreshState(status: WorkoutSessionStatus.paused);  
}
```

```
void resumeWorkout() {  
    final engine = _engine;  
    if (engine == null || state.status != WorkoutSessionStatus.paused) {  
        return;  
    }  
}
```

```

    }

    engine.resume(DateTime.now());
    _refreshState(status: WorkoutSessionStatus.running);
}

Workout? stopWorkout() {
    final engine = _engine;
    final user = ref.read(firebaseWorkoutUserProvider);
    if (engine == null || user == null) {
        return null;
    }

    final workout = engine.stop(userId: user.uid);
    _disposeResources();
    state = state.copyWith(
        status: WorkoutSessionStatus.completed,
        elapsed: workout.duration,
        calories: workout.calories,
        distanceMeters: workout.distanceMeters,
        route: workout.route,
        completedWorkout: workout,
        shouldAskForRouteMap: false,
        shouldShowLocationEnableRequest: false,
    );

    return workout;
}

```

```

Future<WorkoutSaveStatus> saveWorkoutToDatabase() async {
    final workout = state.completedWorkout;
    if (workout == null) {
        throw const WorkoutException(AppErrorCode.workoutSaveFailed);
    }

    state = state.copyWith(status: WorkoutSessionStatus.saving);
    final saveStatus = await
ref.read(saveWorkoutUseCaseProvider).call(workout);

    final savedWorkout = switch (saveStatus) {
        WorkoutSaveStatus.synced => workout.copyWith(isSynced: true),
        WorkoutSaveStatus.savedLocally => workout,
    };

    state = state.copyWith(
        status: WorkoutSessionStatus.completed,
        completedWorkout: savedWorkout,
    );

    return saveStatus;
}

void reset() {
    _disposeResources();
    state = const WorkoutSessionState();
}

void _startTicker() {

```

```

    _ticker = Timer.periodic(const Duration(seconds: 1), (_) {
      if (state.status == WorkoutSessionStatus.running ||
          state.status == WorkoutSessionStatus.paused) {
        _refreshState();
      }
    });
  }

void _handleRoutePoint(WorkoutRoutePoint point) {
  final engine = _engine;
  if (engine == null || state.status != WorkoutSessionStatus.running) {
    return;
  }

  engine.addRoutePoint(point);
  _refreshState();
}

void _startLocationSubscription() {
  _locationSubscription?.cancel();
  _locationSubscription = ref
    .read(workoutLocationDataSourceProvider)
    .watchRoute()
    .listen(_handleRoutePoint);
}

void _refreshState({WorkoutSessionStatus? status}) {
  final engine = _engine;
  if (engine == null) {

```

```

        return;
    }

    final elapsed = engine.elapsedAt(DateTime.now());
    final distanceMeters = engine.distanceMeters;
    final calories = WorkoutMetricsCalculator.calculateCaloriesBurned(
        type: engine.type,
        duration: elapsed,
        distanceMeters: distanceMeters,
        route: engine.route,
    );

    state = state.copyWith(
        status: status ?? state.status,
        workoutType: engine.type,
        elapsed: elapsed,
        calories: calories,
        distanceMeters: distanceMeters,
        route: engine.route,
    );
}

void _disposeResources() {
    _ticker?.cancel();
    _ticker = null;
    _locationSubscription?.cancel();
    _locationSubscription = null;
}
}

```

А.9 Расчет пищевой ценности

Файл: lib/features/nutrition/domain/services/nutrition_macro_calculator.dart

```
import '../entities/food_macros.dart';
```

```
import '../entities/food_product.dart';
```

```
class NutritionMacroCalculator {
```

```
  const NutritionMacroCalculator();
```

```
  FoodMacros calculate({required FoodProduct product, required double  
grams}) {
```

```
    final multiplier = grams / 100;
```

```
    // Пересчитываем БЖУ и калории из значения на 100 грамм в  
    фактическую порцию.
```

```
    return FoodMacros(
```

```
      calories: product.macrosPer100Grams.calories * multiplier,
```

```
      proteins: product.macrosPer100Grams.proteins * multiplier,
```

```
      fats: product.macrosPer100Grams.fats * multiplier,
```

```
      carbs: product.macrosPer100Grams.carbs * multiplier,
```

```
    );
```

```
  }
```

```
}
```

```
// 4607002010012 -> Greek Yogurt
```

```
// 4820001234567 -> Protein Bar
```

```
// 4601234567890 -> Oatmeal
```

```
// 4601111111111 -> Chicken Breast
```

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		146

А.10 Контроллер добавления продукта

Файл: lib/features/nutrition/presentation/controllers/add_food_controller.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../domain/entities/food_entry_draft.dart';
import '../domain/entities/food_input_method.dart';
import '../domain/entities/food_macros.dart';
import '../domain/entities/food_product.dart';
import '../providers/nutrition_providers.dart';
```

@immutable

```
class AddFoodState {
  const AddFoodState({
    this.isLoading = false,
    this.inputMethod = FoodInputMethod.manual,
    this.selectedProduct,
    this.editingProduct,
    this.calculatedMacros = const FoodMacros.zero(),
  });
```

```
  final bool isLoading;
  final FoodInputMethod inputMethod;
  final FoodProduct? selectedProduct;
  final FoodProduct? editingProduct;
  final FoodMacros calculatedMacros;
```

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						147
Изм.	Лист	№ докум.	Подпись	Дата		

```

AddFoodState copyWith({
    bool? isLoading,
    FoodInputMethod? inputMethod,
    Object? selectedProduct = _sentinel,
    Object? editingProduct = _sentinel,
    FoodMacros? calculatedMacros,
}) {
    return AddFoodState(
        isLoading: isLoading ?? this.isLoading,
        inputMethod: inputMethod ?? this.inputMethod,
        selectedProduct: selectedProduct == _sentinel
            ? this.selectedProduct
            : selectedProduct as FoodProduct?,
        editingProduct: editingProduct == _sentinel
            ? this.editingProduct
            : editingProduct as FoodProduct?,
        calculatedMacros: calculatedMacros ?? this.calculatedMacros,
    );
}
}

```

```
const Object _sentinel = Object();
```

```

class AddFoodController extends Notifier<AddFoodState> {
    @override
    AddFoodState build() => const AddFoodState();

    void setInputMethod(FoodInputMethod method) {
        state = state.copyWith(

```

```

        inputMethod: method,
        editingProduct: method == FoodInputMethod.manual
            ? state.editingProduct
            : null,
        selectedProduct: method == FoodInputMethod.quickAccess
            ? state.selectedProduct
            : null,
    );
}

void selectQuickAccessProduct(FoodProduct product) {
    state = state.copyWith(
        inputMethod: FoodInputMethod.quickAccess,
        selectedProduct: product,
        editingProduct: null,
    );
}

void startEditingProduct(FoodProduct product) {
    state = state.copyWith(
        inputMethod: FoodInputMethod.manual,
        selectedProduct: product,
        editingProduct: product,
    );
}

void clearEditingProduct() {
    state = state.copyWith(editingProduct: null, selectedProduct: null);
}

```

```

Future<FoodProduct> findProductByBarcode(String barcode) async {
    state = state.copyWith(isLoading: true);
    try {
        final product = await ref
            .read(findProductByBarcodeUseCaseProvider)
            .call(barcode);
        state = state.copyWith(
            isLoading: false,
            selectedProduct: product,
            editingProduct: null,
        );
        return product;
    } catch (_) {
        state = state.copyWith(isLoading: false);
        rethrow;
    }
}

```

```

FoodMacros calculateMacros({
    required FoodProduct product,
    required double grams,
}) {
    final macros = ref
        .read(calculateMacrosUseCaseProvider)
        .call(product: product, grams: grams);
    state = state.copyWith(selectedProduct: product, calculatedMacros: macros);
    return macros;
}

```

```

Future<void> addFoodEntry(FoodEntryDraft draft) async {
  final user = ref.read(firebaseNutritionUserProvider);
  if (user == null) {
    return;
  }

  state = state.copyWith(isLoading: true);
  try {
    await ref
      .read(addFoodEntryUseCaseProvider)
      .call(userId: user.uid, draft: draft);
    state = state.copyWith(isLoading: false);
    ref.invalidate(savedFoodProductsProvider);
  } catch (_) {
    state = state.copyWith(isLoading: false);
    rethrow;
  }
}

```

A.11 Расчет аналитики целей

Файл:

lib/features/dashboard/domain/services/dashboard_analytics_calculator.dart

```

import 'package:flutter/material.dart';

import '../..../auth/domain/entities/user_goal.dart';
import '../..../auth/domain/entities/user_profile.dart';

```

```
import '../..../auth/domain/entities/weight_history_entry.dart';
import '../..../nutrition/domain/entities/daily_food_diary.dart';
import '../..../steps/domain/entities/daily_step_count.dart';
import '../..../workout/domain/entities/workout.dart';
import '../..../workout/domain/entities/workout_type.dart';
import '../entities/dashboard_analytics.dart';
```

```
class DashboardAnalyticsCalculator {
  const DashboardAnalyticsCalculator({
    this.defaultDailyStepGoal = 10000,
    this.defaultDailyCalorieGoal = 2200,
    this.metersPerStep = 0.78,
  });
```

```
  final int defaultDailyStepGoal;
  final double defaultDailyCalorieGoal;
  final double metersPerStep;
```

```
  DashboardWeeklyStats calculateWeeklyStats({
    required List<Workout> workouts,
    required List<DailyFoodDiary> diaries,
    List<DailyStepCount> stepCounts = const <DailyStepCount>[],
    int? dailyStepGoal,
    double? dailyCalorieGoal,
    DateTime? now,
    DateTime? from,
    DateTime? to,
  }) {
    final endDate = DateUtils.dateOnly(to ?? now ?? DateTime.now());
```

```

final startDate = from != null
    ? DateUtils.dateOnly(from)
    : endDate.subtract(const Duration(days: 6));
final diaryByDay = <String, DailyFoodDiary>{
    for (final diary in diaries) _dateKey(diary.date): diary,
};
final stepsByDay = <String, DailyStepCount>{
    for (final stepCount in stepCounts) _dateKey(stepCount.date): stepCount,
};
final workoutsByDay = <String, List<Workout>>{};

for (final workout in workouts) {
    final workoutDate = DateUtils.dateOnly(workout.startedAt);
    final startsBeforeWindow = workoutDate.isBefore(startDate);
    final startsAfterWindow = workoutDate.isAfter(endDate);
    if (startsBeforeWindow || startsAfterWindow) {
        continue;
    }

    workoutsByDay
        .putIfAbsent(_dateKey(workoutDate), () => <Workout>[])
        .add(workout);
}

final days = List<DashboardDaySummary>.generate(7, (index) {
    final date = startDate.add(Duration(days: index));
    final dayWorkouts = workoutsByDay[_dateKey(date)] ?? const
<Workout>[];
    final diary = diaryByDay[_dateKey(date)];

```

```

final macros = diary?.totalMacros();
final recordedSteps = stepsByDay[_dateKey(date)]?.steps;
final steps =
    recordedSteps ??
    dayWorkouts.fold<int>(
        0,
        (total, workout) => total + _estimateSteps(workout),
    );
final calories = macros?.calories ?? 0;
final resolvedStepGoal = dailyStepGoal ?? defaultDailyStepGoal;
final resolvedCalorieGoal = dailyCalorieGoal ?? defaultDailyCalorieGoal;

return DashboardDaySummary(
    date: date,
    steps: steps,
    calories: calories,
    progress: calculateProgress(
        steps: steps,
        calories: calories,
        stepGoal: resolvedStepGoal,
        calorieGoal: resolvedCalorieGoal,
    ),
);
}, growable: false);

return DashboardWeeklyStats(days: days);
}

DashboardGoalProgress calculateProgress({

```

```

required int steps,
required double calories,
required int stepGoal,
required double calorieGoal,
}) {
    final safeStepGoal = stepGoal <= 0 ? defaultDailyStepGoal : stepGoal;
    final safeCalorieGoal = calorieGoal <= 0
        ? defaultDailyCalorieGoal
        : calorieGoal;
    final stepsProgress = _clamp01(steps / safeStepGoal);
    final caloriesProgress = _clamp01(calories / safeCalorieGoal);

    return DashboardGoalProgress(
        steps: stepsProgress,
        calories: caloriesProgress,
        overall: (stepsProgress + caloriesProgress) / 2,
    );
}

```

```

DashboardAnalytics buildAnalytics({
    required List<Workout> workouts,
    required List<DailyFoodDiary> diaries,
    List<DailyStepCount> stepCounts = const <DailyStepCount>[],
    UserProfile? profile,
    List<WeightHistoryEntry> weightHistory = const <WeightHistoryEntry>[],
    DateTime? now,
}) {
    final targetDate = DateUtils.dateOnly(now ?? DateTime.now());
    final goals = DashboardUserGoals(

```

```

stepGoal: profile?.dailyStepGoal ?? defaultDailyStepGoal,
calorieGoal: profile?.dailyCalorieGoal ?? defaultDailyCalorieGoal,
goalType: profile?.goalType ?? UserGoalType.maintainWeight,
currentWeightKg: profile?.currentWeightKg,
targetWeightKg: profile?.targetWeightKg,
);

final weeklyStats = calculateWeeklyStats(
  workouts: workouts,
  diaries: diaries,
  stepCounts: stepCounts,
  dailyStepGoal: goals.stepGoal,
  dailyCalorieGoal: goals.calorieGoal,
  now: now,
);

final todayDiary = diaries.cast<DailyFoodDiary?>().firstWhere(
  (diary) => diary != null && DateUtils.isSameDay(diary.date, targetDate),
  orElse: () => DailyFoodDiary(date: targetDate, entries: const []),
)!;

final macros = todayDiary.totalMacros();

return DashboardAnalytics(
  weeklyStats: weeklyStats,
  progress: weeklyStats.today.progress,
  goals: goals,
  weightAnalytics: buildWeightAnalytics(
    profile: profile,
    weightHistory: weightHistory,
  ),
  proteins: macros.proteins,

```

```

        fats: macros.fats,
        carbs: macros.carbs,
    );
}

```

```

DashboardRangeAnalytics buildRangeAnalytics( {
    required List<Workout> workouts,
    required List<DailyFoodDiary> diaries,
    required DateTime from,
    required DateTime to,
    List<DailyStepCount> stepCounts = const <DailyStepCount>[],
    UserProfile? profile,
    List<WeightHistoryEntry> weightHistory = const <WeightHistoryEntry>[],
}) {
    final normalizedFrom = DateUtils.dateOnly(from);
    final normalizedTo = DateUtils.dateOnly(to);
    final stats = calculateWeeklyStats(
        workouts: workouts,
        diaries: diaries,
        stepCounts: stepCounts,
        dailyStepGoal: profile?.dailyStepGoal ?? defaultDailyStepGoal,
        dailyCalorieGoal: profile?.dailyCalorieGoal ?? defaultDailyCalorieGoal,
        from: normalizedFrom,
        to: normalizedTo,
    );
    final rangeWorkouts = workouts
        .where((workout) {
            final workoutDate = DateUtils.dateOnly(workout.startedAt);
            return !workoutDate.isBefore(normalizedFrom) &&

```

```

        !workoutDate.isAfter(normalizedTo);
    })
    .toList(growable: false);
final totalWorkoutCalories = rangeWorkouts.fold<double>(
    0,
    (total, workout) => total + workout.calories,
);
final numberOfDays = normalizedTo.difference(normalizedFrom).inDays +
1;

```

```

return DashboardRangeAnalytics(
    from: normalizedFrom,
    to: normalizedTo,
    stats: stats,
    weightAnalytics: buildWeightAnalytics(
        profile: profile,
        weightHistory: weightHistory,
    ),
    totalWorkoutCalories: totalWorkoutCalories,
    totalWorkouts: rangeWorkouts.length,
    averageDailySteps: stats.totalSteps / numberOfDays,
    averageDailyCalories: stats.totalCalories / numberOfDays,
);
}

```

```

DashboardWeightAnalytics buildWeightAnalytics({
    UserProfile? profile,
    List<WeightHistoryEntry> weightHistory = const <WeightHistoryEntry>[],
}) {

```

```

final sortedHistory = List<WeightHistoryEntry>.from(weightHistory)
    ..sort((left, right) => left.recordedAt.compareTo(right.recordedAt));
final latestWeight = sortedHistory.isNotEmpty
    ? sortedHistory.last.weightKg
    : profile?.currentWeightKg;
final firstWeightInPeriod = sortedHistory.isNotEmpty
    ? sortedHistory.first.weightKg
    : latestWeight;
final lastWeightInPeriod = sortedHistory.isNotEmpty
    ? sortedHistory.last.weightKg
    : latestWeight;
final startWeight = profile?.startWeightKg ?? firstWeightInPeriod;
final targetWeight = profile?.targetWeightKg;
final goalType = profile?.goalType ?? UserGoalType.maintainWeight;
final weeklyChange = firstWeightInPeriod != null && latestWeight != null
    ? _goalAdjustedChange(
        start: firstWeightInPeriod,
        end: latestWeight,
        goalType: goalType,
    )
    : null;
final totalChange = startWeight != null && latestWeight != null
    ? _goalAdjustedChange(
        start: startWeight,
        end: latestWeight,
        goalType: goalType,
    )
    : null;
final remainingToGoal = latestWeight != null && targetWeight != null

```

```

    ? _remainingToGoal(
        currentWeight: latestWeight,
        targetWeight: targetWeight,
        goalType: goalType,
    )
    : null;

final goalProgress =
    startWeight != null &&
        latestWeight != null &&
        targetWeight != null &&
        !_nearlyEqual(startWeight, targetWeight)
    ? _clamp01(
        _goalAdjustedChange(
            start: startWeight,
            end: latestWeight,
            goalType: goalType,
        ) /
        _goalAdjustedChange(
            start: startWeight,
            end: targetWeight,
            goalType: goalType,
        ).abs(),
    )
    : null;

return DashboardWeightAnalytics(
    goalType: goalType,
    startWeightKg: startWeight,
    currentWeightKg: latestWeight,

```

```

        targetWeightKg: targetWeight,
        periodStartWeightKg: firstWeightInPeriod,
        periodEndWeightKg: lastWeightInPeriod,
        weeklyChangeKg: weeklyChange,
        totalChangeKg: totalChange,
        remainingToGoalKg: remainingToGoal,
        goalProgress: goalProgress,
    );
}

```

```

int _estimateSteps(Workout workout) {
    final isStepBasedWorkout =
        workout.type == WorkoutType.running ||
        workout.type == WorkoutType.walking;
    if (!isStepBasedWorkout || workout.distanceMeters <= 0) {
        return 0;
    }

    return (workout.distanceMeters / metersPerStep).round();
}

```

```

String _dateKey(DateTime date) {
    final normalized = DateUtils.dateOnly(date);
    return '${normalized.year}-${normalized.month}-${normalized.day}';
}

```

```

double _clamp01(double value) {
    if (value < 0) {
        return 0;
    }
}

```

```

    }
    if (value > 1) {
        return 1;
    }
    return value;
}

```

```

double _goalAdjustedChange(
    required double start,
    required double end,
    required UserGoalType goalType,
)) {
    return switch (goalType) {
        UserGoalType.loseWeight => start - end,
        UserGoalType.gainWeight => end - start,
        UserGoalType.maintainWeight => -(end - start).abs(),
    };
}

```

```

double _remainingToGoal(
    required double currentWeight,
    required double targetWeight,
    required UserGoalType goalType,
)) {
    return switch (goalType) {
        UserGoalType.loseWeight => currentWeight - targetWeight,
        UserGoalType.gainWeight => targetWeight - currentWeight,
        UserGoalType.maintainWeight => (currentWeight - targetWeight).abs(),
    };
}

```

```
}
```

```
bool _nearlyEqual(double a, double b) {  
    return (a - b).abs() < 0.001;  
}  
}
```

А.12 Фоновый учет шагов

Файл: lib/features/steps/data/services/step_tracking_service.dart

```
import 'dart:async';  
import 'dart:ui';  
  
import 'package:flutter/foundation.dart';  
import 'package:flutter/widgets.dart';  
import 'package:flutter_background_service/flutter_background_service.dart';  
import 'package:pedometer/pedometer.dart';  
import 'package:permission_handler/permission_handler.dart';  
import 'package:shared_preferences/shared_preferences.dart';  
  
import '../datasources/step_local_data_source.dart';  
import '../models/daily_step_count_model.dart';  
  
const String stepTrackingActiveUserIdKey = 'step_tracking_active_user_id';  
const String stepTrackingUpdateEvent = 'steps_updated';  
const String _stepTrackingSetUserEvent = 'set_tracking_user';  
const String _stepTrackingStopEvent = 'stop_tracking';  
const String _notificationTitle = 'Liga Gym Step Tracking';
```

```

class StepTrackingService {
  StepTrackingService({required SharedPreferences? sharedPreferences})
    : _sharedPreferences = sharedPreferences;

  final SharedPreferences? _sharedPreferences;

  static bool _isConfigured = false;

  static Future<void> configureBackgroundTracking() async {
    if (_isConfigured || !isStepTrackingSupportedPlatform) {
      return;
    }

    final service = FlutterBackgroundService();
    await service.configure(
      androidConfiguration: AndroidConfiguration(
        onStart: stepTrackingBackgroundEntrypoint,
        autoStart: false,
        autoStartOnBoot: true,
        isForegroundMode: true,
        initialNotificationTitle: _notificationTitle,
        initialNotificationContent: 'Preparing step counter',
        foregroundServiceTypes: [AndroidForegroundType.health],
      ),
      iosConfiguration: IosConfiguration(
        autoStart: false,
        onForeground: stepTrackingBackgroundEntrypoint,
      ),
    );
  }
}

```

```

    _isConfigured = true;
}

Future<bool> ensureTracking({required String userId}) async {
  if (!isStepTrackingSupportedPlatform) {
    return false;
  }

  final permissionStatus = await Permission.activityRecognition.request();
  if (!permissionStatus.isGranted) {
    return false;
  }
  await Permission.notification.request();

  await _sharedPreferences?.setString(stepTrackingActiveUserIdKey, userId);

  final service = FlutterBackgroundService();
  if (!await service.isRunning()) {
    await service.startService();
  }

  service.invoke(_stepTrackingSetUserEvent, <String, Object?>{
    'userId': userId,
  });
  return true;
}

Future<void> stopTracking() async {

```

```

if (!isStepTrackingSupportedPlatform) {
    return;
}

await _sharedPreferences?.remove(stepTrackingActiveUserIdKey);
final service = FlutterBackgroundService();
if (await service.isRunning()) {
    service.invoke(_stepTrackingStopEvent);
}
}

```

```

Future<bool> isRunning() {
    if (!isStepTrackingSupportedPlatform) {
        return Future<bool>.value(false);
    }

    return FlutterBackgroundService().isRunning();
}

```

```

Future<void> saveDailyGoal({
    required String userId,
    required int goal,
}) async {
    await _sharedPreferences?.setInt(stepTrackingGoalKey(userId), goal);
}

```

```

Future<void> resetUserSession({
    required String? previousUserId,
    required String? nextUserId,
}

```

```

    }) async {
        if (_sharedPreferences == null) {
            return;
        }

        if (previousUserId != null && previousUserId.isNotEmpty) {
            await _sharedPreferences.remove(stepTrackingGoalKey(previousUserId));
            await _sharedPreferences.remove(
                stepGoalCelebrationPendingDateKey(previousUserId),
            );
            await _sharedPreferences.remove(
                stepGoalCelebratedDateKey(previousUserId),
            );
        }

        if (nextUserId == null || nextUserId.isEmpty) {
            await _sharedPreferences.remove(stepTrackingActiveUserIdKey);
            return;
        }

        await _sharedPreferences.setString(stepTrackingActiveUserIdKey,
nextUserId);
    }
}

```

```

String stepTrackingGoalKey(String userId) =>
'step_tracking_daily_goal_$userId';

```

```

String stepGoalCelebrationPendingDateKey(String userId) =>

```

```
'step_goal_celebration_pending_date_$userId';
```

```
String stepGoalCelebratedDateKey(String userId) =>
```

```
'step_goal_celebrated_date_$userId';
```

```
bool get isStepTrackingSupportedPlatform =>
```

```
!kIsWeb && defaultTargetPlatform == TargetPlatform.android;
```

```
@pragma('vm:entry-point')
```

```
void stepTrackingBackgroundEntrypoint(ServiceInstance service) async {
```

```
  WidgetsFlutterBinding.ensureInitialized();
```

```
  DartPluginRegistrant.ensureInitialized();
```

```
  final localDataSource = SqliteStepLocalDataSource();
```

```
  final sharedPreferences = await SharedPreferences.getInstance();
```

```
  StreamSubscription<StepCount>? stepSubscription;
```

```
  var trackedUserId
```

=

```
  sharedPreferences.getString(stepTrackingActiveUserIdKey);
```

```
  Future<void> updateForegroundNotification() async {
```

```
    final userId = trackedUserId;
```

```
    if (userId == null ||
```

```
        userId.isEmpty ||
```

```
        service is! AndroidServiceInstance) {
```

```
      return;
```

```
    }
```

```
    final todaySteps = await localDataSource.loadStepsForDate(
```

```
      userId: userId,
```

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						168
Изм.	Лист	№ докум.	Подпись	Дата		

```

        date: DateTime.now(),
    );

    service.setForegroundNotificationInfo(
        title: _notificationTitle,
        content: 'Today: $todaySteps steps',
    );
}

Future<void> handleStepEvent(StepCount event) async {
    final userId = trackedUserId;
    if (userId == null || userId.isEmpty) {
        return;
    }

    await localDataSource.recordSensorReading(
        userId: userId,
        sensorSteps: event.steps,
        recordedAt: event.timeStamp,
    );

    await updateForegroundNotification();

    final todaySteps = await localDataSource.loadStepsForDate(
        userId: userId,
        date: event.timeStamp,
    );

    final goalReachedNow = await _handleGoalReached(
        sharedPreferences: sharedPreferences,
        userId: userId,

```

```

        todaySteps: todaySteps,
    );
    service.invoke(stepTrackingUpdateEvent, <String, Object?>{
        'userId': userId,
        'steps': todaySteps,
        'dateKey': buildStepDateKey(event.timeStamp),
        'goalReachedNow': goalReachedNow,
    });
}

```

```

Future<void> startTracking() async {
    final userId = trackedUserId;
    if (userId == null || userId.isEmpty) {
        service.stopSelf();
        return;
    }
}

```

```

await stepSubscription?.cancel();
stepSubscription = Pedometer.stepCountStream.listen(handleStepEvent);
stepSubscription!.onError((Object error) async {
    if (service is AndroidServiceInstance) {
        service.setForegroundNotificationInfo(
            title: _notificationTitle,
            content: 'Step sensor unavailable',
        );
    }
    debugPrint('Step tracking error: $error');
});

```

```

        await updateForegroundNotification();
    }

    service.on(_stepTrackingSetUserEvent).listen((payload) async {
        final userId = payload?['userId'] as String?;
        if (userId == null || userId.isEmpty) {
            return;
        }

        trackedUserId = userId;
        await sharedPreferences.setString(stepTrackingActiveUserIdKey, userId);
        await startTracking();
    });

    service.on(_stepTrackingStopEvent).listen((_) async {
        trackedUserId = null;
        await sharedPreferences.remove(stepTrackingActiveUserIdKey);
        await stepSubscription?.cancel();
        service.stopSelf();
    });

    await startTracking();
}

Future<bool> _handleGoalReached({
    required SharedPreferences sharedPreferences,
    required String userId,
    required int todaySteps,
}) async {

```

```

final stepGoal =
    sharedPreferences.getInt(stepTrackingGoalKey(userId)) ?? 10000;
if (todaySteps < stepGoal) {
    return false;
}

final todayKey = buildStepDateKey(DateTime.now());
final lastCelebratedDate = sharedPreferences.getString(
    stepGoalCelebratedDateKey(userId),
);
if (lastCelebratedDate == todayKey) {
    return false;
}

await sharedPreferences.setString(
    stepGoalCelebratedDateKey(userId),
    todayKey,
);
await sharedPreferences.setString(
    stepGoalCelebrationPendingDateKey(userId),
    todayKey,
);
return true;
}

```

A.13 AI-помощник и контекст пользователя

Файл: lib/features/ai_coach/domain/services/fitness_ai_agent.dart

```
import 'dart:async';
```

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						172
Изм.	Лист	№ докум.	Подпись	Дата		

```

import 'dart:convert';

import 'package:http/http.dart' as http;

import '../../auth/domain/entities/user_goal.dart';
import '../../nutrition/domain/entities/food_entry.dart';
import '../../nutrition/domain/entities/food_macros.dart';
import '../entities/ai_coach_message.dart';

class FitnessAiContext {
  const FitnessAiContext({
    required this.entries,
    required this.totalMacros,
    required this.calorieGoal,
    required this.stepGoal,
    required this.goalType,
    this.currentWeightKg,
    this.targetWeightKg,
  });

  final List<FoodEntry> entries;
  final FoodMacros totalMacros;
  final double calorieGoal;
  final int stepGoal;
  final UserGoalType goalType;
  final double? currentWeightKg;
  final double? targetWeightKg;
}

```

```
class FitnessAiAgent {
    FitnessAiAgent({AiChatClient? client})
        : _client = client ?? PollinationsAiClient.fromEnvironment();
```

```
    final AiChatClient _client;
```

```
    static const systemPrompt = "
```

```
    Ты фитнес-АИ агент Liga Gym.
```

```
    Ты обязательный помощник только по питанию, продуктам, калориям,
    БЖУ, тренировкам, упражнениям, восстановлению и фи...
```

```
    Если пользователь спрашивает не по теме фитнеса, питания или
    тренировок, отвечай коротко: "Я не знаю: это не входит..."
```

```
    Если пользователь пишет, что он ел, оцени еду мягко и практично: что
    хорошо, что не очень, почему, и какой следующие...
```

```
    Если пользователь спрашивает, какие тренировки делать или не делать,
    учитывай цель, текущую активность и безопасность.
```

```
    Не давай медицинские диагнозы и не назначай лечение. При боли, травмах,
    беременности, хронических заболеваниях или ...
```

```
    ";
```

```
    Future<String> answer({
        required String question,
        required FitnessAiContext context,
        required bool hasFitnessHistory,
        required String languageCode,
        List<AiCoachMessage> history = const [],
    }) async {
        final normalized = question.trim().toLowerCase();
        final isRu = languageCode == 'ru';
```

```

if (normalized.isEmpty) {
    return isRu
        ? 'Спросите про питание, продукт или тренировку.'
        : 'Ask me about nutrition, a food choice, or training.';
}

```

```

final    isFollowUp    =    _isFollowUpQuestion(normalized)    &&
hasFitnessHistory;

```

```

if (!_isFitnessQuestion(normalized) && !isFollowUp) {
    return _outOfScope(isRu);
}

```

```

try {
    return await _client.complete(
        messages: _buildMessages(
            question: question,
            context: context,
            history: history,
            languageCode: languageCode,
        ),
    );
} on AiProviderException catch (error) {
    return _providerUnavailableMessage(error.message, isRu);
} on Object {
    return _providerUnavailableMessage('network error', isRu);
}
}

```

```

List<AiApiMessage> _buildMessages({
    required String question,
    required FitnessAiContext context,
    required List<AiCoachMessage> history,
    required String languageCode,
}) {
    final isRu = languageCode == 'ru';
    final messages = <AiApiMessage>[
        const AiApiMessage(role: 'system', content: systemPrompt),
        AiApiMessage(role: 'system', content: _contextPrompt(context, isRu)),
    ];
    final recentHistory = history.length > 8
        ? history.sublist(history.length - 8)
        : history;

    for (final message in recentHistory) {
        messages.add(
            AiApiMessage(
                role: message.isUser ? 'user' : 'assistant',
                content: message.text,
            ),
        );
    }

    messages.add(AiApiMessage(role: 'user', content: question));
    return messages;
}

String _contextPrompt(FitnessAiContext context, bool isRu) {

```

```

final macros = context.totalMacros;
final goal = switch (context.goalType) {
  UserGoalType.loseWeight => isRu ? 'снижение веса' : 'fat loss',
  UserGoalType.maintainWeight => isRu ? 'поддержание веса' :
'maintenance',
  UserGoalType.gainWeight => isRu ? 'набор массы' : 'muscle gain',
};
final foods = context.entries
  .take(8)
  .map((entry) => entry.localizedName(isRu ? 'ru' : 'en'))
  .where((name) => name.trim().isNotEmpty)
  .join(', ');

if (isRu) {
  return 'Контекст пользователя Liga Gym: цель - $goal; дневная цель
калорий - ${context.calorieGoal.toStringAs...
}

  return 'Liga Gym user context: goal - $goal; daily calorie target -
${context.calorieGoal.toStringAsFixed(0)} k...
}

bool _isFitnessQuestion(String value) {
  const keywords = <String>[
    'еда',
    'ел',
    'ела',
    'съел',
    'съела',

```

'ем',
'есть',
'питание',
'рацион',
'калор',
'ккал',
'бжу',
'белок',
'жир',
'углевод',
'продукт',
'завтрак',
'обед',
'ужин',
'перекус',
'сладк',
'пицца',
'бургер',
'газиров',
'тренинг',
'упражнения',
'зал',
'спорт',
'фитнес',
'кардио',
'силов',
'бег',
'ходьба',
'шаг',

'вес',
'похуд',
'набрать',
'масса',
'мышц',
'восстанов',
'растяж',
'nutrition',
'food',
'meal',
'calorie',
'protein',
'carb',
'workout',
'training',
'exercise',
'fitness',
'cardio',
'strength',
'running',
'walking',
'weight',
'muscle',
'recovery',
'who are you',
];

return keywords.any(value.contains) ||

RegExp(r'\b(eat|ate|fat)\b').hasMatch(value);

					40.X-1441-2026 09.02.07 ДП-ПЗ	Лист
						179
Изм.	Лист	№ докум.	Подпись	Дата		

```
}
```

```
bool _isWhyQuestion(String value) {  
    return value.contains('почему') ||  
           value.contains('зачем') ||  
           value.contains('why') ||  
           value.contains('объясни');  
}
```

```
bool _isFollowUpQuestion(String value) {  
    return _isWhyQuestion(value) ||  
           value.contains('а что лучше') ||  
           value.contains('что именно') ||  
           value.contains('как исправить') ||  
           value.contains('а если');  
}
```

```
String _outOfScope(bool isRu) {  
    return isRu  
        ? 'Я не знаю: это не входит в мои знания. Я могу помогать только с  
питанием и тренировками.'  
        : 'I do not know: this is outside my knowledge. I can only help with nutrition  
and training.';  
}
```

```
String _providerUnavailableMessage(String message, bool isRu) {  
    final detail = message.trim().isEmpty ? 'unknown error' : message.trim();  
    return isRu
```

? 'Нейросеть сейчас не ответила: \$detail. Бесплатный провайдер может быть занят. Для стабильной работы можн...

: 'The neural model did not respond: \$detail. The free provider may be busy. For stable AI, run the app wit...

}

}

```
class AiApiMessage {  
    const AiApiMessage({required this.role, required this.content});
```

```
    final String role;
```

```
    final String content;
```

```
    Map<String, String> toJson() => <String, String>{
```

```
        'role': role,
```

```
        'content': content,
```

```
    };
```

```
}
```

```
abstract class AiChatClient {
```

```
    Future<String> complete({required List<AiApiMessage> messages});
```

```
}
```

```
class PollinationsAiClient implements AiChatClient {
```

```
    PollinationsAiClient({
```

```
        required this.chatUrl,
```

```
        required this.model,
```

```
        required this.apiKey,
```

```
        http.Client? httpClient,
```

```

    }) : _httpClient = httpClient ?? http.Client();

    factory PollinationsAiClient.fromEnvironment() {
      const openAiApiKey = String.fromEnvironment('OPENAI_API_KEY');
      const pollinationsApiKey = String.fromEnvironment('POLLINATIONS_API_KEY');
      const configuredChatUrl = String.fromEnvironment('AI_CHAT_URL');
      const configuredModel = String.fromEnvironment('AI_MODEL');
      final apiKey = openAiApiKey.isNotEmpty ? openAiApiKey : pollinationsApiKey;
      final defaultChatUrl = openAiApiKey.isNotEmpty
        ? 'https://api.openai.com/v1/chat/completions'
        : pollinationsApiKey.isNotEmpty
        ? 'https://gen.pollinations.ai/v1/chat/completions'
        : 'https://text.pollinations.ai/openai';
      final defaultModel = openAiApiKey.isNotEmpty ? 'gpt-4.1-mini' : 'openai';

      return PollinationsAiClient(
        chatUrl: Uri.parse(
          configuredChatUrl.isEmpty ? defaultChatUrl : configuredChatUrl,
        ),
        model: configuredModel.isEmpty ? defaultModel : configuredModel,
        apiKey: apiKey,
      );
    }

    final Uri chatUrl;
    final String model;
    final String apiKey;

```

```

final http.Client _httpClient;

static const int _maxTextFallbackUrlLength = 7600;

bool get _supportsTextFallback =>
    chatUrl.host == 'text.pollinations.ai' ||
    chatUrl.host == 'gen.pollinations.ai';

@override
Future<String> complete({required List<AiApiMessage> messages}) async {
    final modelsToTry = _modelsToTry(model);
    AiProviderException? lastError;

    for (final candidateModel in modelsToTry) {
        try {
            return await _completeWithModel(
                messages: messages,
                candidateModel: candidateModel,
            );
        } on AiProviderException catch (error) {
            lastError = error;
            if (!_isRetryable(error.message)) {
                rethrow;
            }
        }
    }

    if (_supportsTextFallback) {
        try {
            return await _completeWithTextEndpoint(messages: messages);
        }
    }
}

```

```

    } on AiProviderException catch (error) {
        lastError = error;
    }
}

throw lastError ?? const AiProviderException('AI model is not configured');
}

Future<String> _completeWithModel({
    required List<AiApiMessage> messages,
    required String candidateModel,
}) async {
    final headers = <String, String>{
        'Content-Type': 'application/json',
        'Accept': 'application/json',
    };
    if (apiKey.isNotEmpty) {
        headers['Authorization'] = 'Bearer $apiKey';
    }

    final http.Response response;
    try {
        response = await _httpClient
            .post(
                chatUrl,
                headers: headers,
                body: jsonEncode(<String, Object?>{
                    'model': candidateModel,
                    'messages': messages.map((message) => message.toJson()).toList(),

```

```

        'temperature': 0.35,
        'top_p': 0.8,
        'max_tokens': 700,
    }},
)
.timeout(const Duration(seconds: 45));
} on TimeoutException {
    throw const AiProviderException('timeout');
}

if (response.statusCode < 200 || response.statusCode >= 300) {
    throw AiProviderException(_errorMessage(response));
}

final content = _extractContent(utf8.decode(response.bodyBytes)).trim();

if (content.isEmpty) {
    throw const AiProviderException('empty completion response');
}
if (_isProviderQueueMessage(content)) {
    throw AiProviderException(content);
}

return content;
}

Future<String> _completeWithTextEndpoint({
    required List<AiApiMessage> messages,
}) async {

```

```

final query = <String, String>{'model': 'openai', 'json': 'false'};
final usesGenApi = chatUrl.host == 'gen.pollinations.ai';
if (apiKey.isNotEmpty && !usesGenApi) {
    query['token'] = apiKey;
}
final fallbackUrl = _buildTextFallbackUrl(
    messages: messages,
    usesGenApi: usesGenApi,
    query: query,
);
final headers = <String, String>{'Accept': 'text/plain'};
if (apiKey.isNotEmpty) {
    headers['Authorization'] = 'Bearer $apiKey';
}
final http.Response response;
try {
    response = await _httpClient
        .get(fallbackUrl, headers: headers)
        .timeout(const Duration(seconds: 45));
} on TimeoutException {
    throw const AiProviderException('timeout');
}

if (response.statusCode < 200 || response.statusCode >= 300) {
    throw AiProviderException(_errorMessage(response));
}

final content = utf8.decode(response.bodyBytes).trim();
if (content.isEmpty) {

```

```

        throw const AiProviderException('empty text response');
    }
    if (_isProviderQueueMessage(content)) {
        throw AiProviderException(content);
    }

    return content;
}

Uri _buildTextFallbackUrl({
    required List<AiApiMessage> messages,
    required bool usesGenApi,
    required Map<String, String> query,
}) {
    final compactPrompt = _textFallbackPrompt(messages);
    final compactUrl = _textFallbackUrl(
        prompt: compactPrompt,
        usesGenApi: usesGenApi,
        query: query,
    );
    if (compactUrl.toString().length <= _maxTextFallbackUrlLength) {
        return compactUrl;
    }

    final shortestPrompt = _textFallbackPrompt(
        messages,
        includeContext: false,
        maxUserChars: 360,
        maxAssistantChars: 0,

```

```

);
final shortestUrl = _textFallbackUrl(
    prompt: shortestPrompt,
    usesGenApi: usesGenApi,
    query: query,
);
if (shortestUrl.toString().length <= _maxTextFallbackUrlLength) {
    return shortestUrl;
}

throw const AiProviderException('text fallback prompt too long');
}

Uri _textFallbackUrl({
    required String prompt,
    required bool usesGenApi,
    required Map<String, String> query,
}) {
    final fallbackBaseUrl = usesGenApi
        ? 'https://gen.pollinations.ai/text/${Uri.encodeComponent(prompt)}'
        : 'https://text.pollinations.ai/${Uri.encodeComponent(prompt)}';
    return Uri.parse(fallbackBaseUrl).replace(queryParameters: query);
}

String _extractContent(String responseBody) {
    final trimmed = responseBody.trim();
    if (trimmed.isEmpty) {
        return "";
    }
}

```

```

Object? payload;
try {
    payload = jsonDecode(trimmed);
} on Object {
    return trimmed;
}

if (payload is String) {
    return payload;
}

if (payload is! Map) {
    return trimmed;
}

final directContent =
    payload['content'] ?? payload['text'] ?? payload['response'];
final direct = _contentToText(directContent);
if (direct.trim().isEmpty) {
    return direct;
}

final choices = payload['choices'];
if (choices is List && choices.isNotEmpty) {
    final firstChoice = choices.first;
    if (firstChoice is Map) {
        final message = firstChoice['message'];
        if (message is Map) {
            final content = _contentToText(message['content']);

```

```

        if (content.trim().isEmpty) {
            return content;
        }
    }

    final text = _contentToText(firstChoice['text']);
    if (text.trim().isEmpty) {
        return text;
    }
}

return ";
}

String _contentToText(Object? value) {
    if (value == null) {
        return ";
    }
    if (value is String) {
        return value;
    }
    if (value is List) {
        return value
            .map((item) {
                if (item is String) {
                    return item;
                }
                if (item is Map) {

```

```

        return item['text'] ?? item['content'] ?? "";
    }
    return "";
})

.where((part) => part.toString().trim().isNotEmpty)
.join('\n');
}

return value.toString();
}

```

```

String _textFallbackPrompt(
    List<AiApiMessage> messages, {
    bool includeContext = true,
    int maxUserChars = 900,
    int maxAssistantChars = 360,
}) {
    final lastUser = messages.lastWhere(
        (message) => message.role == 'user',
        orElse: () => messages.last,
    );
    final contextMessages = messages
        .where((message) => message.role == 'system')
        .skip(1)
        .map((message) => message.content)
        .where((content) => content.trim().isNotEmpty)
        .toList(growable: false);
    final previousAnswers = messages
        .where((message) => message.role == 'assistant')

```

```

        .map((message) => message.content)
        .where((content) => content.trim().isEmpty)
        .toList(growable: false);
final previousAssistant = previousAnswers.isEmpty
    ? null
    : previousAnswers.last;

final buffer = StringBuffer()
    ..writeln(
        'You are Liga Gym AI. Answer only about nutrition, workouts, recovery,
and fitness goals. Be brief and prac...
    );
if (includeContext && contextMessages.isNotEmpty) {
    buffer.writeln('Context: ${_ellipsize(contextMessages.last, 520)}');
}
if (maxAssistantChars > 0 && previousAssistant != null) {
    buffer.writeln(
        'Previous answer: ${_ellipsize(previousAssistant, maxAssistantChars)}',
    );
}
buffer
    ..writeln('User: ${_ellipsize(lastUser.content, maxUserChars)}')
    ..write('Assistant:');

return buffer.toString();
}

String _ellipsize(String value, int maxChars) {
    final normalized = value.replaceAll(RegExp(r'\s+'), ' ').trim();

```

```

if (normalized.length <= maxChars) {
    return normalized;
}

return '${normalized.substring(0, maxChars).trimRight()}...';
}

```

```

List<String> _modelsToTry(String configuredModel) {
    final normalizedModel = _normalizeModel(configuredModel);
    final isPollinations =
        chatUrl.host == 'text.pollinations.ai' ||
        chatUrl.host == 'gen.pollinations.ai';
    if (!isPollinations) {
        return normalizedModel.isEmpty
            ? const <String>[]
            : <String>[normalizedModel];
    }

    return <String>{
        normalizedModel,
        'openai',
        'openai-fast',
    }.where((model) => model.isNotEmpty).toList(growable: false);
}

```

```

String _normalizeModel(String value) {
    final normalized = value.trim().toLowerCase();
    if (normalized == 'openaiflutter') {
        return 'openai';
    }
}

```

```

    }
    if (normalized.endsWith('flutter')) {
        return normalized.substring(0, normalized.length - 'flutter'.length);
    }

    return normalized;
}

bool _isRetryable(String message) {
    final normalized = message.toLowerCase();
    return normalized.contains('model not found') ||
        normalized.contains('empty completion') ||
        normalized.contains('empty text') ||
        normalized.contains('429') ||
        normalized.contains('500') ||
        normalized.contains('502') ||
        normalized.contains('503') ||
        normalized.contains('504') ||
        normalized.contains('414') ||
        normalized.contains('already queued') ||
        normalized.contains('already in queue') ||
        normalized.contains('queue') ||
        normalized.contains('queued') ||
        normalized.contains('uri too long') ||
        normalized.contains('prompt too long') ||
        normalized.contains('rate limit') ||
        normalized.contains('too many requests') ||
        normalized.contains('timeout');
}

```

```

bool _isProviderQueueMessage(String value) {
    final normalized = value.trim().toLowerCase();
    return normalized.contains('already queued') ||
        normalized.contains('already in queue') ||
        normalized.contains('queue is full') ||
        normalized.contains('too many requests') ||
        normalized.contains('rate limit');
}

```

```

String _errorMessage(http.Response response) {
    final body = utf8.decode(response.bodyBytes);
    if (body.trim().isEmpty) {
        return 'HTTP ${response.statusCode}';
    }
}

```

```

try {
    final payload = jsonDecode(body);
    if (payload is Map) {
        final message =
            payload['message'] ??
            payload['error_description'] ??
            payload['error'];
        if (message != null) {
            return 'HTTP ${response.statusCode}: $message';
        }
    }
} on Object {
    // Keep the raw body below.
}

```

```

    }

    return 'HTTP ${response.statusCode}: $body';
  }
}

class AiProviderException implements Exception {
  const AiProviderException(this.message);

  final String message;
}

```

A.14 Сообщения чата

Файл: lib/features/social/domain/entities/chat_message.dart

```

import 'package:cloud_firestore/cloud_firestore.dart';

class ChatMessage {
  const ChatMessage({
    required this.id,
    required this.senderId,
    required this.senderName,
    required this.senderEmail,
    this.senderCity,
    required this.message,
    required this.sentAt,
    this.type = ChatMessageType.text,
    this.sharedResult,
    this.sharedExercise,

```

```
});
```

```
factory ChatMessage.fromFirestore(
    QueryDocumentSnapshot<Map<String, dynamic>> document,
) {
    final data = document.data();
    final timestamp = data['sentAt'] as Timestamp?;

    return ChatMessage(
        id: document.id,
        senderId: data['senderId'] as String? ?? "",
        senderName: data['senderName'] as String? ?? 'Athlete',
        senderEmail: data['senderEmail'] as String? ?? "",
        senderCity: (data['senderCity'] as String?)?.trim(),
        message: data['message'] as String? ?? "",
        sentAt: timestamp?.toDate() ?? DateTime.now(),
        type: ChatMessageType.fromName(data['type'] as String?),
        sharedResult: SharedChatResult.fromMap(data['sharedResult']),
        sharedExercise: SharedChatExercise.tryFromMap(data['sharedExercise']),
    );
}
```

```
final String id;
final String senderId;
final String senderName;
final String senderEmail;
final String? senderCity;
final String message;
final DateTime sentAt;
```

```

final ChatMessageType type;
final SharedChatResult? sharedResult;
final SharedChatExercise? sharedExercise;
}

enum ChatMessageType {
    text,
    dailyResult,
    sharedExercise;

    static ChatMessageType fromName(String? value) {
        return switch (value) {
            'daily_result' => ChatMessageType.dailyResult,
            'shared_exercise' => ChatMessageType.sharedExercise,
            _ => ChatMessageType.text,
        };
    }

    String get firestoreName {
        return switch (this) {
            ChatMessageType.text => 'text',
            ChatMessageType.dailyResult => 'daily_result',
            ChatMessageType.sharedExercise => 'shared_exercise',
        };
    }
}

class SharedChatResult {
    const SharedChatResult({

```

```

required this.title,
required this.date,
required this.steps,
required this.caloriesBurned,
required this.caloriesConsumed,
required this.workoutsCount,
required this.workoutMinutes,
required this.progressPercent,
required this.proteins,
required this.fats,
required this.carbs,
});

```

```

factory SharedChatResult.fromMap(Object? raw) {
  final data = raw is Map ? raw : const <Object?, Object?>{};
  final timestamp = data['date'] as Timestamp?;

```

```

return SharedChatResult(
  title: data['title'] as String? ?? "",
  date: timestamp?.toDate() ?? DateTime.now(),
  steps: (data['steps'] as num?)?.toInt() ?? 0,
  caloriesBurned: (data['caloriesBurned'] as num?)?.toDouble() ?? 0,
  caloriesConsumed: (data['caloriesConsumed'] as num?)?.toDouble() ?? 0,
  workoutsCount: (data['workoutsCount'] as num?)?.toInt() ?? 0,
  workoutMinutes: (data['workoutMinutes'] as num?)?.toInt() ?? 0,
  progressPercent: (data['progressPercent'] as num?)?.toInt() ?? 0,
  proteins: (data['proteins'] as num?)?.toDouble() ?? 0,
  fats: (data['fats'] as num?)?.toDouble() ?? 0,
  carbs: (data['carbs'] as num?)?.toDouble() ?? 0,

```

```
);  
}
```

```
final String title;  
final DateTime date;  
final int steps;  
final double caloriesBurned;  
final double caloriesConsumed;  
final int workoutsCount;  
final int workoutMinutes;  
final int progressPercent;  
final double proteins;  
final double fats;  
final double carbs;  
}
```

```
class SharedChatExercise {  
    const SharedChatExercise({  
        required this.sourceExerciseId,  
        required this.title,  
        required this.description,  
        required this.muscleGroups,  
        required this.equipment,  
        required this.techniqueText,  
        required this.defaultCategory,  
        required this.customCategory,  
        required this.avatarDataUrl,  
        required this.iconName,  
        required this.photoDataUrls,  
    });  
}
```

```
});
```

```
static SharedChatExercise? tryFromMap(Object? raw) {  
  if (raw is! Map) {  
    return null;  
  }  
}
```

```
return SharedChatExercise(  
  sourceExerciseId: raw['sourceExerciseId'] as String? ?? "",  
  title: raw['title'] as String? ?? "",  
  description: raw['description'] as String? ?? "",  
  muscleGroups: raw['muscleGroups'] as String? ?? "",  
  equipment: raw['equipment'] as String? ?? "",  
  techniqueText: raw['techniqueText'] as String? ?? "",  
  defaultCategory: raw['defaultCategory'] as String? ?? 'strength',  
  customCategory: raw['customCategory'] as String? ?? "",  
  avatarDataUrl: raw['avatarDataUrl'] as String? ?? "",  
  iconName: raw['iconName'] as String? ?? 'dumbbell',  
  photoDataUrls:  
    (raw['photoDataUrls'] as List<Object?>?).whereType<String>().toList(  
      growable: false,  
    ) ??  
    const <String>[],  
);  
}
```

```
final String sourceExerciseId;  
final String title;  
final String description;
```

```

final String muscleGroups;
final String equipment;
final String techniqueText;
final String defaultCategory;
final String customCategory;
final String avatarDataUrl;
final String iconName;
final List<String> photoDataUrls;
}

```

A.15 Тренерские шаблоны тренировок

Файл: lib/features/coach/domain/entities/trainer_workout_template.dart

```

class TrainerWorkoutTemplate {
  const TrainerWorkoutTemplate({
    required this.id,
    required this.trainerId,
    required this.title,
    required this.goal,
    required this.instructions,
    required this.exerciseIds,
    required this.createdAt,
  });

```

```

final String id;
final String trainerId;
final String title;
final String goal;
final String instructions;

```

```
final List<String> exerciseIds;  
final DateTime createdAt;  
}
```

ПРИЛОЖЕНИЕ Д

Фрагменты фактически проверенной структуры проекта Liga Gym

Таблица Д.1 – Проверенные файлы проекта

Проверенный блок	Файл	Подтвержденное содержание
Маршруты экранов	lib/core/navigation/app_routes.dart	Содержит splash, login, register, dashboard, workout, nutrition, steps, social, coach, aiCoach
Профиль	lib/features/auth/data/models/user_profile_model.dart	Поля профиля, цели, роли и социальные счетчики
Тренировки	lib/features/workout/data/models/workout_model.dart	Поля тренировки, маршрут routeJson, упражнения exerciseEntriesJson
Питание	lib/features/nutrition/data/models/food_entry_model.dart	Записи еды, КБЖУ, способ ввода manual/barcode
Шаги	lib/features/steps/data/datasources/step_local_data_source.dart	SQLite-таблицы daily_steps и step_sensor_state
Социальные функции	lib/features/social/data/datasources/social_remote_data_source.dart	Друзья, заявки, лидерборд, чаты, приватность
Тренерский режим	lib/features/coach/data/repositories/coach_repository_impl.dart	Упражнения, рецепты, шаблоны, назначения, Supabase Storage
AI-помощник	lib/features/ai_coach/domain/services/fitness_ai_agent.dart	Контекст дневника питания, целей и ограничение темой фитнеса